전역 조명

리얼타임 렌더링 4판 · 11장 Global Illumination

1차시 : 복습 ~ 일반적인 전역 조명 모델

2차시 : 주변 폐색 (Ambient Occlusion)

3차시 : 방향성 폐색 (Directional Occlusion) ,확산 전역 조명

4차시 : 반사 전역 조명, 기타 & 최신 엔진 UseCase

오늘의 순서 - 1차시

00 서론 — 컴퓨터 그래픽스처럼 보이지 않기

01 복습, GI와는 무엇이 다른가 — 7·9·10장

02 렌더링 방정식과 전역 조명 알고리즘 — 11.1

03 일반적인 전역 조명 알고리즘 — 11.2

서론

“컴퓨터 그래픽스처럼 보이면, 좋은 컴퓨터 그래픽스가 아니다.”

— 제레미 번, 『디지털 라이팅 & 렌더링』

CG를 다루는 고전적인 교과서 포지션의 서적에서 인용한 문구라고 하고
프로그래밍지식보다는 그냥 사실적인 조명, 그림자, 텍스쳐, 라이팅, 렌더링 기법을 다루는 책이라고 합니다.

3D 아티스트를 위한 지식과 TA, 그리고 비주얼 프로그래머에게 도움이 되는 서적으로 보이는 내용들이 많이 있음
(물론 게임에서 사용하는 지식은 일부가 될 것 같습니다. 아 슬프구나)

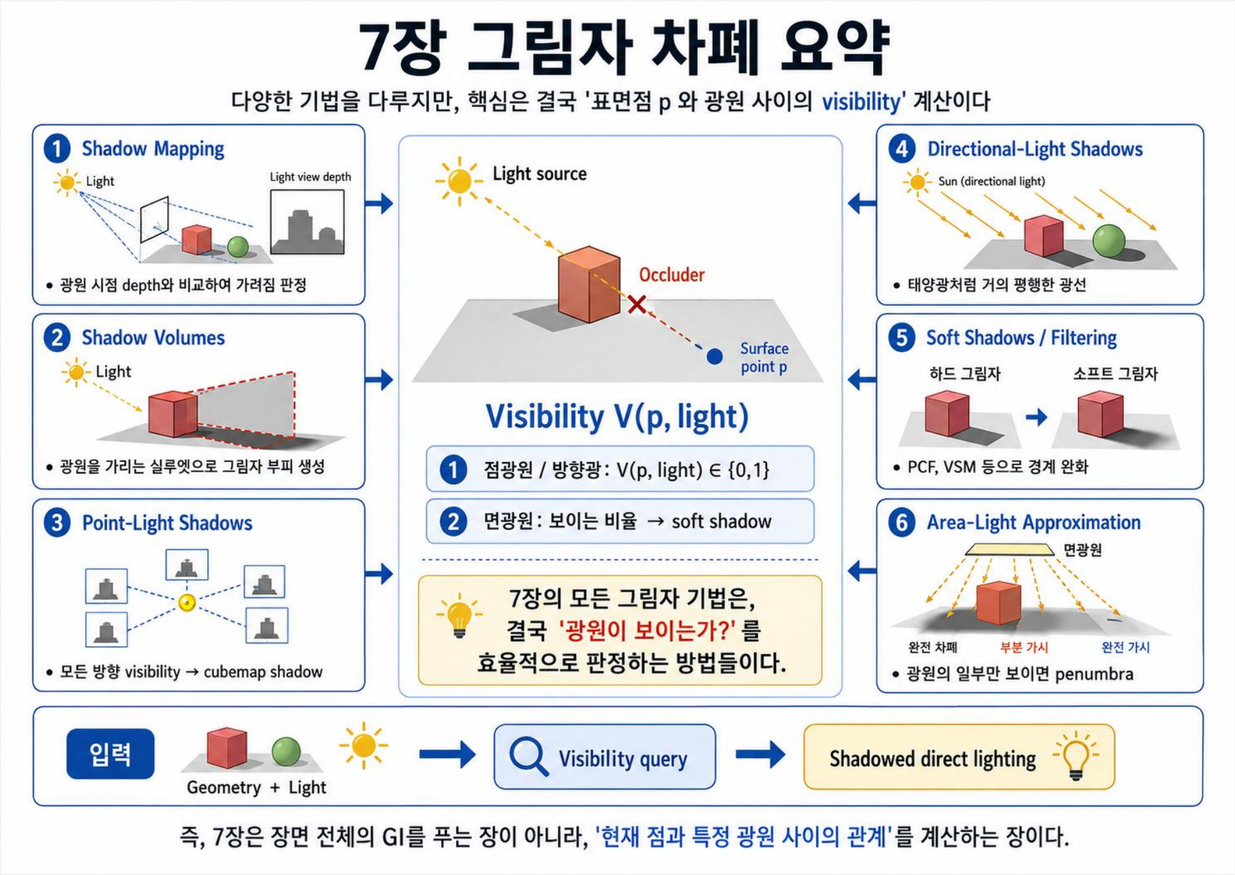
리얼타임 렌더링 4판 · 11장 전역 조명 · 1차시

복습 — 7·9·10

본격적인 GI파트를 다루기 전에
그림자, PBR, Local Illumination 파트에서 다룬 내용과의 연관점, 차이점을 먼저 확인하고
앞으로 보게될 GI파트에서 해결하고자 하는 문제가 무엇인지 확인합시다.

7장 • 그림자

Shadow Map, Shadow Volume등으로 다른 오브젝트가 드리우는 그림자를 계산했지만,
이때 계산한 그림자는 직접광에 대한 가시성만을 판단하는 그림자였습니다.



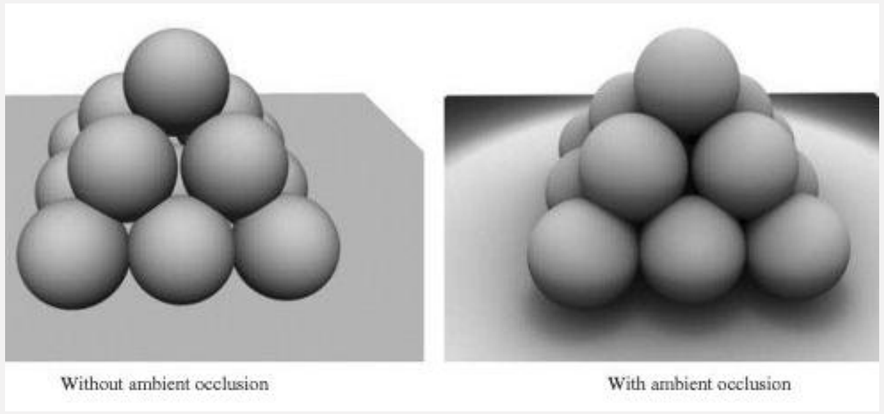
고마워요 GPT!

7장 • 현실의 빛과 어긋나는 지점

7장에서 계산한 것

지정한 광원 방향의 가시성

광원은 점(Point)거나 방향(Directional)이였고,
판정은 이진 값으로 처리되었습니다 (가려짐/안가려짐)
광원이 여러 개일 경우, 각 광원마다의 가시성을 판단했습니다.



광원의 위치 (위쪽) 기준으로 직접광의 차폐만을 고려했을 때와 간접광의 차폐를 비교한 모습

실제로 일어나는 일

다른 물체가 반사하는 빛도 차폐 될 수 있다

현실세계는 직접적인 광원 (Light Source)외에도, 대부분의 물체가 빛을 반사합니다.
그렇기 때문에, 동일하게 직접광을 받는 상황에도 직접광이 없는 방향의 차폐가
그림자를 만들게 됩니다.

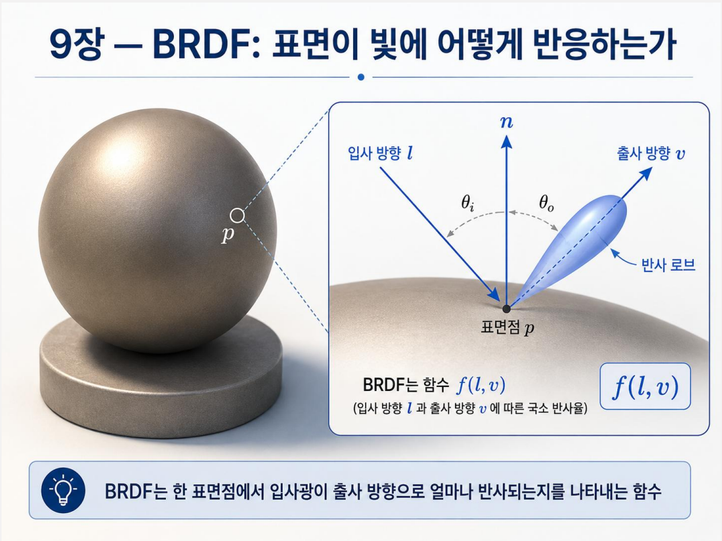
9장 · BRDF

양방향 반사도 분포 함수

$f(l, v)$

방향 l에서 들어온 빛이 방향 v로 얼마나 나가는가에 대한 함수
재질에 따라 특수한 함수를 사용하거나, 미세 표면을 고려하는 등의 내용이 있었지만
재질과 입사/출사 방향이 동일한 경우 언제나 같은 값을 냄.

입사 방향 l에서 광량이 얼마나 들어오는지에 대해 다룬건 아니고, 단순히 재질(material)의 특성을 다룬 파트라고 이해



9장 · BRDF

반사율 방정식 · 9장

$$L_o(p, v) = \int_{l \in \Omega} f(l, v)\, L_i(p, l)\, (n \cdot l)^+ dl$$

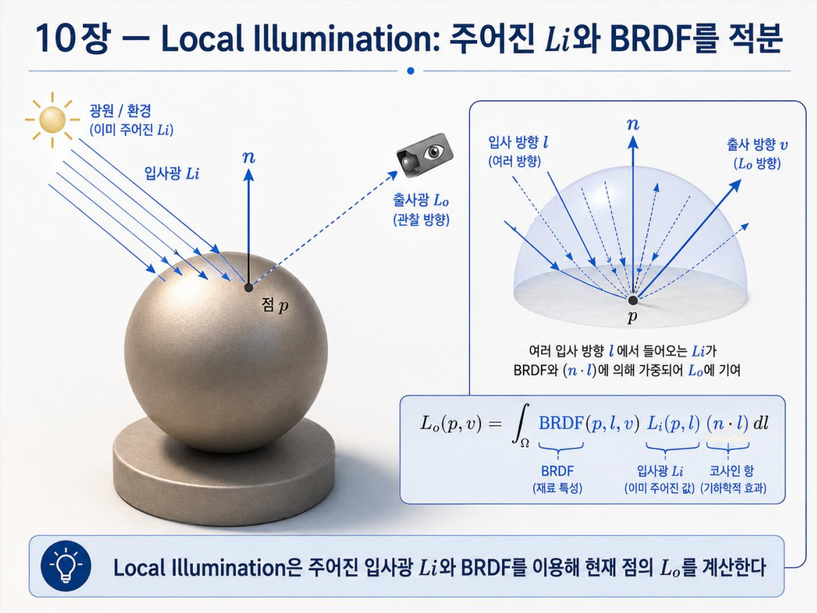
9장에서 다룬 항은 반사율 방정식을 기준으로 위에서 푸른빛으로 강조한 항이 됩니다.
빛이 어디서 들어오는지에 대해서는 다루지 않았습니다.

10장 · Local Illumination

10장에서 다룬 것은 조명 항

$$L_o(p, v) = \int_{l \in \Omega} f(l, v)\, L_i(p, l)\, (n \cdot l)^+ dl$$

10장은 L_i를 알고 있는 값 (프로그램기준으로는 알 수 있는 값)으로 놓고 반사율 방정식을 푸는 방법에 대해 다루었습니다.
점광원/방향광만을 고려하는 경우에는 적분은 유한한 합으로 나눌 수 있고
환경 조명인 경우에는 미리 적분해둔 텍스쳐 (또는 SH등)를 읽어 옵니다.
L_i가 유한하거나, 장면의 변화/상태와 무관하게 주어질 수 있다는 가정이 지역 조명의 핵심 구분 방법입니다.



10장 • 환경 조명이 저장한 것

환경 맵

무한히 먼 곳에서 오는 입사 광도를 방향의 함수로 저장합니다.
위치는 인자가 아닙니다.

방사 조도 맵

코사인 가중 적분을 미리 끝내 둔 확산 반사 전용 조명입니다.
실행 시에는 법선으로 한 번 조회합니다.

구면 조화

Image-Based-Light 대신 낮은 주파수 조명을 아홉 개 남짓한 계수로 압축합니다.
저장과 보간이 싸고 회전도 쉽습니다.

셋 다 “빛이 어디서 오는가”를 장면과 무관한 함수 로 고정합니다.
물체가 움직여도, 옆에 빨간 벽을 세워도 이 함수는 변하지 않습니다.

11장 · 다룰 내용

항목	지역 조명 모델의 가정	실제 장면
광원까지의 거리	무한히 멀다 — 방향만 남는다	유한하고, 위치마다 방향이 다르다
물체 간 차폐	지정한 광원 방향에만 그림자	모든 입사 방향에서 간접광이 차폐 될 수 있음
물체 간 상호 반사	빛은 물체 표면에서 한 번만 튕긴다	벽의 색이 바닥으로 번진다 (간접광)
조명	조명은 고정된 텍스처거나 punctual light	물체가 움직이면 조명도 함께 바뀐다

렌더링 방정식과 전역 조명

9, 10장에서 다루지 않은 렌더링 방정식과 전역 조명에서 필요한 광선 투사함수에 대해 다룬다는 내용 다듬어서 쓰기

11.1 The Rendering Equation

반사율 방정식에서 렌더링 방정식으로

9·10장 — 반사율 방정식

$$L_o(p, v) = \int_{l \in \Omega} f(l, v)\, L_i(p, l)\, (n \cdot l)^+ dl$$

11장 — 렌더링 방정식

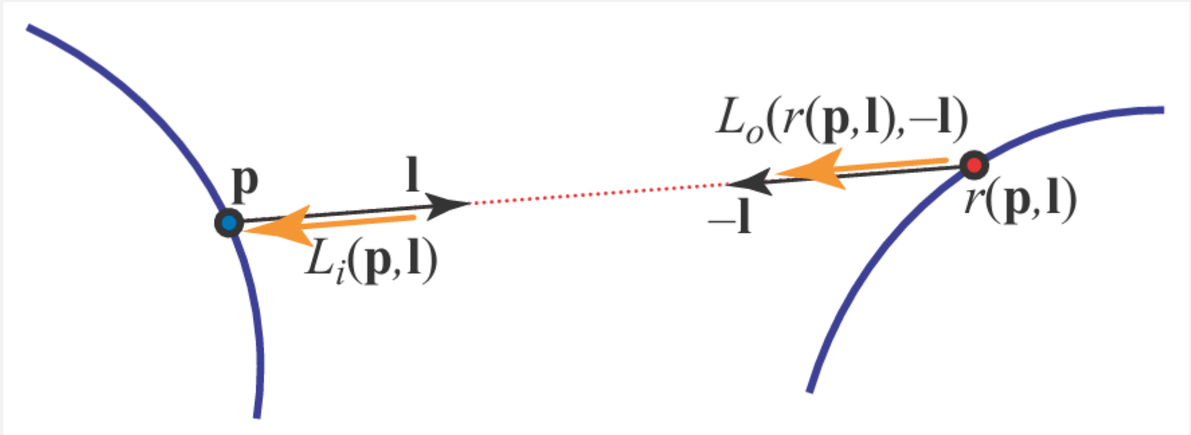
$$L_o(p, v) = L_e(p, v) + \int_{l \in \Omega} f(l, v)\, L_o(r(p, l), -l)\, (n \cdot l)^+ dl$$

9장과 10장에서 다룬 반사율 방정식은, 1986년 Kajiya가 제시한 렌더링 수식의 제한된 특수사례.
빛이 들어와서 반사한다고 했지만, 현실의 물체는 스스로 빛을 내는 경우도 있으며 (빨간 항)
물체에 들어오는 빛은 어디 허공에서 생성되는게 아니라 어딘가에서 방출되어 옵니다. (파란항)



빨간 항의 예시 : 스스로 빛을 내는 야광공룡 이건 못참지

광선 투사 함수 $r(p, l)$



r(p, l)은 점 p에서 방향 l로 광선을 쏘았을때 가장 먼저 만나는 표면 위의 점을 돌려줍니다.
기존의 반사방정식에서 다룬 빛이, 어느 지점에서 얼마나 오는지 알기 위해서는 r(p, l)지점을 찾아야겠죠?

r(p, l)지점에서 p방향으로 오는 빛(Li)은 Lo(r(p,l),–l) 이 됩니다.

r인이유 : ray casting function입니다

- 반투명한 물체에 대해서는 다루지 않습니다. BRDF함수부터가 빛의 반사를 가정한 함수지 빛의 투과를 고려하지 않았습니다.
- 그렇기 때문에 ray cast하는 방향에서 오는 빛은 언제나 단일 지점임을 보장할 수 있습니다.
- 반투명한 물체는 14장에서 다룰 예정이니 잠시동안은 불투명한 세상을 상상합시다.

렌더링 방정식이 의미하는것

$$L_o(p, v) = L_e(p, v) + \int_{l \in \Omega} f(l, v)\, L_o(r(p, l), -l)\, (n \cdot l)^+ dl$$

$L_e(p, v)$ 는 이미 10장에서 광원의 방출을 다루는 방법을 보았으므로 생략합니다.
그런데 $L_o(p, v)$ 값을 구하려면 반구 전체 방향으로 ray cast를 시도해서 $L_o(r(p, l), -l)$ 를 구해야합니다.

그렇다면 각 L_o 값들을 구하기 위해서 다시 해당 지점들에서 반구 전체 방향으로 ray cast를 시도하고...
모든 빛이 흡수될때까지 재귀적으로 계산해야 합니다

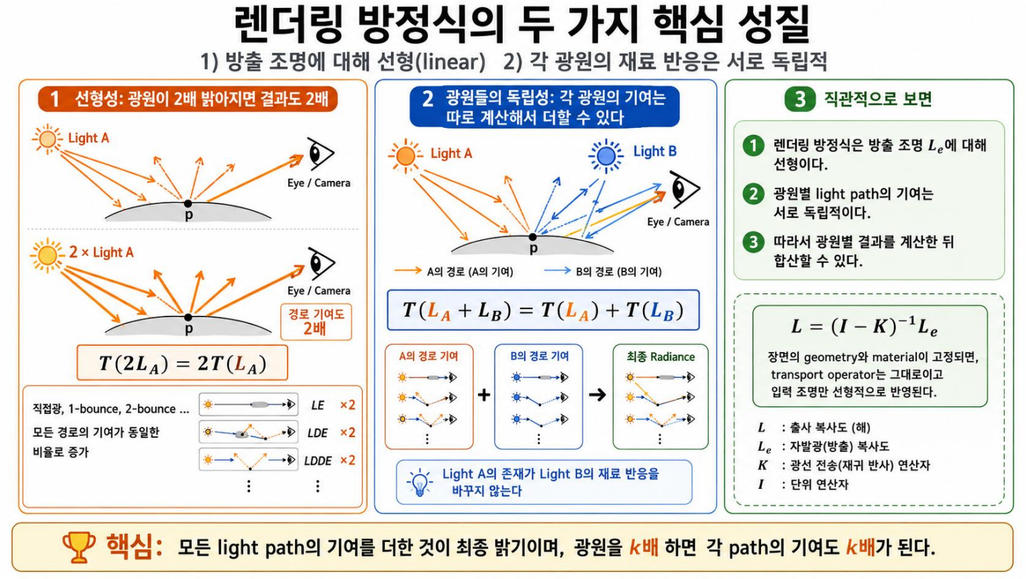
저 L_o 계산하는게 해석적으로 풀리진 않을 것 같은데 아마 없겠죠? AI 피셜로는 제2종 프레드홀듬 적분 방정식이라는데 모르겠음

렌더링 방정식의 성질

모든 광원의 emission이 2배가 되면, 음영 처리 결과가 2배 밝아지고
각 Light Source에 대한 재료의 반응은 다른 Light Source와 독립적입니다.

Light는 기본적으로 Additive하게 작동하기 때문에 (가산혼합)
광원별 결과를 계산 한 뒤 합산할 수 있음을 보일 수 있습니다.

PS 하셨던 분들은 기대값의 선형성 태그라던가, 그런곳에서 본 선형성이라는 단어가 와닿을듯



실시간 렌더링에서 주로 지역 조명만 사용하는 이유

렌더링 방정식을 프레임마다 전부 풀 수는 없다

목표로 하는 픽셀 하나를 렌더링하기 위해 씬의 모든 지점에 대해 BRDF함수를 평가하고 적분하는 것은 실시간 내에 처리하기 어렵습니다.
정밀도를 낮추기 위해 반구를 샘플링 하는 비율을 낮추고, 낮은 광량을 가진 Light Pass계산을 생략하더라도
런타임 (60프레임 기준 16ms)환경에서 작동하기에는 현실적인 제약이 아닙니다.

GI는 전통적인 단일 Rasterization Pass의 Fragment Shader만으로는 완전한 GI를 계산하기 어렵다.

Rasterization 파이프라인의 Fragment Shader는 현재 rasterize된 surface point를 독립적으로 shading하는 데 최적화되어 있습니다.
반면 Global Illumination은 현재 점의 조명을 계산하기 위해 장면의 다른 surface point들의 조명·가시성 정보가 필요합니다.
이러한 지점들은 현재 화면에 fragment를 생성하지 않거나, 같은 pass에서 아직 shading 결과가 존재하지 않을 수 있으므로,
단일 Fragment Shader pass만으로 GI의 surface 간 light transport를 해결하기 어렵습니다.

(지역 조명은 Li가 주어진 값이기 때문에, 대부분의 데이터를 fragment 셰이더에서 알 수 있습니다)

- 그림자 처리도 그렇지만, 위와 같은 이유로 런타임 GI는 기본적으로 DrawCall이 내려지기전, 미리 계산하는 과정을 거치게 됩니다.
- 그게 이미 구성된 씬에서 미리 구워두는 방법이든, 런타임에 Shadow Map처럼 다른 정보를 굽게 되든…

Heckbert의 광자 경로 표기법

기호

L — 광원(light), 경로의 출발점

E — 눈(eye), 경로의 도착점

D — 확산 반사(diffuse)

S — 정반사(specular)

연산자

$k*$ — 0회 이상 반복

$k+$ — 1회 이상 반복

$k?$ — 0회 또는 1회

$(k|k')$ — 둘 중 하나

는 UseCase들을 보고 기호에 매핑했을때 이해가 더 빠를 것 같기 때문에 지금은 넘깁니다.



전역 조명을 실시간에 넣는 두 가지 전략

전략 1

단순화 — 식의 일부에 강한 가정을 건다

조명이 모든 방향에서 균일하다고 보거나, 반사가 확산뿐이라고 보거나, 빛이 멀리 가지 않는다고 보는등의 제약으로 식을 단순화 합니다.

전략 2

사전 계산 — 변하지 않는 것을 미리 푼다

정적인 기하 구조와 정적인 광원 사이의 빛 전달을 미리 풀어 라이트 맵이나 계수로 저장하고, 실행 시에는 조회와 보간만 합니다.

어떻게 하드웨어에 호환되는 기법으로 단순하게 만들거나 미리 구워두지 (또는 둘다 해서 퉁치기) 같은 고민이 요즘 GI기술의 집합체
11.2장에서는 그나마 덜 다루고, 11.3장부터 적극적으로 위 두 주제를 다룰 예정입니다.

최근 GI 기술에는 최근에 풀었던 값을 캐싱해서 사용하는 방법에 대해서도 다루고, 별도의 하드웨어 유닛의 도움을 받아 병목을 해결하기도 합니다.

일반적인 전역 조명 알고리즘

10장에서는 반사율 수식을 푸는 다양한 방법에 중점을 두고, 들어오는 광도 Li의 분포를 가정하며 그것이 음영에 어떻게 영향을 미치는지 분석했었다.
이번 장에서는 방금 설명한 렌더링 수식을 풀도록 설계된 알고리즘 2개를 제시한다.

한 지점에 도달하는 광도는 다른 지점에서 방출되거나, 반사된 광도가 된다.

퀄리티가 굉장하지만, 기존에 설명했듯 런타임에 계산하기에는 무리가 있다.

그럼에도 해당 과정을 다루는 이유는
1. 정적이거나 / 부분적으로 정적인 씬에서 이런 알고리즘으로 사전처리해 나중에 사용할 수 있도록 저장할 수 있기 때문
2. 실시간 솔루션을 설계할때도 올바른 방법이 무엇인지 비교하거나, 유사한 유형의 추론을 적용할 수 있기 때문

특히 유사한 유형의 추론을 적용한다는 부분에 집중해서 일반적인 전역 조명 알고리즘을 보면 되겠다.
해당 파트에서 나오는 개념과, 이를 구현하기 위해 사용하는 아이디어들은 이후 각기 다른 GI 구현체들의 기본 개념이 될 수 있다.

렌더링 수식을 푸는 두가지 일반적인 방법은 유한 요소 해석(방법) 과 몬테 카를로 방법이 있다.
라디오시티(Radiosity)는 유한 요소 해석을, 다양한 레이트레이싱(RayTracing)은 몬테 카를로 방법을 기반에 두고 있다.

- 유한 요소 해석 : 공간을 작은 요소들로 분할하고, 각 요소별 물리 법칙을 식으로 세워 연립방정식을 푸는 형태. 고체, 유체, 열역학 등에서 사용한다고 함
- 몬테 카를로 방법 : 유명한걸로는 원의 면적을 계산하는것이 있음. RayTracing을 몬테 카를로 방법을 사용하면 임의의 방향으로 Ray를 쏘는것을 상상하면 된다

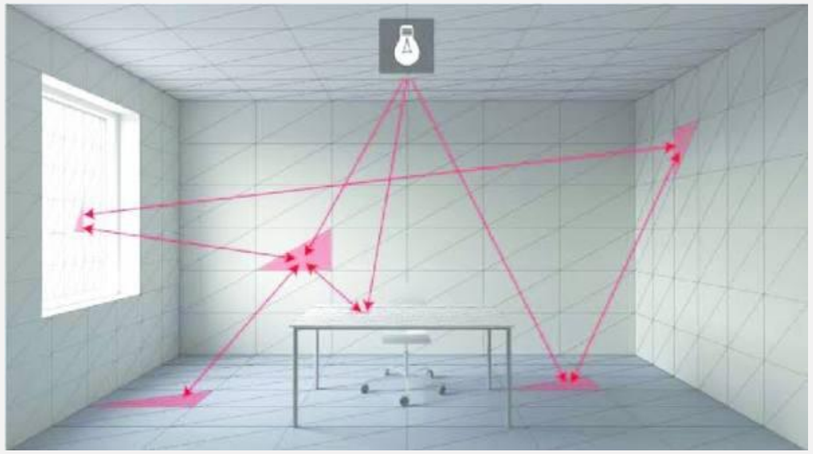
Radiosity

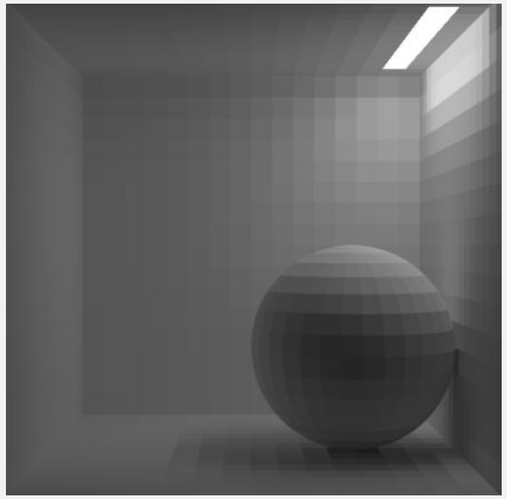
모든 표면이 확산 반사만 한다고 가정하면, 한 점의 밝기는 보는 방향과 무관해집니다. (Lambertian)

방향 의존성이 사라지면 미지수는 방향의 함수가 아니라 위치만의 함수가 됩니다.

장면을 패치로 나누고 패치들 사이의 에너지 교환을 연립방정식으로 풉니다.

열전달 공학에서 넘어온 접근으로, 1984년 코넬대에서 그래픽스에 처음 적용되었습니다.





리얼타임 렌더링 4판 · 11장 전역 조명 · 1차시

Radiosity

패치 i의 방사도

$$B_i = B_{ei} + \rho_i \sum_j F_{ij} B_j$$

F_{ij}는 패치 i를 떠난 에너지 중 패치 j에 도달하는 비율입니다. 재질과 무관한 순수한 기하 문제입니다.

형태 계수 계산이 전체 비용을 지배하고, 패치 수의 제곱에 비례해 늘어납니다.

점진적 세밀화는 가장 밝은 패치부터 순서대로 에너지를 뿌려, 중간 결과를 미리 보여 줍니다.

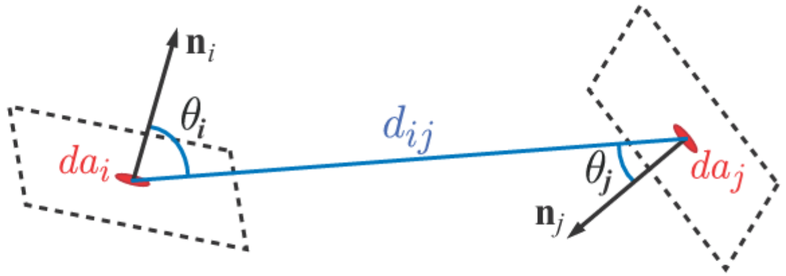
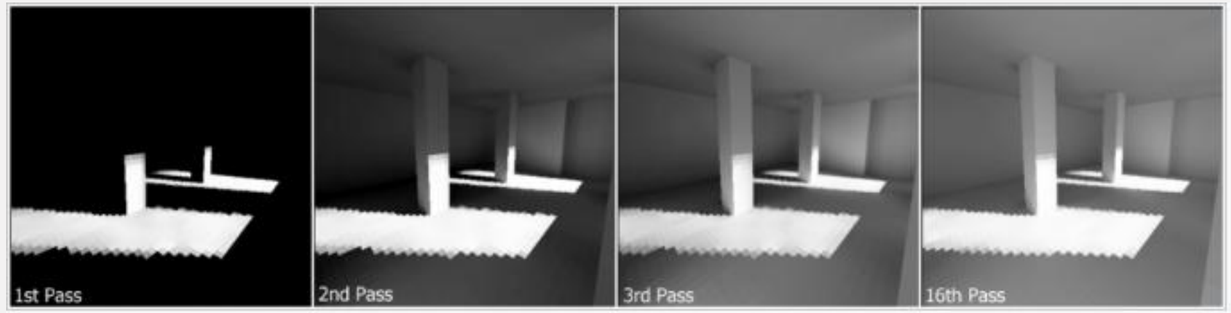


Figure 11.4. The form factor between two surface points.



As the algorithm iterates, light can be seen to flow into the scene, as multiple bounces are computed. Individual patches are visible as squares on the walls and floor.

리얼타임 렌더링 4판 · 11장 전역 조명 · 1차시

Radiosity의 장단점

얻은 것

시점과 무관한 해

한 번 풀어 두면 카메라를 어디에 두어도 그대로 쓸 수 있음.
결과가 정점이나 텍셀에 저장되므로 실행 시 비용이 거의 없음.

잃은 것

정반사와 메시 독립성

방향정보를 손실하고 표면 patch가
받은 빛이 균일하게 반사된다 가정하였기 때문에
거울과 하이라이트를 표현하지 못함 (Diffuse 경로에만 특화)
메시 분할 정도가 품질이라 그림자 경계에서 계단이 드러남.

Ray Tracing

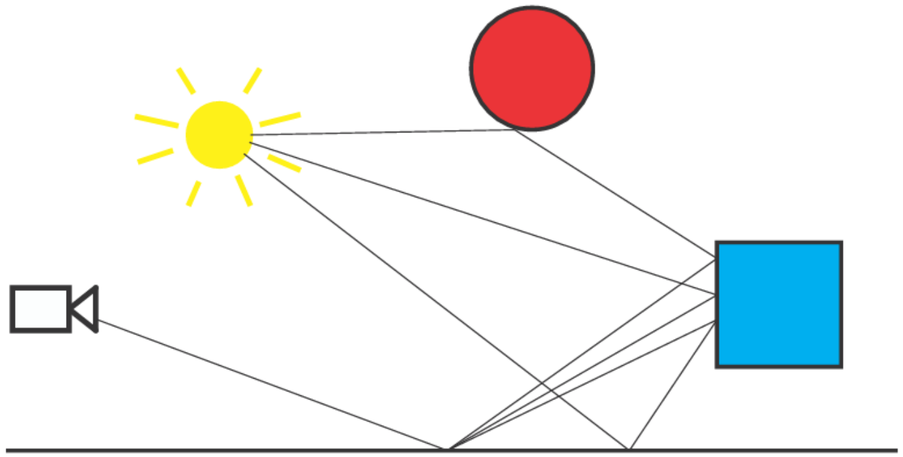


Figure 11.5. Example paths generated by a path tracing algorithm. All three paths pass through the same pixel in the film plane and are used to estimate its brightness. The floor at the bottom of the figure is highly glossy and reflects the rays within a small solid angle. The blue box and red sphere are diffuse and so scatter the rays uniformly around the normal at the point of intersection.

Ray Tracing의 장단점

얻은 것

재질과 상관없이 빛의 반사를 시뮬레이션 할 수 있음

RadioSity는 재질이 가진 glossy함이나, Mirror같은 반사 로브의 특징을 살릴 수 없었으나, Ray Tracing은 BRDF의 형태에 크게 구애받지 않고 사용할 수 있는 기술이 된다.

엄청난 샘플링 비용을 줄이기 위해
대부분의 빛이 방향이 들어오는 방향으로 더 많은 광선을 쏘거나
BRDF 함수의 특성에 따라 샘플링 전략을 변경하는 등의 기술들이 연구되었고, 연구되는 중
이후 GI파트에서도 약간 다루겠지만, 적은량의 샘플링을 어떻게 커버할지도 중요한 주제

잃은 것

샘플링 비용과 노이즈

모든 방향의 Ray를 Trace할 수 없고, trace의 수가 줄어들수록 노이즈의 분산이 커지게 된다.
동일한 방법으로 샘플링한다고 가정했을때 노이즈(표준편차)를 2배 줄이기 위해서는, 4배 더 많은 샘플이 필요하게 된다.

Radiosity의 장점으로 쓴 시점과 무관한 해는 RayTracing의 단점으로 쓰진 않았습니다.
Unity / Unreal에서 static한 오브젝트에 대해 미리 굽는 과정에서는 RayTracing을 사용하고 있고 (거울등의 처리때문에라도), 이때는 광원 (Light Sources)에서 Ray를 쏘아 장면에 반영되는 광량을 계산해, 물체의 표면에 반영합니다.

1차시 마무리

주변 폐색 하다가 어디서 끊어야할지도 모르겠고 준비도 미흡해서 일단 여기서 끊기
앞 파트 발표 시간 얼마나 소요되는지 보고 판단하기

오늘의 순서 - 2차시

00 서론 — 지역 조명은 어디서 티날까?

01 주변 폐색 — 11.3

02 주변 폐색 — 11.3 * 오타 아닙니다. 주변 폐색만 하루종일 합니다

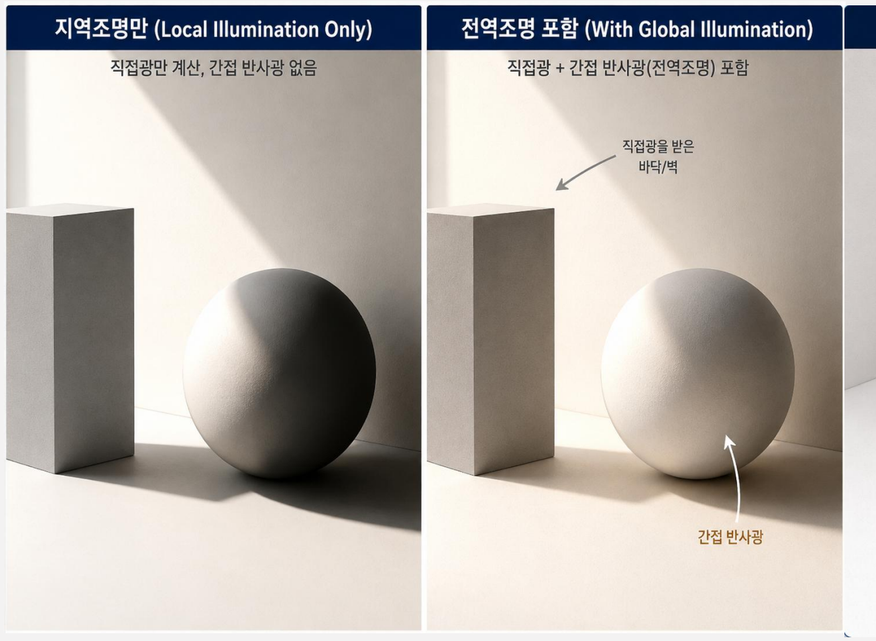
03 방향성 폐색 — 11.4

서론

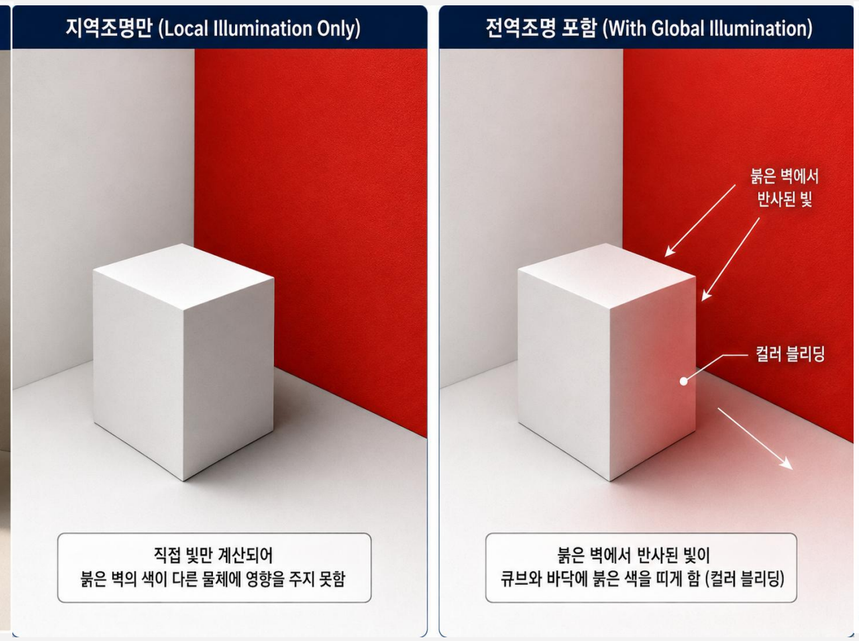
11장 1차시에 다룬 일반적인 전역 조명 알고리즘은 무겁지만,
7장에서 다룬 그림자와 10장에서 다룬 지역조명만으로는 물체 표면에서 반사된 간접광을 표현하지 못합니다.

아래 예시는 간접광이 만드는 대표적인 시각적 효과 3가지를 설명합니다. * 이미지는 AI 생성이라 실제 렌더링 결과와 차이가 있을 수 있음

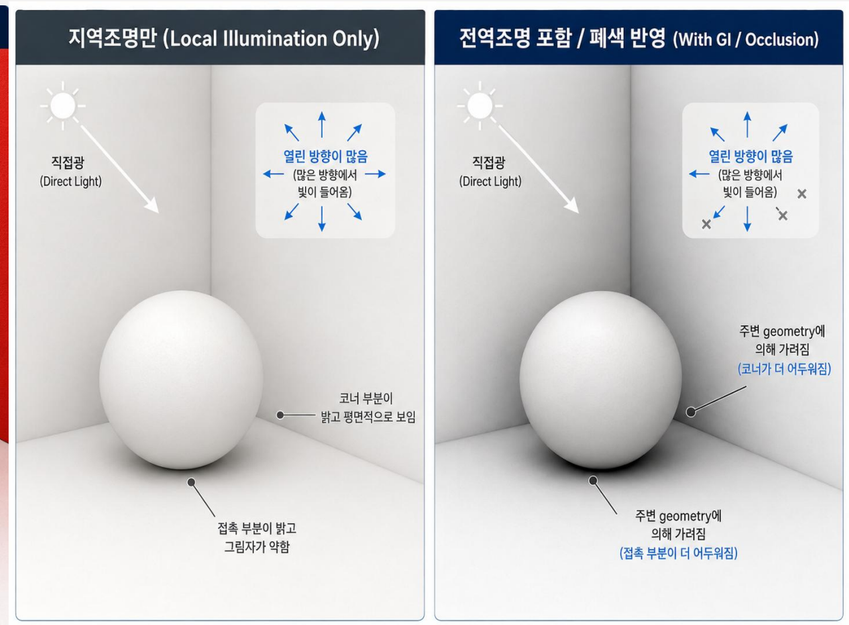
Indirect Fill : 직접광이 닿지 않더라도, 간접광을 닿아 밝아질 수 있음



Color Bleeding : 표면의 색상이 다른 표면의 반사광의 영향을 받음

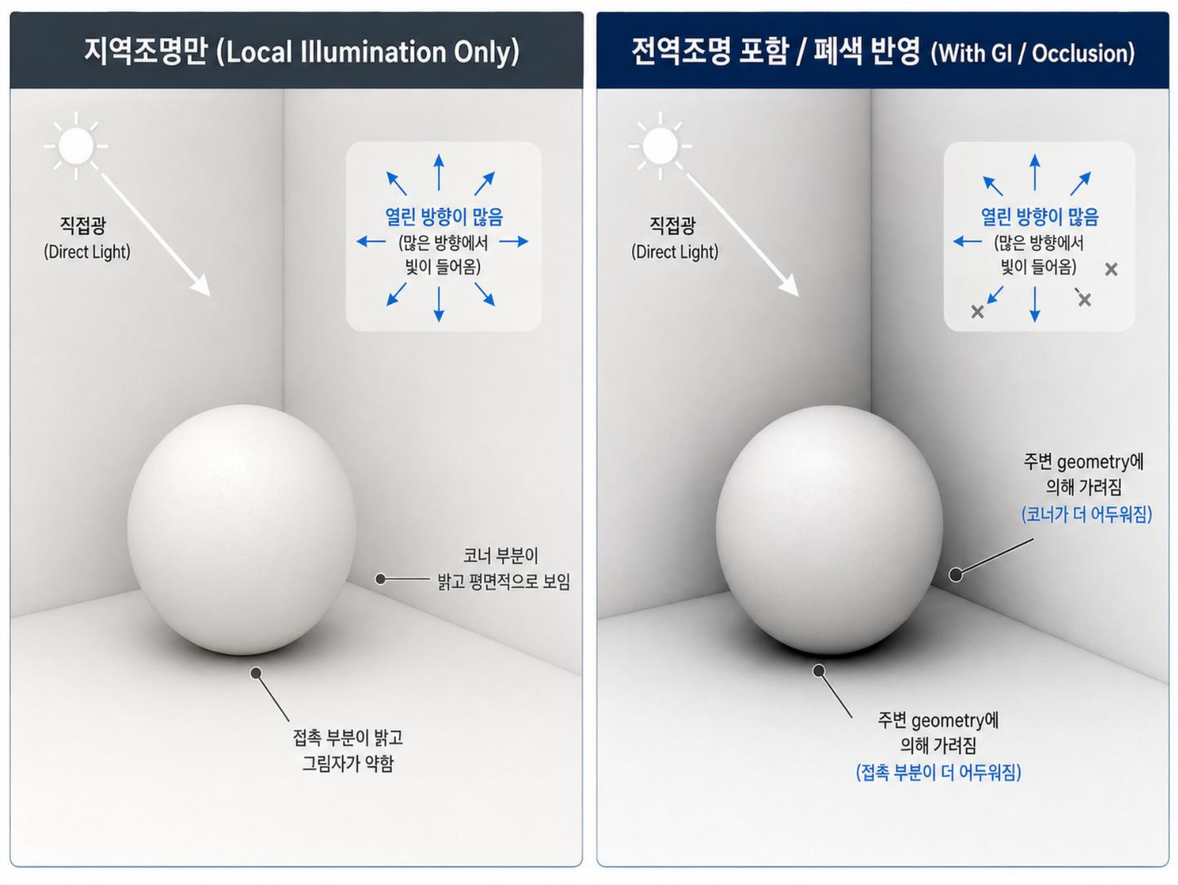


Occlusion : 빛이 들어오는 영역이 좁아지는 경우 상대적으로 어두워짐



서론

11.3 ~ 11.4는 주변광의 차폐를 어떻게 처리할 것인지에 대한 맥락을 담고 있습니다.



주변 폐색 (Ambient Occlusion)

1차시에 다룬 일반적인 전역 조명 알고리즘은 런타임에 사용하기에는 계산 비용이 너무 크고 복잡합니다.

이번 장에서 다룰 주변 폐색 (Ambient Occlusion)은 기본적이면서도 대표적인 GI 효과입니다.
영화 진주만 (2001)에서 비행기에 사용하기 위한 환경 조명의 품질을 개선하기 위해 개발되었다고 합니다.

매번 1900년대, 심하면 1800년대 사람들 따라가느라 자괴감 느껴지고 벅찼는데, GI는 실사용 영역에서는 2000년대 기술들 나와서 좀 감동

이때도 많은 영화의 렌더링에는 RayTracing이 사용되고 있었지만, 진주만은 (그 당시 기준으로는) 씬이 복잡해서 메모리 문제를 해결하기 위한 다른 수단이 필요했습니다.

* RayTracing 기반 렌더링을 위해서는 씬 전체가 메모리에 올라가야만 했습니다. 반사광은 반사되는 위치의 재질 정보에 따라서도 달라지기 때문
* 하지만 단순 차폐 검사는 오브젝트 단위로 미리 구워둘 수 있고, 재질의 변경에도 영향을 받지 않습니다. 왜 그런지는 이제 알아보도록 합시다.



진주만 영화에서 사용된 AO 효과의 예시 (출처)

주변 폐색 이론 (Ambient Occlusion Theory)

주변 폐색 이론의 이론적 배경은 반사율 수식에서 유도할 수 있습니다.
단순화를 위해 Lambertian 표면을 가정합니다.

이러한 표면에서 나가는 radiance L_o 는 surface irradiance E에 비례합니다. *irradiance = 복사조도, 표면이 받은 빛 (에너지)의 양

반사율 방정식

$$L_o(p,v) = \int_{l\in\Omega} f\,(l,v)L_i(p,l)(n\cdot l)^+ dl$$

Lambertian BRDF

$$f(l,v) = \frac{\rho_{ss}}{\pi}$$

Surface irradiance

$$E(p,n) = \int_{\Omega} L_i\,(p,l)(n\cdot l)^+ dl$$

$$L_o(p,v) = \frac{\rho_{ss}}{\pi} E(p,n)$$

* 10장에서 다룬 램버시안

여기에 더해, **단순화를 위해 들어오는 모든 방향 l에 대해, incoming radiance가 일정하다고도 가정**해봅시다. $L_i(p,l) = L_A$

*위 가정을 diffuse illumination(확산 조명)만 존재하는 환경이라고도 할 수 있습니다.

한글본(4판)에서는 *다시 말하지만 단순함을 위해 어쩌구*로 써놨는데
Lambertian 표면과 incoming radiance가 일정하다는건 서로 다른 가정 두개입니다.

$$E(p,n) = \int_{\Omega} L_A\,(n\cdot l)^+ dl$$

$$E(p,n) = L_A \int_{\Omega} (n\cdot l)^+\, dl = \pi L_A$$

* 수식 (11.6)

일정하고 균일한 조명이 Lambertian표면에 들어오면 irradiance (결과적으로는 radiance도) 표면 위치와 법선(normal)에 의존하지 않으며, 오브젝트 전체에 일정합니다.
이는 물체의 형태에 따른 명암이 보이지 않게 되며 flat하게 보이게 됩니다.

주변 폐색 이론 (Ambient Occlusion Theory)

방금 유도한 식은 가시성을 고려하고 있지 않습니다.
반구의 모든 방향에서 동일한 빛이 들어오더라도, 오브젝트 본인에게 가리거나 다른 오브젝트에 의해 빛이 차단될 수 있습니다.

이런 방향에서는 L_A 가 아닌 다른 radiance가 들어오게 됩니다.
여기서 또 **단순화를 위해 가려진 방향에 대한 radiance는 0이라고 가정**합시다.

차단된 오브젝트에서 오는 반사광 / 해당 방향에서 오는 산란된 빛 과 같은 요소를 무시하지만 결과를 단순화 할 수 있습니다.
이는 Cook과 Torrance가 처음 제안한 다음의 식을 얻게 됩니다.

$$E(p,n) = L_A \int_\Omega v\,(p,l)(n \cdot l)^+ dl$$ * 수식 (11.7)

여기서 v(p, l)은 p에서 l방향으로 투사되는 광선이 차단되면 0이고 차단되지 않으면 1인 가시성 함수(visibility function)입니다.

이 가시성 함수의 정규화된 코사인 가중치 적분(normalized, cosine-weighted integral)을 이번 장에서 다룰 주변 폐색(AO)이라고 합니다.

$$k_A(p) = \frac{1}{\pi} \int_\Omega v\,(p,l)(n \cdot l)^+ dl$$ * 수식 (11.8), 주변 폐색

완전히 가려진 표면의 경우 0, 어떤 방향도 가려지지 않은 경우 1이 됩니다.
n·l=cosθ 이기 때문에, 빛이 들어오는 방향에 따라 가중치가 다르게 됩니다 (코사인 가중치)

주변 폐색 이론 (Ambient Occlusion Theory)



주변 폐색 k_A을 정의했기 때문에, 폐색을 고려한 irradiance를 구할 수 있게 됩니다.

$$E(p,n) = L_A \int_\Omega v\,(p,l)(n \cdot l)^+ dl \qquad k_A(p) = \frac{1}{\pi}\int_\Omega v\,(p,l)(n \cdot l)^+ dl$$

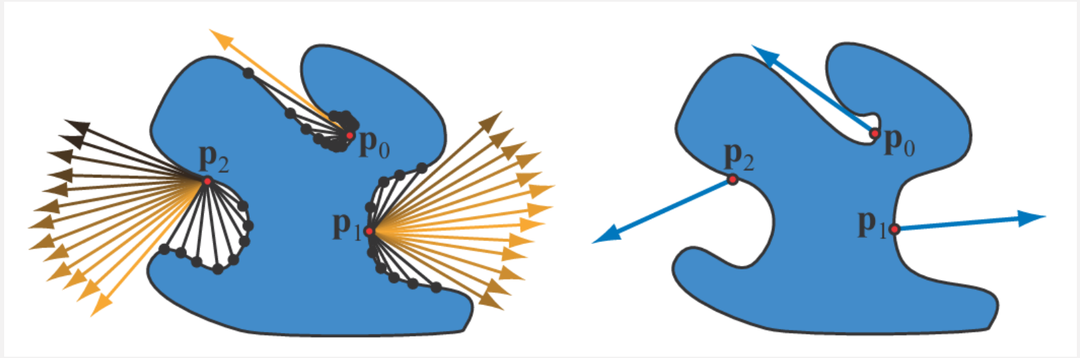
$$E(p,n) = k_A(p)\pi L_A$$ * 수식 (11.9)

이제 표면 위치 p가 바뀌면 주변 폐색 k_A이 변경되기 때문에, irradiance가 위치에 따라 변경되게 됩니다.
이는 그림의 예시에서 볼 수 있듯이 훨씬 현실적인 결과로 이어집니다.

오목한 표면 위치는 k_A값이 낮기 때문에 어두워지게 됩니다. (그림의 p0과 p1 비교)

가시성 함수 v(p, l)이 통합될때 코사인(n·l)값에 따라 가중되어 표면 방향도 영향을 미칩니다.
p1과 p2는 동일한 폐색되지 않은 각도를 가졌음에도 p1이 더 높은 값을 가지게 됩니다.

이후 다룰 내용부터는 간결한 표현을 위해 표면위치 p에 대한 의존성은 명시적으로 표시하지 않습니다.

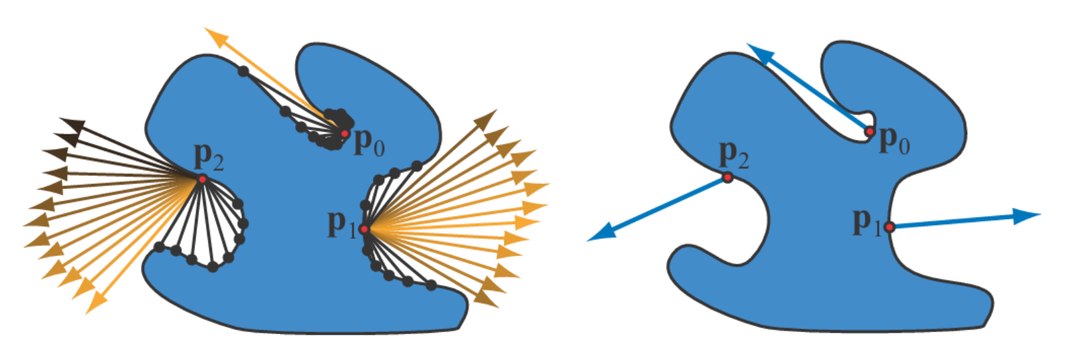


주변 폐색 이론 (Ambient Occlusion Theory)

주변 폐색 k_A 이외에도 Landis는 폐색되지 않은 방향의 코사인 가중 평균을 이용하여
구부러진 법선(bent normal)으로 알려진 폐색되지 않은 평균 방향도 계산하였습니다.

$$n_{bent} = \frac{\int_{l\in\Omega} l\, v(l)(n\cdot l)^{+} dl}{\left\| \int_{l\in\Omega} l\, v(l)(n\cdot l)^{+} dl \right\|}$$ * 수식 (11.10)

Bent normal은 셰이더 처리에서 실제 geometr의 normal 보다 더 정확한 결과를 만들기 위해 사용될 수 있습니다.
(실제 도달 가능한 라이트맵, irradiance volume, 그리고 이후 다룰 GI 결과들의 조회 등…)



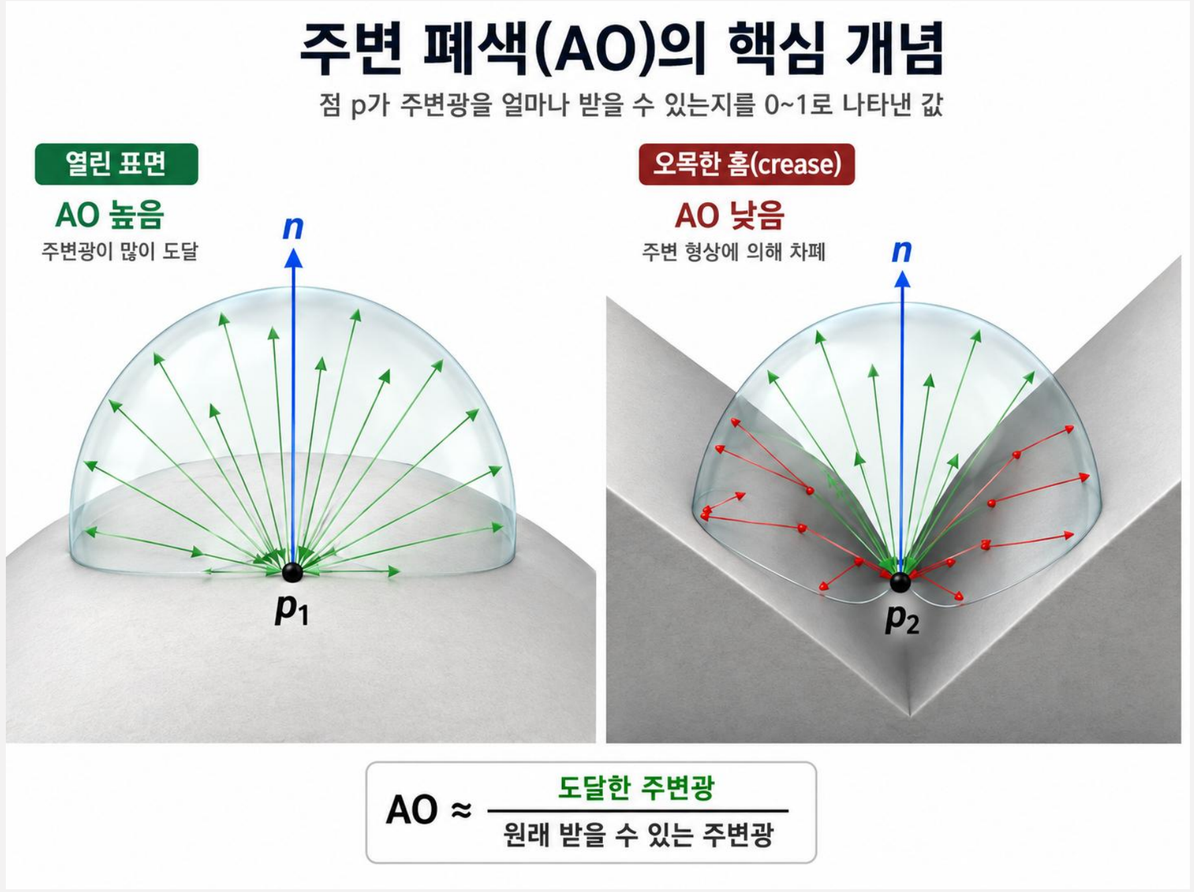
* p0, p1, p2에서 뻗어나가는 파란색 선분의 방향이 bent normal, 실제 물체의 normal 방향과 다를 수 있다

주변 폐색 이론 (Ambient Occlusion Theory) - 요약

주변 폐색은 주변 형상의 영향으로 주변광을 얼마나 받을 수 있는지를 0~1로 나타낸 값이 됩니다.

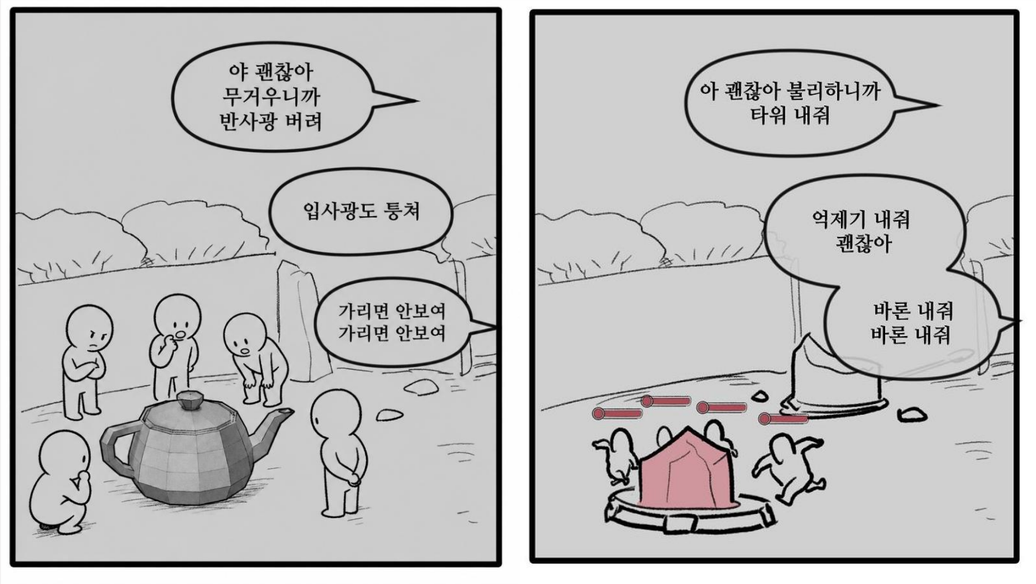
$E(n) = k_A \pi L_A$ * 수식 (11.9)

아니 이렇게 수식이랑 싸워서 얻은결론이
이거 저거 퉁치고 나면 빛이 얼마나 들어오는지 그냥 scalar하나랑 vector하나 남는다고



퉁치고 넘어간 가정 3개

표면은 Lambertian – (각도에 관계 없이 균일한 outgoing radiance)
Incoming radiance도 균일 (diffuse illumination만 존재)
차폐면 (또는 차폐 방향)에서 오는 radiance는 0



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

가시성과 차폐도(Visibility and Obscurance)

한글판은 Obscurance를 모호성이라고 번역했는데, 차폐도가 더 맞는 번역같습니다
뭐가 모호하다는건지 진짜 보면서 너무 모호했어...

주변 폐색 계수를 계산하기 위해서는 가시성 함수 v(l)가 필요했습니다.

$$k_A = \frac{1}{\pi}\int_{\Omega} v\,(l)(n \cdot l)^{+} dl$$

* 수식 (11.8), 주변 폐색

오브젝트의 표면p에서 l방향으로 ray를 쏘아서, 다른 물체에 닿게 되는지를 기반으로 v(l)을 정의하는건 간단하지만,
이는 닫힌 공간에서 결과가 좋지 못합니다.

다양한 오브젝트가 있는 닫힌 방에서는 표면에서 발사한 모든 광선은 무언가에 닿기 때문에
V(l)은 언제나 0이 되고, 자연스럽게 k_A 값도 0이 되는 문제가 생깁니다.

이런 닫힌 공간에서는 물리적인 가시성을 시뮬레이션 하지 않고 주변 폐색(AO)의 외형을 재현하려는 경험적인 접근 방법이
더 나은 결과를 내는 경우가 많습니다.

가시성과 차폐도(Visibility and Obscurance)

Miller의 접근성 셰이딩 개념에서 영감을 받아 Zhukov, Iones, Kronin는
가시성 함수 v(l)을 거리 매핑 함수 p(l)로 대체해, 주변 폐색 계산을 수정하는 obscurance라는 개념을 도입합니다.

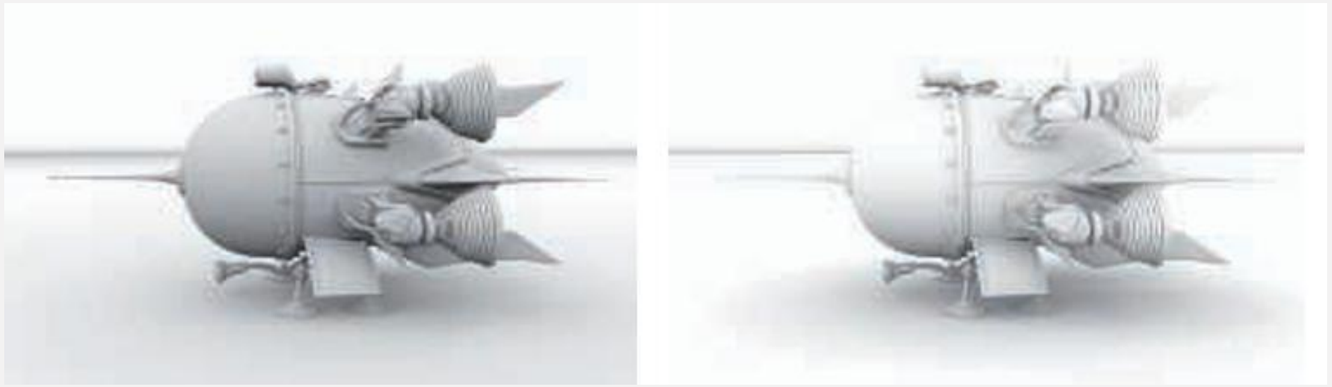
먼지가 쌓이거나, 변색된 표면은 주로 닦는 물체가 안 닿아서 생긴다는 내용
Accessibility를 표면p가 구형 probe에 얼마나 쉽게 닿을 수 있는가로 정의했다고 함

$$k_A = \frac{1}{\pi}\int_{\Omega} p\,(l)(n \cdot l)^{+} dl$$

* 수식 (11.11), 주변 폐색 – 거리매핑 함수

두 가지 유효한 값 (Shadow때도 그랬듯이, 0과 1)이 있는 v(l)과 달리, p(l)은 표면에 도달하기 전까지 광선이 이동한 거리를 기반으로 하는 연속함수입니다.
교차 거리가 0일때 0이며, 미리 지정한 $d_{\max}$ 보다 먼 거리에서 교차하거나 아예 교차하지 않는 경우는 1이 됩니다.
$d_{\max}$를 넘는 교차 거리는 검사할 필요가 없기 때문에, k_A의 계산 속도를 상당히 높힐 수 있게 됩니다.

그림은 주변 폐색과 주변 차폐도(ambient obscurance)의 차이를 보여줍니다.
주변 폐색은 먼 거리에서도 교차점이 감지되어 k_A 값에 영향을 미치기 때문에, 상대적으로 더 어둡게 나옵니다.



가시성과 차폐도(Visibility and Obscurance)

Obscurance의 거리 감쇠를 실제 광전달 현상과 연결해 물리적으로 설명하려는 시도도 있었지만,
차폐 효과는 단순히 occluder까지의 거리만으로 결정되지 않으므로 Obscurance 자체는 물리적으로 정확한 모델이 아닙니다.

그럼에도 불구하고 보는 사람의 기대에 부합하는 그럴듯한 결과를 내는 경우가 많습니다.
한가지 단점은 $d_{\max}$ 값을 수작업으로 조정해야 한다는 것입니다.

이처럼 직접적인 물리적 근거는 없지만 지각적으로 설득력 있는(perceptually convincing) 기법을 통한 절충안은 컴퓨터 그래픽스에서 흔한 일입니다.

대부분의 목표는 믿을만한 이미지를 만드는 것이기 때문에, 문제될 것은 없습니다.
이론에 기반을 둔 방법의 장점은 자동으로 동작할 수 있고, 현실 세계에 대한 추론을 통해 더 개선할 수 있다는 점입니다.

* 앞으로 다룰 많은 파트도 퉁치고 넘어갔던 부분을 어떻게 살릴지(개선할지)에 대해서 꾸준히 다룹니다.



훈련소에서 물리 박사님에게 그래픽스의 빨간약을 전했을 때 세상 무너진 표정이였다.

뭐라고 수치해석도 모자라서 이런 구라들을 섞어 치고 있었다고

상호 반사의 반영(Accounting for Interreflections)

AO로 생성된 결과는 시각적으로 설득력이 있지만, 완전한 전역 조명 모델(full global illumination)의 결과보다는 어둡게 나옵니다.

AO와 Full GI의 차이를 만드는 주요 원인은 상호 반사(Interreflections)입니다. 이전에 식 11.8에서 차폐된 방향에서 들어오는 radiance를 0이라고 가정했지만,
실제로는 그 방향들에서도 상호 반사에 의해 0이 아닌 radiance가 들어오게 됩니다.

이 Full GI와의 차이는 $\mathbf{k}_A$ 값을 키움으로서 보정할 수 있습니다.
가시성 함수 대신 직전에 다룬 거리 매핑 함수를 사용하면, 차폐된 방향에 대해서도 0보다 큰 값을 가지는 경우가 많아 문제를 완화하는데에 도움이 됩니다.





상호 반사의 반영 (Accounting for Interreflections)

상호 반사를 조금 더 정확하게 추적하려면 Full GI에서 다루는 재귀적인 문제를 풀어야 하므로 비용이 많이 들게 됩니다.
k_A 값을 계산하는 비용은 Full GI에 비해서 훨씬 저렴하지만, 과도하게 어두워지는 것을 막기 위해서는 누락된 빛을 어떤 형태로든 포함시키는 것이 좋습니다.

Stewart와 Langer는 AO가 가정했던 Lambertian surface와 diffuse illumination 환경에서는
한 점 p에서 보이는 주변 표면들의 radiance가 자신의 radiance와 비슷하다는 경향이 있다는 관찰에 기반하여 재귀 없이 해석적인 식을 얻어내었습니다.
책에서는 다루지 않았지만, 식이 유도되는 과정을 따라가봅시다.

기존 AO에서 차폐되지 않은 부분을 통해 받는 irradiance는 다음과 같습니다. $k_A \pi L_i$ * 수식 (11.9) 참고

차폐 방향에서의 incoming radiance는 현재 점의 outgoing radiance와 비슷합니다 (같다고 가정) $L_{blocked} \approx L_o(p)$

막힌 영역의 cosine-weighted 비율이 1-k_A 이기 때문에 차폐 방향에서 받은 irradiance는 다음과 같습니다. $\pi(1 - k_A)L_o$

따라서 전체 irradiance는 이렇게 정리 할 수 있고 $E = \pi k_A L_i + \pi(1 - k_A)L_o$

$L_o = \frac{\rho_{ss} E}{\pi}$ *Lambertian surface radiance

Lambertian surface를 가정했기 때문에, outgoing radiance항을 다시 irradiance로 표현할 수 있습니다.

$$E = \pi k_A L_i + \pi(1 - k_A)\frac{\rho_{ss} E}{\pi}$$

해당 식을 정리하면 다음과 같은 식을 찾을 수 있습니다.

$$E = \pi k_A L_i + \rho_{ss}(1 - k_A)E$$

$$E - \rho_{ss}(1 - k_A)E = \pi k_A L_i$$

$$E[1 - \rho_{ss}(1 - k_A)] = \pi k_A L_i$$

$$E = \frac{\pi k_A L_i}{1 - \rho_{ss}(1 - k_A)}$$ * 수식 (11.12)

위 식을 기존 AO 공식의 형태로 변환하면 k_A와 ρ_{ss}값을 통해 상호 반사를 복원하여 밝아진 k'_A 값을 구할 수 있게 됩니다.

$$k'_A = \frac{k_A}{1 - \rho_{ss}(1 - k_A)}$$ * 수식 (11.13)

$$E = \pi k'_A L_i$$ * 수식 (11.12)와 (11.13)을 합치면 기존의 AO와 동일한 형태의 식을 사용할 수 있다.

상호 반사의 반영(Accounting for Interreflections)

이렇게 유도된 결과는 상호 반사를 포함한 Full GI의 결과에 시각적으로 더 가깝게 만들어 주지만,
ρ_{ss} (subsurface albedo)값에 크게 의존적입니다.

* 물론 실제 Color Bleeding처럼, 주변 표면의 색상이 실제로 반영되진 않습니다.
* 같은 albedo값이라고 퉁쳤으니까요

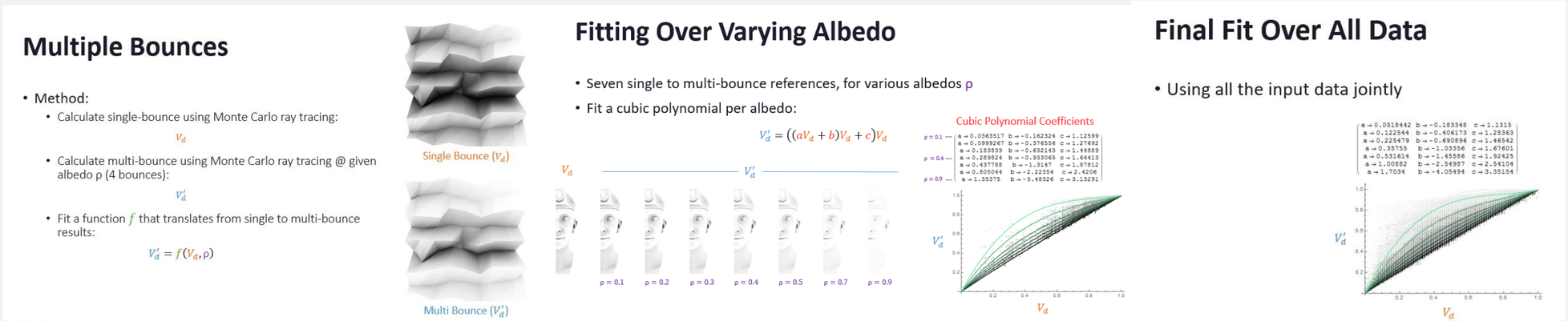
이 과정은 표면의 색상과 인접한 표면이 동일하다고 가정하여 작동하게 되며, 그렇기에 결과가 color bleeding과 비슷한 원리로 밝아지게 됩니다.

Hoffman과 Mitchell은 이 방법을 사용해 지형(terrain)에 하늘 전체에서 오는 확산 조명(diffuse lighting) 정보로 AO 정보를 미리 구워두었습니다.
이는 시간에 따라 sky color/radiance가 변경되더라도 GI를 다시 계산할 필요 없이, 변경된 값만 곱해주면 되게 되었습니다.

Jimenez는 주변 표면의 albedo가 크게 다르지 않고, diffuse illumination 환경에서 path tracing한 정보들을 모아 3차 다항식의 계수를 튜닝하였습니다.
k_A 와 ρ_{ss} 값만으로 k'_A 값을 예측할 수 있는 함수를 만들었다는 내용으로 이해하시면 될 것 같습니다.

Stewart-Langer의 방법은 해석적인 식을 통해 근사했지만, Jimemez는 실제 렌더링 환경의 path tracing정보를 기반으로 polynamial에 압축해두었기 때문에
물리적으로 올바름과 별개로 path tracing 결과와 평균적으로 더 유사하게 k'_A 값을 유추해내게 되었습니다.

*Jimenez의 방법은 블리자드 소속 인원 4명이 SIGGRAPH 2016에서 발표한 내용이네요 (출처)



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO 계수의 계산은 시간이 많이 소요될 수 있어서, 렌더링에 앞서 오프라인으로 수행하는 경우가 많습니다.
엠비언트 오클루전을 비롯해 조명 관련 정보를 사전에 계산해두는 과정을 흔히 베이킹(baking)이라고 부릅니다.

AO를 사전 계산하는 가장 일반적인 방법은 몬테카를로 기법입니다.
1차시 RayTracing에서 짚고 넘어갔듯이, 반구 방향으로 광선을 쏘아 오브젝트의 차폐 여부를 검사하고, AO 계수를 수치적으로 계산합니다.

예를 들자면 법선 n 주위의 반구에 균일하게 분포된 N개의 무작위 방향 l을 고른 뒤, 그 방향으로 광선을 추적한다고 해봅시다.
RayCast 결과를 바탕으로 가시성 함수 v를 평가하면 AO는 다음과 같이 계산됩니다.

* 수식 (11.14), 몬테 카를로 방법 가시성 주변 폐색

$$k_A = \frac{1}{N}\sum_{i=1}^{N} v\,(l_i)(n \cdot l_i)^+$$

* 참고용 수식 (11.8), 주변 폐색

$$k_A = \frac{1}{\pi}\int_{\Omega} v\,(l)(n \cdot l)^+ dl$$

주변 차폐도(Ambient Obscurance)를 계산할 때는 광선의 거리를 최대치로 제한할 수 있으며, v의 값은 검출된 교차 거리를 기준으로 결정됩니다. (11.3.2 내용)

AO 계수의 계산에는 코사인 가중 인자가 포함됩니다 (각도에 따라 중요도가 다르다)
수식 11.14처럼 이를 직접 곱해서 넣을수도 있지만, 더 효율적으로 반영하는 방법은 중요도 샘플링(importance sampling)을 이용하는 것입니다.
반구 전체에 균일하게 광선을 쏘고 n·l을 곱하는 대신 광선 방향의 분포 자체를 normal vector 방향에 가까운 쪽으로 많이 쏘아지게 하는 것입니다.
이런 샘플링 기법을 Mally's method라고 합니다.

이러한 AO의 사전 계산은 CPU / GPU에서 수행할 수 있다. 두 경우 모두 복잡한 geometry에 대한 raycast를 가속하는 라이브러리를 사용할 수 있으며
가장 널리 쓰이는것은 Embree(CPU), OptiX(GPU)입니다.

과거에는 깊이 맵(depth map)이나 오클루전 쿼리 같은 GPU 파이프라인의 결과물을 AO계산에 사용하기도 했으나,
GPU에서 동작하는 RayCast 솔루션이 널리 보급되며 오늘날에는 그런 방식을 쓰는 경우가 줄었습니다.

상용 모델링/렌더링 소프트웨어 패키지는 대부분 AO를 사전 계산하는 기능을 제공하고 있습니다.

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO는 표면의 모든 위치 p마다 값이 다르지만, 무한히 많은 p에 대한 모든 정보를 저장할 수는 없습니다.
이를 해결하기 위해 일반적으로는 Texture, Volume, Mesh의 Vertex에 저장하고 런타임에는 이를 복원하는 과정을 거치게 됩니다.
저장 방식에 따른 특성과 문제점은 신호의 종류와 무관하게 대체로 비슷합니다.
이번 장에서 다루는 AO를 포함해서 다음장에서 다룰 방향성 차폐(DO), 사전계산된 조명등의 정보는 동일한 방법론을 적용할 수 있고, 3차시에 설명하게 될 예정입니다.

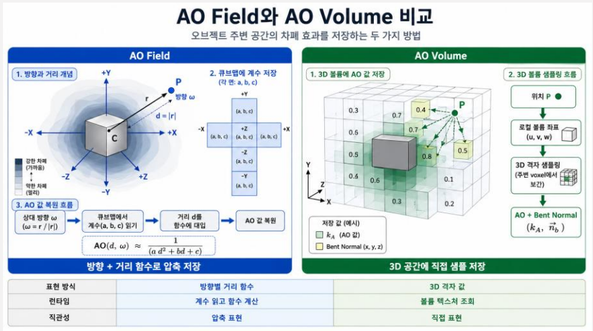
사전 계산된 데이터는 오브젝트들이 서로에게 미치는 AO효과를 모델링하는데에도 사용할 수 있습니다.

Kontkanen과 Laine는 오브젝트로부터의 거리에 따른 AO 값의 감쇠를 2차 다항식의 역수로 근사하고,
방향 ω별 계수를 큐브 맵에 저장하는 Ambient Occlusion Field를 제안하였고,

Malmer등은 AO계수와 bent normal(선택)을 3D Grid상에 저장하는
Ambient Occlusion Volume을 제안하였습니다.

동적인 AO처리가 무거우니 오브젝트단위로라도 구워둘 수 있을까?를 고민한 흔적이고
현대 AO는 대부분 이렇게 처리하지 않아서 넘어갑니다

*게임 엔진에서 이런 방식으로 처리하는거 본적도 없기도 하고, 비슷한 이름의 기능들은 현대 AO처리 방식에 더 가까웠음
*2000년 후~ 2010년대 상용 게임의 커스텀 엔진에서 동적인 AO처리를 위한 기능들이였다 보시면 될 듯



*물론 뒤에 다루겠지만, 동적인 오브젝트의 AO를 정말 싸게 처리해야하는 상황이 있습니다. 모바일 기기에서 동적인 오브젝트에 꼭 필요한 AO를 구현할 때 같은...

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO 값을 저장하는 방법으로 어떤것을 선택하더라도, 다루는 대상이 연속 신호라는 점을 유념해야 합니다.
공간상의 특정 지점에서 Ray를 쏘는것은 샘플링이고, 그 결과로부터 셰이딩에 사용할 값을 보간하는것은 복원입니다.

Kavan등은 최소 제곱 베이킹이라는 방법을 제안했습니다.
Mesh 전체에 균일하게 AO 계수를 샘플링하고, 보간된 값과 샘플링된 값의 차의 제곱의 합이 최대한 작게 만드는 값을 선정해 계수를 저장하게 합니다.
*분산이나 표준편차가 가진 아이디어랑 비슷하고, 이것을 최소화 하는 값을 만들어 저장하게 한다는 아이디어

예시로는 Mesh의 Vertex를 들었지만, Texture나 Volume에 해당 값을 저장할때도 동일한 방법론을 사용할 수 있습니다.

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

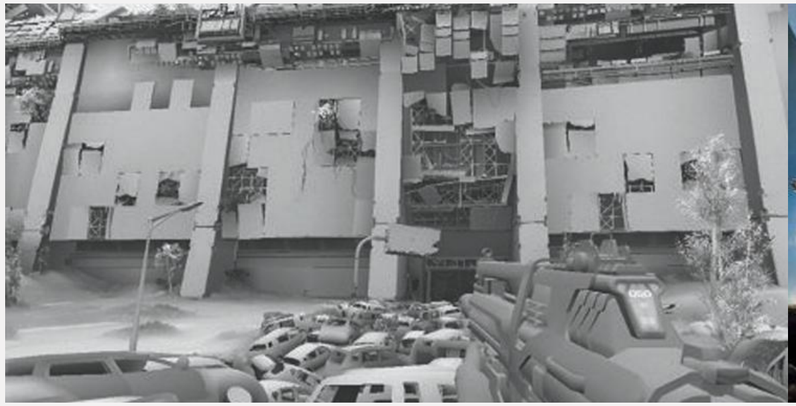
Destiny(2014)는 PS3(2006)에서 PS4(2013말) / XBOX360(2005)에서 XBOX ONE(2013말)으로 넘어가는 과도기에 출시되어
신규 플랫폼에 기대되는 높은 품질과 구세대 플랫폼의 성능-메모리 제약 사이의 균형잡힌 솔루션이 필요했습니다.

게임에는 동적으로 시간이 변하는 솔루션이 있어, 어떤 사전 계산 방식을 사용하더라도 이를 올바르게 반영할 수 있어야 했습니다.

개발진은 그럴듯한 외형과 낮은 런타임 처리 비용을 이유로 AO를 선택했으며
AO는 가시성 계산을 조명과 분리하여 시간대와 무관한 사전 데이터를 재사용할 수 있었습니다.

Ubisoft의 어세신크리드와 파크라이 시리즈도 간접 조명 솔루션을 강화하고자 사전 계산된 AO 형태를 사용했습니다.
월드를 위에서 내려다보는 시점에서 렌더링하여 깊이 맵을 여러 휴리스틱을 통해 월드 공간 AO맵을 만들어
각 오브젝트의 월드 공간 위치를 텍스쳐 공간으로 투영하는 방식으로 모든 오브젝트에 AO를 적용했습니다. (Ubisoft는 이를 World AO라고 부름)

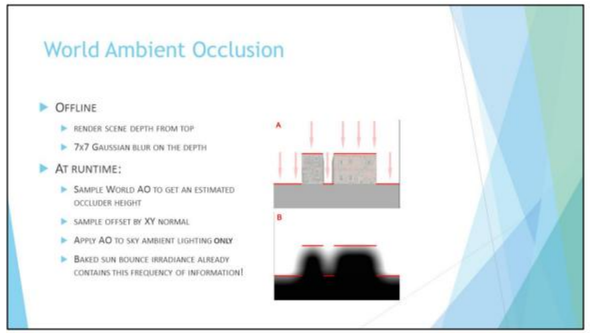
*Digital Dragons 2014 유비소프트 발표 자료 (출처)







Looks of “World ambient occlusion” in Assassin’s Creed 3



Algorithm very approximate and requires artists to tweak some “magic values”, but extremely cheap to evaluate and quite robust.
…As long as your scene can be approximated by heightfield!

리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

정적인 장면의 경우 AO 계수와 bent normal을 사전 계산할 수 있습니다.
하지만 오브젝트가 움직이거나 형태가 바뀌는 장면에서는 이런 계수들을 실시간으로 계산하는 편이 더 나은 결과를 만들어냅니다.
이를 위한 방법은 크게 오브젝트 공간(object space)에서 동작하는 것과, 스크린 공간(screen space)에서 동작하는 것으로 나눌 수 있습니다.

11.3.5에서는 주로 object/world space에서 동작하는 AO를 다룰 예정입니다.

AO를 계산하는 오프라인 방식은 표면에서 장면을 향해 수십에서 수백개에 이르는 다량의 광선을 쏘고 교차 여부를 검사하는 과정을 포함합니다.
이는 비용이 큰 연산이기 때문에 실시간 기법들은 이 계산의 상당 부분을 근사하거나 피해가는 방법에 초점을 맞춥니다.

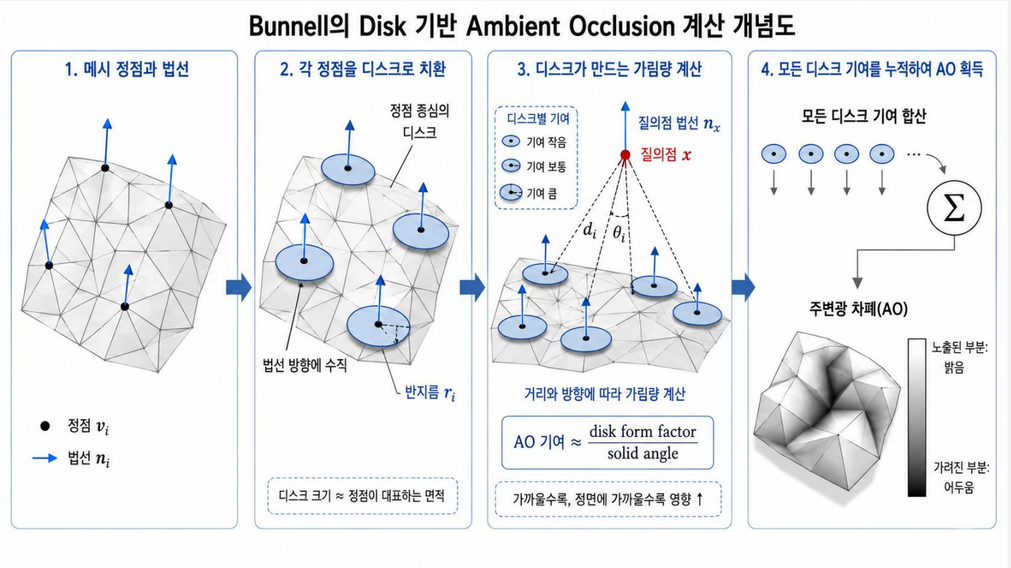
Bunnell은 표면을 Mesh Vertex에 배치된 원반(Disk)모양 요소들의 집합으로 모델링 하여 AO 계수와 bentnormal을 계산합니다.
원반을 택하면 한 원반이 다른 원반을 가리는 이유를 해석적으로 계산할 수 있어 ray를 쏠 필요가 없어지기 때문입니다.

하지만 단순히 모든 Disk의 폐색 계수를 합산하면 이미 가려진 부분을 새로 가리는 문제가 발생할 수 있습니다.
Bunnell은 이 문제를 2Pass로 처리해서, 두번째 Pass에서는 첫번째 Pass에서 계산한 AO계수에 따라 다른 Disk (Surface)에 주는 영향을 줄이도록 하여 해결했습니다.

각 Disk사이의 폐색을 계산하는것은 O(n²) 연산으로, 비용이 많이 나가게 됩니다.
이것은 Disk를 hierarchical tree로 구성하여 서로 먼 표면에 있을 때는 parent 하나로 처리해 실질적으로 O(nlogn)까지 처리 비용을 낮추게 됩니다.

해당 기술은 영화 캐리비안의 해적의 최종 렌더링에 사용되었습니다.

Hoberock은 11.3.2에서 다룬 Obsurance와 유사한 거리 감쇠 계수를 포함해 Bunnell의 알고리즘을 조금 더 비싸지만 고품질로 수정하였습니다.



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

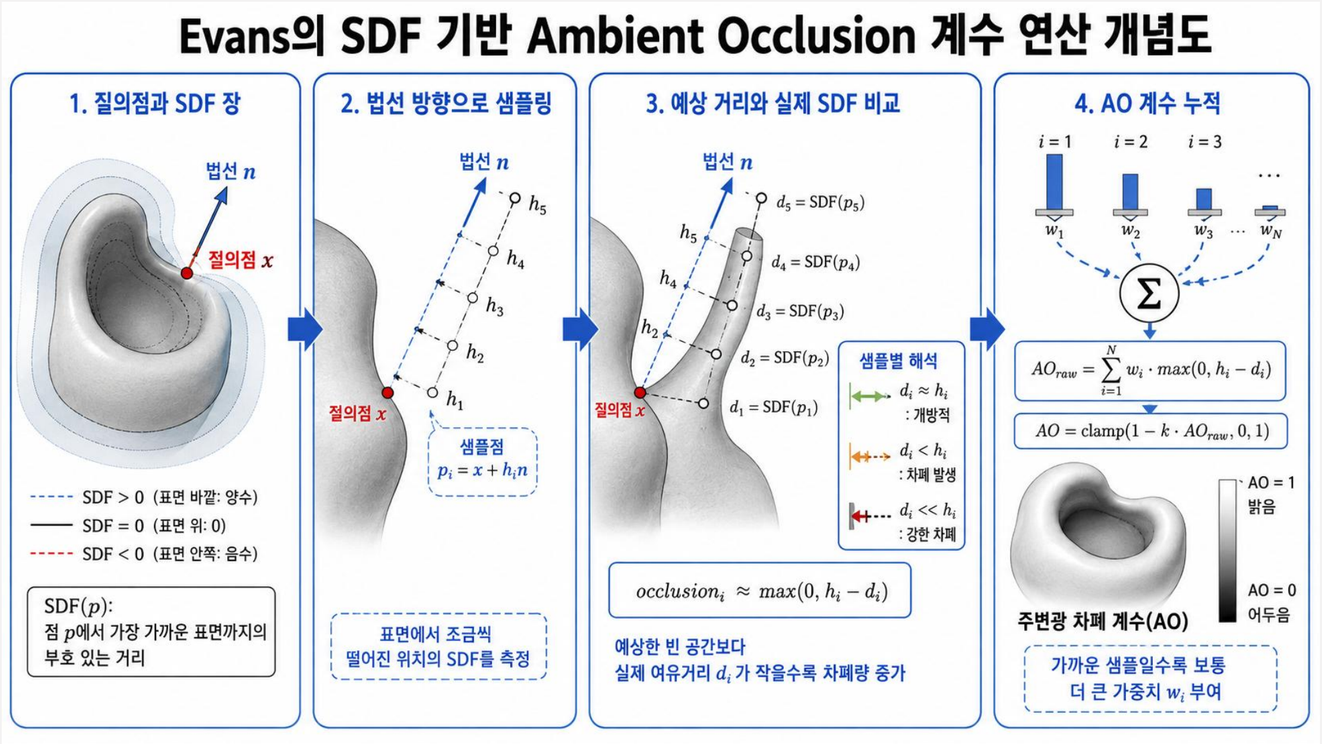
부호 있는 거리 필드 (SDF, Signed Distance Field)는
object 내부 깊숙할수록 음수, object의 표면(surface) 지점에서는 0, object 외부에서 멀어질수록 양수가 되는 함수입니다.

AO에서는 SDF를 읽었을때, 해당 지점에서 가장 가까운 geometry의 존재를 검사할 수 있게 되기 때문에 사용합니다.

Evans는 이 특징을 살려 기존의 hemisphere 방향을 검사하던 AO대신 normal방향으로 점점 먼 점에 대해 SDF를 sampling하는 방법을 제안하였습니다.

이는 SDF값이 3차원 텍스쳐 (Volume)에 저장되는 대신
해석적으로 표현하는 경우에도 같은 접근법을 쓸 수 있습니다.

이 방법은 물리적이지는 않지만,
시각적으로 만족스러운 결과를 만들 수 있습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

SDF를 AO에 사용하는 방법은 Wirght에 의해 한층 더 발전했습니다.
휴리스틱으로 normal 방향의 SDF를 샘플링하여 AO 계수를 계산하는 대신 Cone tracing을 수행합니다.

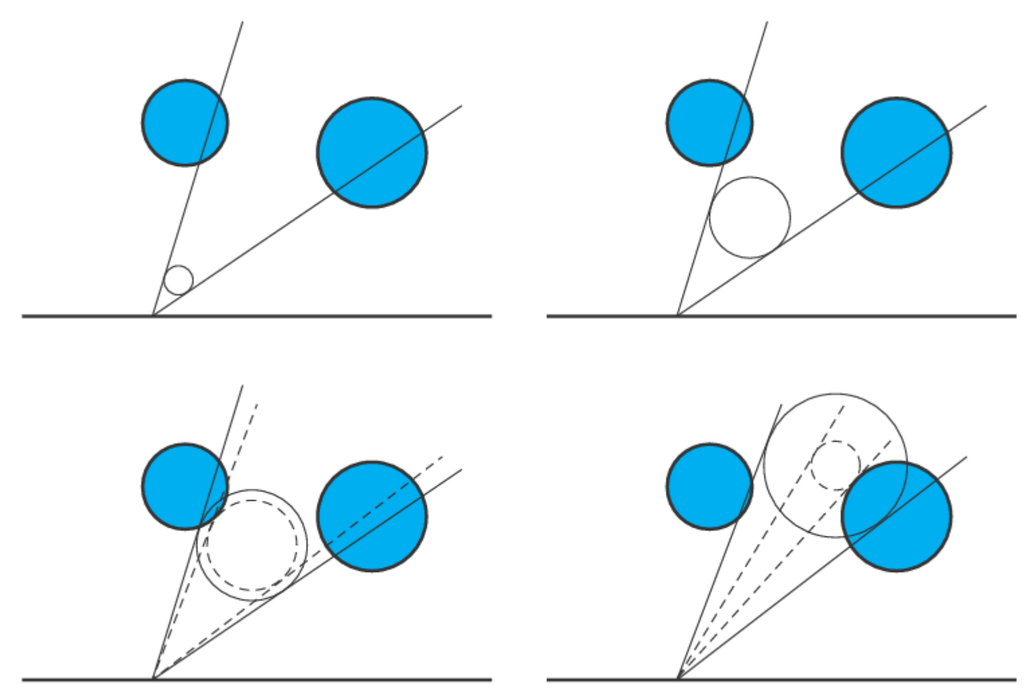
법선 기준 반구에 약 9개의 cone을 배치하고
각 cone 축을 따라 SDF sphere / cone steepping을 수행합니다.
하나의 cone내에서는 값을 합산하는게 아니라 minimum visibility를 유지하도록 합니다.

9개의 cone의 visibilit를 결합하여 AO계수와 bentnormal을 생성합니다.

SDF를 통해 빛이 실제로 통과할 수 있는 가능성을 근사하고 있었다 보심 될 듯

장면에 대한 전역 SDF뿐 아니라 개별 object나
논리적인 object group의 SDF도 함께 사용하여
정밀도를 높혔습니다.

*동적 AO에서 SDF를 사용한다는 것은 SDF를 매 프레임 생성한다는 뜻이 아닙니다.
일반적으로 geometry의 distance-field representation은 미리 준비하고,
런타임에는 현재 shading point와 object transform에 따라 SDF를 query하여 AO를 동적으로 계산합니다

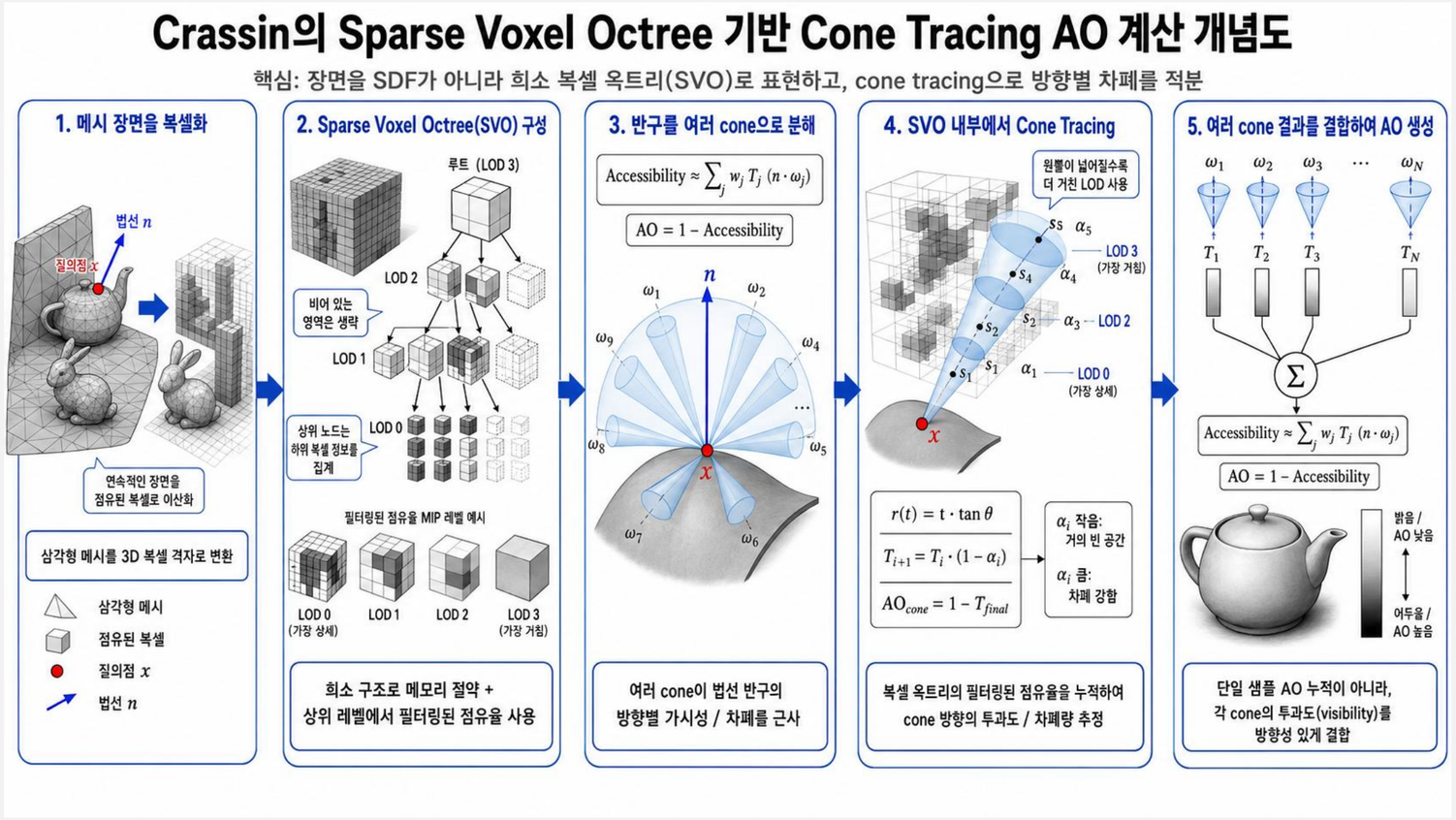


리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Crassin은 장면을 먼저 voxel로 변환합니다.
거대한 3D 공간을 dense volume으로 저장하면 너무 크기 때문에, 빈 공간은 크게 묶고 geometry가 있는 곳만 세분화 하는 Sparse Voxel Octree(SVO)를 구성합니다.

Octree를 구성하였기 때문에, Cone Tracing에서 멀어진 cone은 상위 레벨의 octree를 참조할 수 있게 됩니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

오늘의 순서 - 3차시

00 주변 폐색 — 11.3 * 생각보다 많이 남았습니다

01 방향성 폐색 — 11.4

02 Heckbert 표기법 — 11.1 * 추진력을 얻기 위해 버렸지만 추진력은 얻지 못했다. 업보 청산의 시간

03 확산 전역 조명 — 11.5 * 근데 얘도 AO랑 분량이 비슷할거같아서 무서워요



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Ren은 AO가 흐릿하다는 점을 이용해 Occulsion Geometry를 구 모음으로 근사했습니다.

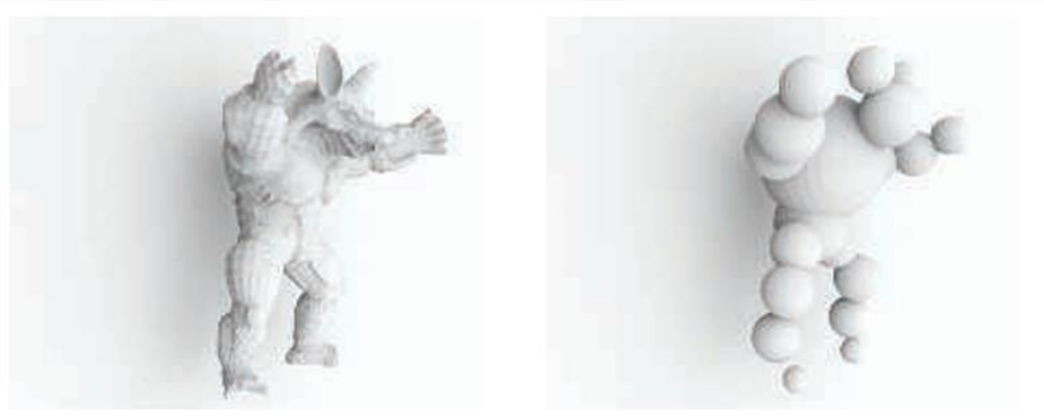
단일 구에 가려지는 표면점을 표현하는 가시성 함수를 구면 조화 함수를 사용해 표현할 수 있고
여러 개의 구에 의한 차폐의 결과는 개별 구의 가시성 함수를 곱한 결과가 됩니다.

하지만 구면조화 함수끼리의 곱을 계산하는것은 비용이 큰 연산이고
이를 해결하기 위해 개별 구면 조화 함수의 로그를 더한 뒤, 그 결과에 지수를 취하는 것입니다.
이렇게 하면 가시성 함수를 곱한것과 동일한 최종 결과를 얻으면서도, 훨씬 저렴하게 처리할 수 있습니다.

$$V_1(l)V_2(l)V_3(l)$$

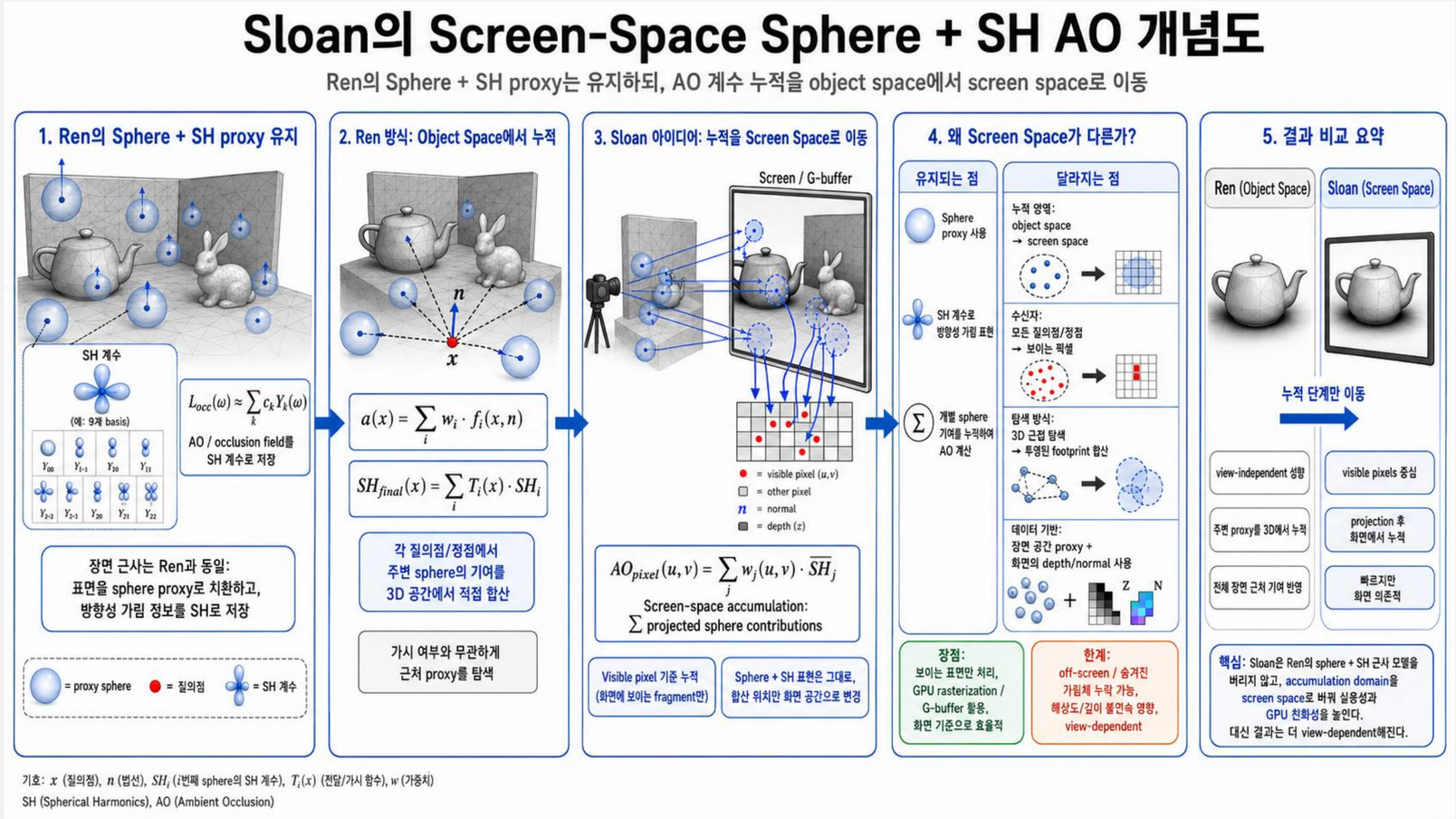
$$\log V_1(l) + \log V_2(l) + \log V_3(l)$$

$$\log \prod_{i=1}^{N} V_i(l) = \sum_{i=1}^{N} \log V_i(l)$$



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Sloan은 Ren의 Sphere + SH방식의 visibility 누적 연산을 screen space에서 처리하는 방법을 제안하였고,
Hill은 SH 함수를 2차 계수까지만 저장하여 성능이 제한적인 하드웨어에서도 사용할 수 있지만, 흐릿한 AO 처리만 가능한 방법을 제안했습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

스크린 공간 기법(Screen-Space Methods)

직전까지 다룬 object-space를 기반으로 만들어지는 AO들은 scene geometry를 계산해야 하기 때문에
구성된 씬의 복잡도에 따라 영향을 받게 됩니다. (안보이는 물체들도 연산 대상이 됩니다)

하지만 차폐에 대한 정보 중 일부는 깊이(Depth)나 법선(normal)처럼 이미 확보되어있는 스크린 공간 데이터 만으로도 유추해 낼 수 있습니다.

이러한 기법들은 장면이 얼마나 정밀한지와 무관하게, 오직 렌더링에 사용하는 해상도에만 좌우되는 일정한 비용을 갖습니다.
(이전 파트에서 Sloan의 방법이 가지는 메리트, 하지만 여기부터는 AO계수 누적을 위한 샘플링도 ScreenSpace에서 처리합니다.)

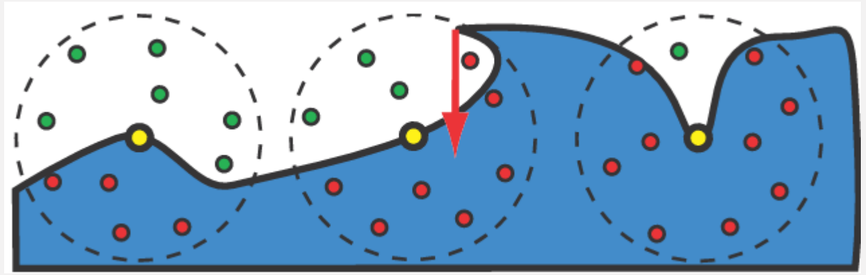
화면에 보이지 않는 geometry 정보는 알 수 없게 되는 문제가 있지만,
기존 렌더링 과정중에 남는 데이터거나, 렌더링 과정중에 조회한 데이터를 별도 버퍼에 저장해두기 때문에 GPU 친화적인 장점이 있습니다.

스크린 공간 기법(Screen-Space Methods)

Crytek은 Crysis (2007)에 사용된 동적 **스크린 공간 엠비언트 오클루전** (screen-space ambient occlusion, **SSAO**) 기법을 개발했습니다.

이들은 Z-Buffer만을 입력으로 사용하는 전체 화면 패스에서 AO를 계산합니다.

각 pixel의 폐색 계수 k_A는 픽셀 위치를 중심으로 한 구 내부에 분포시킨 점들의 집합을 z-buffer와 비교 검사하며 추정합니다.
k_A의 값은 현재 pixel의 z-buffer 값보다 앞에 있는 샘플의 개수에 대한 함수입니다.
이를 통과하는 샘플수가 적을수록 k_A값이 낮아지게 됩니다.



샘플들은 Obscurance factor와 마찬가지로 픽셀로부터 거리에 따라 감소하는 가중치를 가집니다.

스크린 공간 기법(Screen-Space Methods)

여기서 샘플에 $(n \cdot l)^+$ 인자로 가중치를 주지 않기 때문에, 결과로 나타나는 AO는 부정확하다는 점에 유의해야한다.
표면 위쪽 반구(surface hemisphere)대신 전체 sphere를 대상으로 하기 때문에, 평평한 면도 조금 더 어두워지고
Edge는 주변보다 밝아지는 현상이 생깁니다.

하지만 그럼에도 결과물은 시각적으로 만족스러운 경우가 많습니다.

*SSAO Only





*Albedo

*SSAO + Albedo





*SSAO + Albedo + Shadow + Specular

리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

스크린 공간 기법(Screen-Space Methods)

코사인 가중치도 없고, 반구도 아니고 구형의 샘플링으로 정확하지 않은 부분이 있는 SSAO를 보완하기 위해 여러가지 연구들이 있었지만
시간 관계상, 이후 바로 HBAO (2008)부터 소개합니다.

방금 다룬 SSAO기술을 몬테 카를로 적분으로 해석해 수식으로 표현하면 다음과 같은 형태가 됩니다.

$$v_A = \int_{x \in X} \rho\left(d(x)\right) o(x)\, dx$$

* 수식 (11.15), 몬테 카를로 적분으로 해석한 SSAO

ρ = obscurance파트에서 다룬것과 유사한 거리 매핑 함수
d = 거리 함수
$o(x)$ = 점유함수 (x가 depth buffer기준으로 가려져있으면 1, 아니면 0)

Bavoil등은 z-buffer정보가 연속적인 높이 장(height field)를 나타낸다고 가정하고,
특정 점에서 보이는 지평선(horizon)을 추정 할 수 있음을 관찰한 Max의 연구 결과에 집중했습니다.

코사인 항을 무시한다면, 어떤 점에서의 가시성은 탄젠트평면 위쪽의 차폐되지 않은 부분에 대한 적분으로 계산할 수 있고,
다르게는 1에서 차폐된 부분의 적분을 뺀 값으로 계산할 수 있습니다.

$$k_A = 1 - \frac{1}{2\pi} \int_{\varphi=-\pi}^{\pi} \int_{\alpha=t(\varphi)}^{h(\varphi)} W(\omega) \cos(\theta)\, d\theta\, d\varphi$$

* 수식 (11.16), HBAO

$W(\omega)$ = 거리기반 감쇠함수

Bavoil등은 z-buffer정보가 연속적인 높이 장(height field)를 나타낸다고 가정하고,
특정 점에서 보이는 지평선(horizon)을 추정 할 수 있음을 관찰한 Max의 연구 결과에 집중했습니다.

어떤 점에서의 가시성은 탄젠트평면 위쪽의 차폐되지 않은 부분에 대한 적분으로 계산할 수 있고,
다르게는 1에서 차폐된 부분의 적분을 뺀 값으로 계산할 수 있습니다.

스크린 공간 기법(Screen-Space Methods)

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi}\int_{\alpha=t(\varphi)}^{h(\varphi)} W(\omega)\cos(\theta)\, d\theta\, d\varphi$$

* 수식 (11.16), HBAO

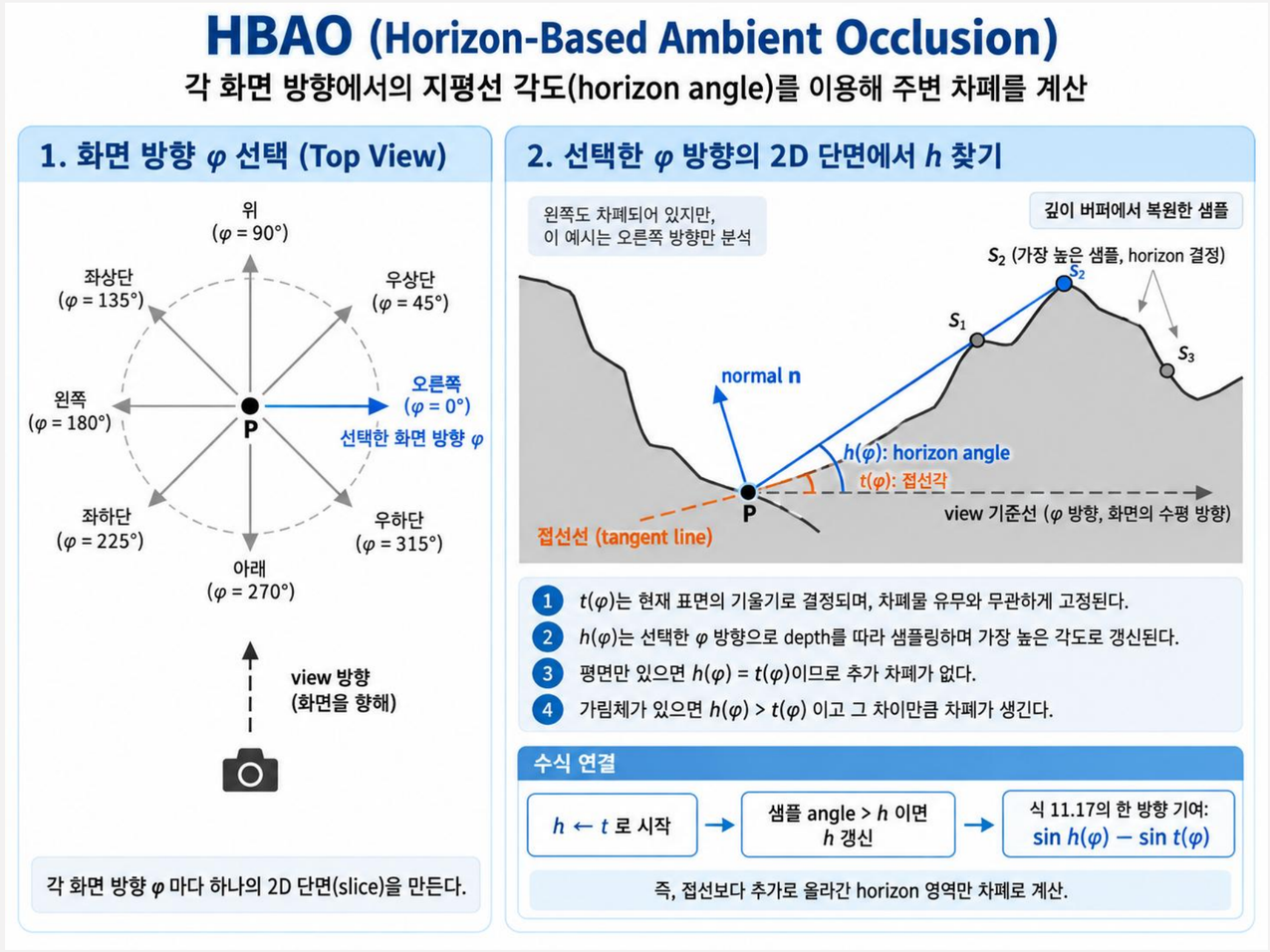
HBAO 아이디어는 알겠는데, 결국 반구 방향으로 다 Ray쏘라는 소리 아닌가? 싶겠지만 식을 잘 보면 다음과 같은 관찰이 가능합니다.

$$\int_a^b \cos\theta\, d\theta = \sin b - \sin a \quad \longrightarrow \quad \int_{t(\varphi)}^{h(\varphi)} \cos\theta\, d\theta = \sin\big(h(\varphi)\big) - \sin\big(t(\varphi)\big)$$

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi} [\sin(h(\varphi)) - \sin(t(\varphi))]\, W(\varphi)\, d\varphi$$

* 수식 (11.17), 안쪽 적분을 푼 HBAO

Surface에서 tangentspace를 참고해 여러 ϕ방향으로 ray marching을 시행해 차폐된 영역의 AO값을 점진적으로 누적하는 방식으로 계산할 수 있습니다.



스크린 공간 기법(Screen-Space Methods)

HBAO의 구현체의 예시입니다. (의사코드)

```
float EstimateHBAO(Pixel P, Normal N)
{
   float occlusion = 0;

   for (Direction dir : directions)
   {
      // 현재 surface의 tangent를 기준으로 시작
      float horizon = TangentAngle(P, N, dir) + angleBias;

      for (int step = 1; step <= numSteps; ++step)
      {
         // 1. 화면상 다음 위치의 depth를 읽음
         vec2 sampleUV = P.uv + dir * step * stepSize;

         // 2. depth -> view-space 위치 복원
         vec3 S = ReconstructPosition(sampleUV);

         // 3. P에서 S를 보았을 때 elevation angle
         float angle = ElevationAngle(P.position, S, N);

         // 4. 기존 horizon보다 높은 occluder인가?
         if (angle > horizon)
         {
            float distance = length(S - P.position);

            // 경험적인 거리 감쇠
            float W = 1.0 - Square(distance / radius);

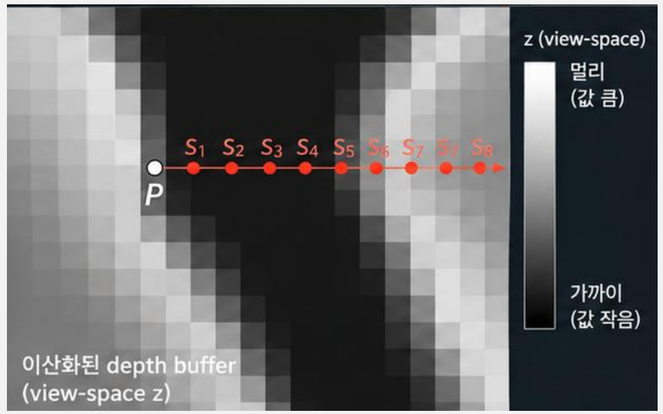
            // 새롭게 가려진 angular 영역만 추가
            occlusion += W * (sin(angle) - sin(horizon));

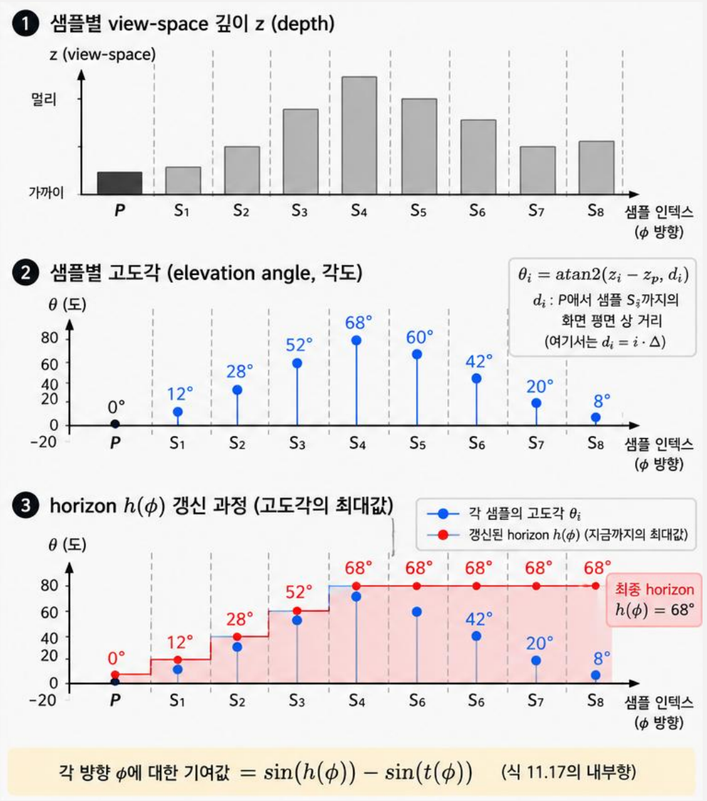
            horizon = angle;
         }
      }
   }

   return 1.0 - occlusion / numDirections;
}
```

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi} [\sin(h(\varphi)) - \sin(t(\varphi)))]\, W(\varphi)\, d\varphi$$

* 수식 (11.17), 안쪽 적분을 푼 HBAO





리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

스크린 공간 기법(Screen-Space Methods)

SSAO도 그렇지만, HBAO도 코사인 가중치가 없습니다.
거리기반 감쇠함수도 경험적으로 설정하게 됩니다.

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi} [\sin(h(\varphi)) - \sin(t(\varphi)))]\, W(\varphi)\, d\varphi$$

$W(\omega)$ = 거리기반 감쇠함수

* 수식 (11.17), 안쪽 적분을 푼 HBAO

이를 해결하기 위해 GTAO는 depth buffer의 heightfield를 실제 geometry라고 가정하여 이를 계산하는 아이디어를 사용합니다.

$$k_A = \frac{1}{\pi}\int_{\Omega} v\,(l)(n \cdot l)^+ dl$$

* 수식 (11.8), 주변 폐색

$$k_A = \frac{1}{\pi}\int_{0}^{\pi}\int_{h_1(\phi)}^{h_2(\phi)} \cos(\theta - \gamma)^+ \,|\sin\theta|\, d\theta\, d\phi$$

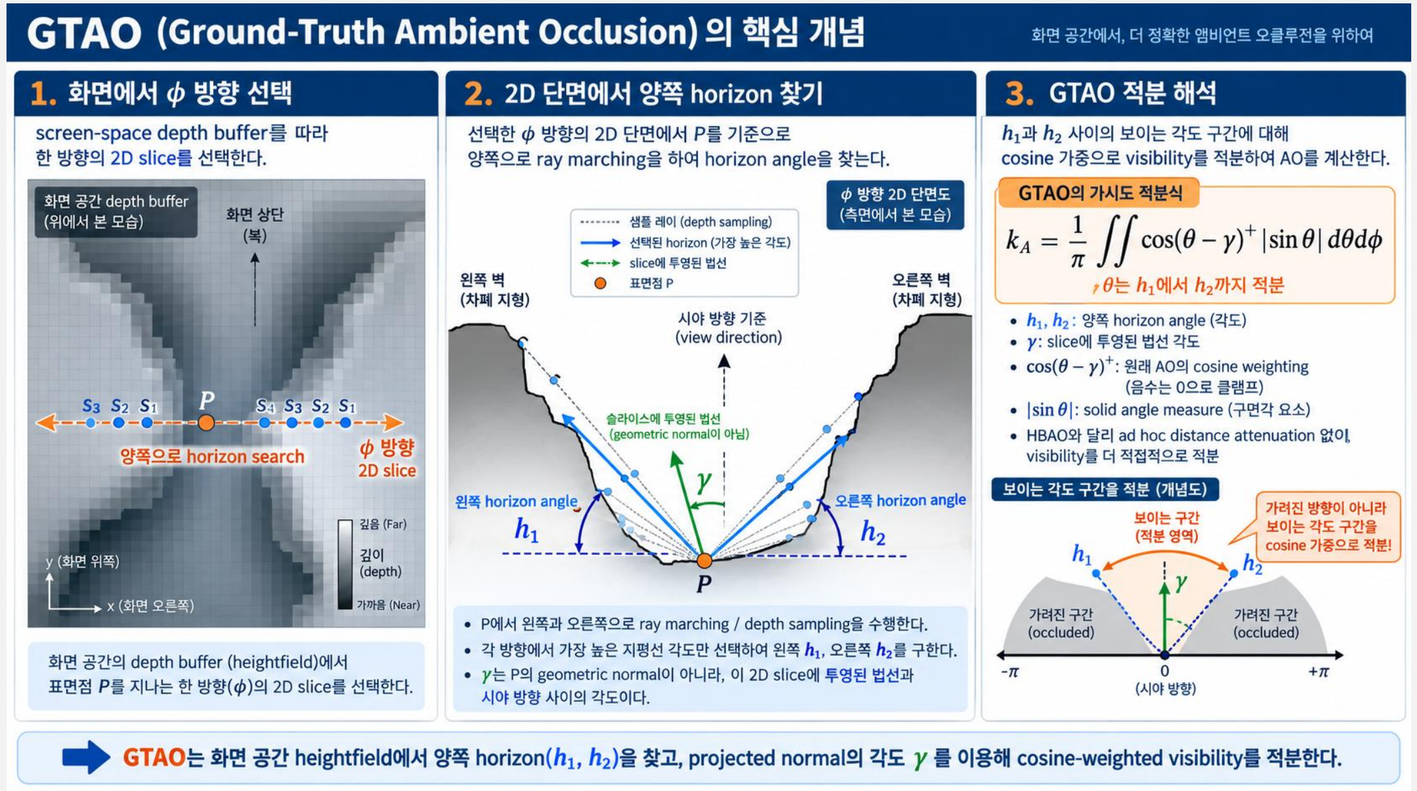
* 수식 (11.18), GTAO

γ = surface normal과 view direction 사이의 각

GTAO의 수식도 이중 적분이지만
HBAO와 같이 안쪽 적분은 해석적으로 풀 수 있어

HBAO와 GTAO가 상수 시간복잡도를 제외하면
크게 차이나지 않습니다.

(둘다 horizon 탐색 기반)



스크린 공간 기법(Screen-Space Methods)



1
기둥뒤에 공간 있어요

* Z Buffer기반으로는 기둥 뒤에 공간이 있는지 알 방법이 없다!
* 저런 공간에서 AO를 그리면 공간이 없다고 생각하고 차폐된 영역으로 계산하게 될 것

Z Buffer를 기반으로하는 Screen-Space Method들은 주어진 방향에 대해 가장 가까운 오브젝트 위치만 기록할 뿐,
차폐된 뒤에 무엇이 있는지를 나타내지 않습니다.
많은 기법이 보이는 오브젝트의 두께를 추론하기 위한 갖가지 휴리스틱을 사용하곤 합니다.
이런 근사는 많은 상황에서 충분하며, 사람의 눈은 부정확함에 관대한 편입니다 (책에는 그렇다는데, AO같은 변곡점이 눈에 띄지 않는 차폐라 더 그런편)

여러겹의 Depth Buffer를 사용하는등의 기법들도 연구되었지만, 렌더링 엔진과의 통합이 복잡하고 런타임 비용이 비싸 널리 쓰이지 못했습니다.

이를 해결하기 위해 최신 엔진이나 게임에서는 하드웨어의 발전으로 나타난 RayTrace기반의 AO기법들도 사용되곤 하지만,
이에 대한 구체적인 내용은 4차시에 다루도록 합시다.

스크린 공간 기법(Screen-Space Methods)

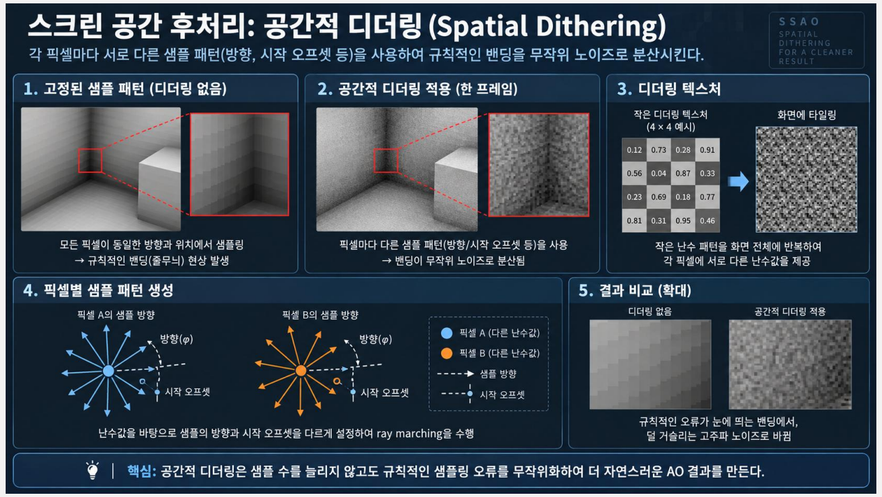
이런 스크린 공간 기법들은 대부분 주어진 점 주변 지오메트리의 단순화된 모델을 만들기 위해 z-버퍼를 반복적으로 샘플링하는 데 의존합니다.
실험에 따르면 높은 시각적 품질을 얻기 위해서는 수백개의 샘플이 필요하지만,
실시간 렌더링에서는 기기 사양에 따라 10~20개, 종종 그보다도 적은 수의 샘플이 작동할 수 있습니다.

이론과 실무 사이의 간극을 메우기 위해 스크린 공간 기법들은 일종의 공간적 디더링을 사용합니다.
화면의 픽셀마다 조금씩 다른 (회전시키거나 이동시킨) 무작위 샘플 집합을 사용하는것입니다.

적은 샘플로 이루어진 AO계단의 주 단계가 끝나면, 전체 화면 필터링 패스를 수행합니다.

표면의 불연속면을 가로질러 필터링하는것을 막고 선명한 경계를 보존하기 위해 결합 양방향 필터링(joint bilateral filtering)을 사용합니다.
이에 대해서는 12.1.1에서 다룰 예정입니다.

이 필터는 깊이나 normal정보를 활용해 동일한 표면에 속한 샘플만 사용하도록 필터링을 제한합니다.



스크린 공간 기법(Screen-Space Methods)

또한 AO계산은 시간에 걸쳐 슈퍼샘플링 되는 경우도 많습니다.

매 프레임마다 다른 샘플링 패턴을 적용하고 차폐 계수를 지수 평균하는 방식으로 이루어집니다.
이전 프레임들의 z-buffer, moving vector, 카메라 변환등의 정보를 이용해 현재 프레임의 pixel위치와 맞추고
신뢰할 수 없는 데이터의 경우 폐기합니다.

이 데이터를 만들때는 depth, normal, moving vector등에 기반한 휴리스틱이 많이 (정말 많이!) 사용됩니다.

AO에서는 이것이 치명적인 아티팩트를 만드는 경우가 드물지만, 신뢰 할 수 없는 데이터가 많아 샘플링할 데이터가 없는 경우
더 적극적으로 공간적 디더링을 사용하는 등의 방법으로 한 프레임의 샘플링이 인접 픽셀과 크게 다른 결과를 만드는 것을 방지합니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

주변 폐색을 이용한 셰이딩(Shading with Ambient Occlusion)

지금까지 주변 폐색 값을 일정하고 먼 거리의 조명 (diffuse illumination)이라는 맥락에서 유도하기는 했지만, 이를 더 복잡한 조명 상황에도 적용할 수 있습니다.

$$L_o(v) = \int_{l \in \Omega} f(l, v) L_i(l) v(l) (n \cdot l)^+ \, dl$$

출력 = 재질함수 × 입사광 × 가시성 × 입사각

* 수식 (11.19), 반사율 방정식

반사율 방정식을 다시 살펴봅시다. 위 형태는 11.3.1 (렌더링 방정식) 파트에서 도입했던 가시성 함수 v(l)을 포함합니다.
AO를 처음 정의했을때, 저희는 Lambertian surface를 가정했으므로, 재질 함수를 적분 밖으로 분리할 수 있습니다.

$$L_o = \int_{l \in \Omega} \frac{\rho_{ss}}{\pi} L_i(l) v(l) (n \cdot l)^+ \, dl = \frac{\rho_{ss}}{\pi} \int_{l \in \Omega} L_i(l) v(l) (n \cdot l)^+ \, dl$$

* 수식 (11.20), Lambertian 표면에서의 반사율 방정식

식 정리 과정은 생략하고, AO 정의까지 포함해 다음과 같이 식을 정의하면 다음과 같은 식이 나오게 됩니다.

$$L_o = k_A \rho_{ss} \int_{l \in \Omega} L_i(l) K(n, l) \, dl$$

* 수식 (11.22), Lambertian 표면에서의 주변 폐색을 포함한 반사율 방정식

$$K(n, l) = \frac{V(l) (n \cdot l)^+}{\int_{l \in \Omega} V(l) (n \cdot l)^+ dl}$$

* 수식 (11.23), 방향성 필터링 커널

식 11.22의 적분은 입사 래디언스 Li에 방향성 필터링 커널 K를 적용하는 것으로 생각할 수 있습니다.
필터 K는 위치와 방향에 따라 복잡하게 변하지만, 반구 범위에 있고 전체 적분은 1이 되도록 normalize되어있습니다.

주변 폐색을 이용한 셰이딩(Shading with Ambient Occlusion)

수식 11.22를 기반으로 셰이딩을 수행하기 위해서는 각 방향의 입사 radiance Li에 대해 방향성 필터링 커널 K를 적용하는 것으로 생각할 수 있지만

$$K(n,l) = \frac{V(l)(n \cdot l)^+}{\int_{l \in \Omega} V(l)(n \cdot l)^+ dl}$$ * 수식 (11.23), 방향성 필터링 커널

이를 수행하려면 입사 래디언스 Li와 필터함수 K라는 두 함수의 곱을 적분해야합니다.
Li와 K를 모두 구면 조화 함수등의 저비용으로 처리하는 방안도 있지만, 필터를 비슷한 성질을 가진 더 단순한 필터로 근사하는 것입니다.

가장 흔히 선택되는 것은 정규화된 코사인 커널 H입니다.

$$H(n,l) = \frac{(n \cdot l)^+}{\int_{l \in \Omega} (n \cdot l)^+ \, dl}$$ * 수식 (11.24), 정규화된 코사인 커널

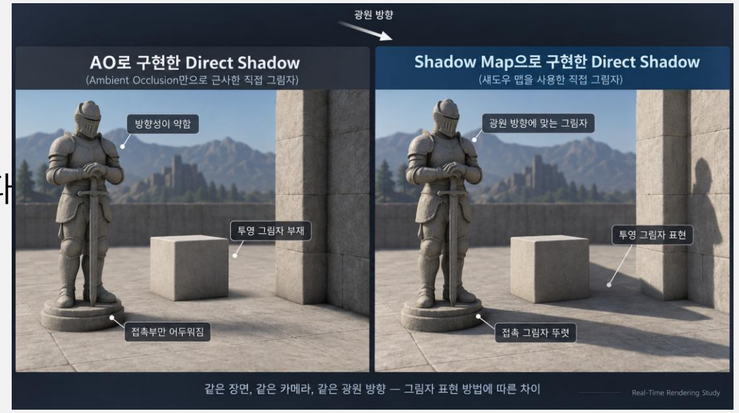
방향성 필터링 커널 K의 경우는 가시성 함수 V가 필요했지만, 코사인 커널 H는 가시성 함수가 필요하지 않습니다.

$$L_o = k_A \rho_{ss} \int_{l \in \Omega} L_i(l) \frac{(n \cdot l)^+}{\int_{l \in \Omega} (n \cdot l)^+ \, dl} \, dl = \frac{k_A}{\pi} \rho_{ss} E$$ * 수식 (11.25), 정규화된 코사인 커널

$$E(n) = k_A \pi L_A$$ * 수식 (11.9), 주변 폐색

주변폐색의 정의를 다루었을때와 마찬가지로 단순한 형태에서 AO를 이용하면 셰이딩이 Irradiance와 AO값을 곱하는 것 만으로 시행할 수 있게 됩니다.

주변 폐색을 이용한 셰이딩(Shading with Ambient Occlusion)



이 수식은 AO가 점광원이나 작은 면 광원의 가시성을 근사하기에는 부적절한지에 대한 통찰도 제공합니다.
그런 광원은 표면에서 볼때 작은 입체각만 차지하며, 이때 가시성 함수는 조명 적분값에 결정적인 영향을 미칩니다.

사실상 이진적으로 광원의 기여를 제어하는데 방향에 따른 광원의 기여를 날려버린 AO는 이를 잘 표현할 수 없습니다
그림자는 윤곽이 뚜렷하지 못하고 기대되는 방향성도 전혀 드러나지 않게 됩니다.

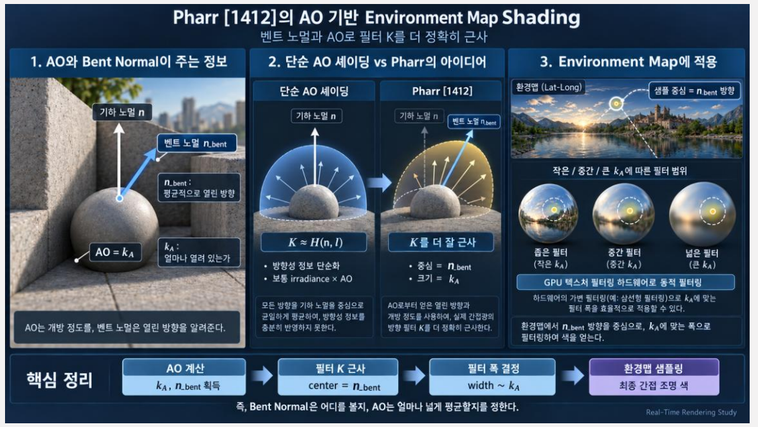
또한 지금까지는 램버시안 표면을 셰이딩한다고 가정해왔습니다.
더 복잡한 상수가 아닌 BRDF를 다룰때는 재질함수를 적분 밖으로 분리할 수 없고,
스페큘러 재질의 경우 필터 K(방향성 필터링 커널)는 가시성과 법선 뿐 아니라 시선 방향에도 의존합니다.

전형적인 microfacet BRDF로브를 정해둔 단일 형태로 근사하는것도 너무 거칠은 결과를 만들어 내기 때문에
이후 다룰 다른 방법을 사용하는것이 좋습니다.

AO 정의 초반에 다룬 Bent normal을 사용하는것은 필터 K를 좀 더 정밀하게 근사하는 한가지 방법으로 볼 수 있게 됩니다.
기하 법선과 bent normal이 일치하지 않는 경우 조금 더 정확한 결과를 제공하게 됩니다.

환경맵을 이용한 셰이딩의 경우 GPU의 텍스쳐 필터링 하드웨어를 통해 동적으로 수행하는 대안을 제시했습니다.

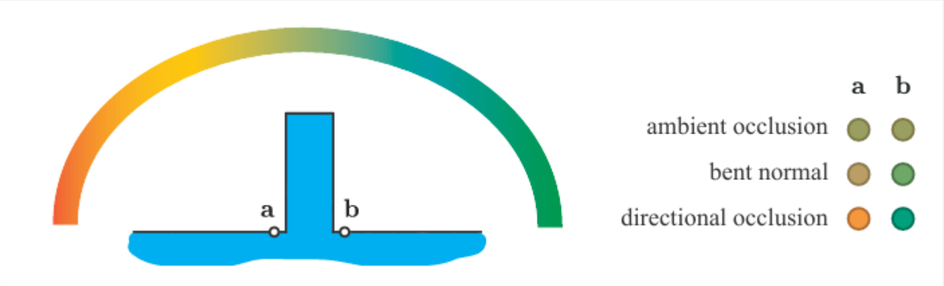
필터 K의 형태는 bent normal의 방향과 AO값에 따라 정해집니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

방향성 폐색 (Directional Occlusion)

11.3 파트에서 다룬 주변 폐색은 방향성 차폐 요소들을 계산하더라도, 셰이딩 시점에는 단일 scalar값으로 처리하여 방향성 정보를 손실했습니다.
Incoming light가 방향에 따른 색이나 세기가 다른 상황에는 AO값만으로는 이를 표현할 수 없게 됩니다.



이번 장에서 다루는 내용은 AO로는 처리하지 못했던, 어떤 방향이 보이는지에 대한 정보를 인코딩 하는 방법을 다룹니다.
해당 정보를 이용해 위치 광원의 그림자를 그릴 수 있지만, 이것이 방향성 폐색의 주 목적은 아닙니다.

(7장은 Shadow Map등을 통해, 명확한 visibility를 검사할 수 있었음)

미리 계산된 방향성 폐색(Precomputed Directional Occlusion)

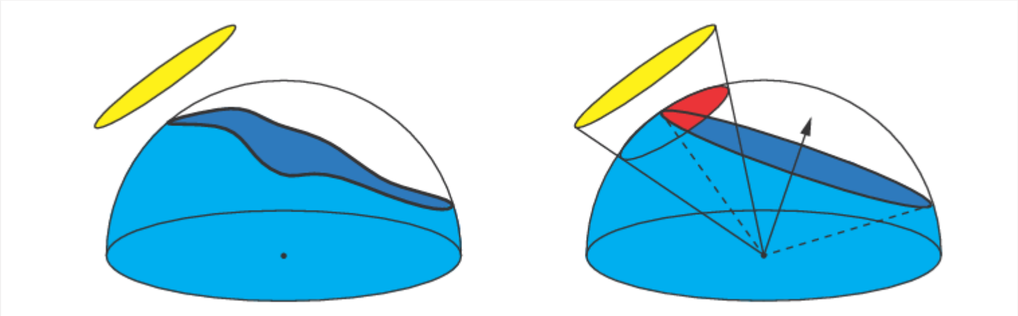
주변 폐색을 다루며, 반구 방향으로의 visibility를 계산하는 방법은 이미 다루었습니다.
하지만 런타임에는 v(l)함수를 평가하는 대신 이를 단일 AO계수로 압축했었는데, 이를 단일 AO계수로 압축하는 대신
미리 계산된 방향 가시성 정보들을 어떻게 저장할지에 초점을 맞춥니다.

Max는 heightfield 표면의 자기 차폐를 표현하기 위해 수평선 매핑(horizon mapping) 개념을 도입했습니다.
표면의 각 점마다 일정한 방위각 방향들에 대해 지평선의 고도각을 구합니다. (HBAO에서 사용하던 그 개념 맞습니다)

정해진 방위 방향별로 지평선 각도를 저장하는 대신, 차폐되지 않은 3차원 방향들의 집합 전체를 타원형 또는 원형 조리개로 모델링 할 수 도 있습니다.
이를 ambient aperture lighting이라고 합니다.

이는 수평선 매핑보다 저장 공간이 적게 들지만, 차폐되지 않은 방향의 집합이 타원이나 원과 닮지 않았다면 그림자가 잘못 나올 수 있습니다.

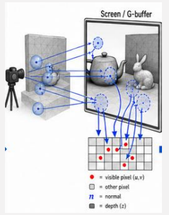
차폐 기법에는 여러가지 변형도 있습니다.
Wang등은 가시성을 표현하기 위해 구면 부호 거리 함수 (SSDF)등을 사용하였고, 구면 위에서 차폐 영역의 경계까지의 부호 있는 거리를 인코딩합니다.
10.3에서 다룬 구면/반구면 기저도 가시성 인코딩에 사용할 수 있으며, AO와 마찬가지로 해당 정보는 텍스쳐, 메시 정점, 볼륨등에 저장할 수 있습니다.



* Ambient aperture lighting

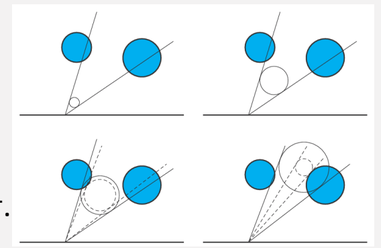
방향성 폐색의 동적 계산(Dynamic Computation of Directional Occlusion)

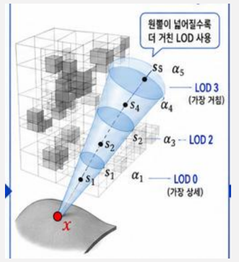




AO를 생성하는 기법 중 상당수는 방향성 가시성 정보를 생성할때도 쓸 수 있습니다.
Ren등의 구면 조화 함수 거듭제곱과 이를 화면 공간으로 옮긴 Sloan등의 기법은 가시성을 구면 조화 함수(SH) 벡터 형태로 생성합니다.
2개 이상의 SH 대역이 사용되는 경우, 별도 처리 없이도 방향 정보를 제공할 수 있으며 대역을 더 많이 사용할 수록 더 정밀하게 인코딩 할 수 있게 됩니다.

Crassin등과 Wright의 기법 같은 콘 트레이싱 방식은 트레이스 한번마다 차폐 값을 구하고,
AO를 추정할때조차 이를 여러 번 수행하므로 방향 정보를 확보할 수 있습니다.
특정 방향의 가시성 정보가 필요하면 원뿔을 더 적게 트레이스 하면 됩니다.





화면 공간 기법 중에서도 방향성 차폐 정보를 제공하도록 확장 할 수 있는 것이 다수 있습니다.

Klehm등은 z-buffe데이터로 화면 공간 bent cone을 계산합니다.
이는 사실상 원형 조리개로 픽셀 주변을 샘플링하면서 차폐되지 않은 방향들을 합산합니다.
합산된 벡터의 길이로 가시성 원뿔의 꼭지각을 추정할 수 있고, 벡터의 방향이 원뿔의 축이 됩니다.

Jimenez등은 지평선(horizon) 각도로부터 원뿔 축의 방향을 수정하고, 원뿔의 각도는 AO 계수로부터 유도합니다.



* 어디서 본 것 같은 내용이 다시 보이는 것 같다구요?
맞습니다

방향성 폐색을 이용한 셰이딩(Shading with Directional Occlusion)

방향성 폐색을 인코딩 하는 방법이 다양하므로, 셰이딩 방법을 하나로 정해서 제시할 수는 없습니다.
해법은 구체적으로 어떤 효과를 구현하려는지에 따라 달라지지만, 현대 게임엔진등에서는 아이디어를 차용할 뿐, 방향성 폐색을 직접 사용하는 경우는 드뭅니다.

* 언리얼의 경우 DFAO를 지원했지만, Lumen에서 GI계산 과정에 개념이 통합되었음

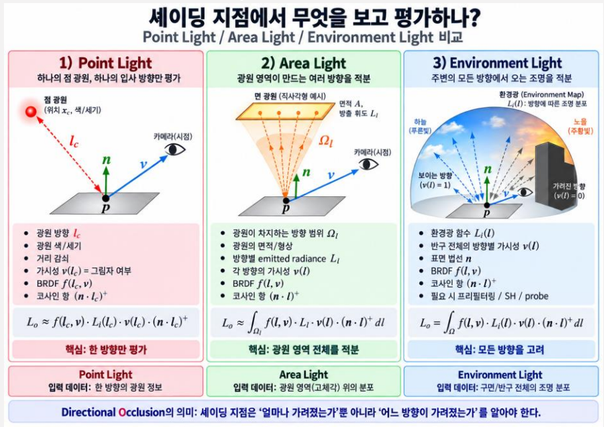
서적에서 다룬 구체적인 use case들은 생략하고, 이 파트에서 다룬 핵심 아이디어들만 짚고 넘어갑니다.

$$L_o(v) = \int_{l \in \Omega} f(l,v) L_i(l) v(l) (n \cdot l)^+ \, dl$$

출력 = 재질함수 × 입사광 × 가시성 × 입사각

* 수식 (11.26), 반사율 방정식

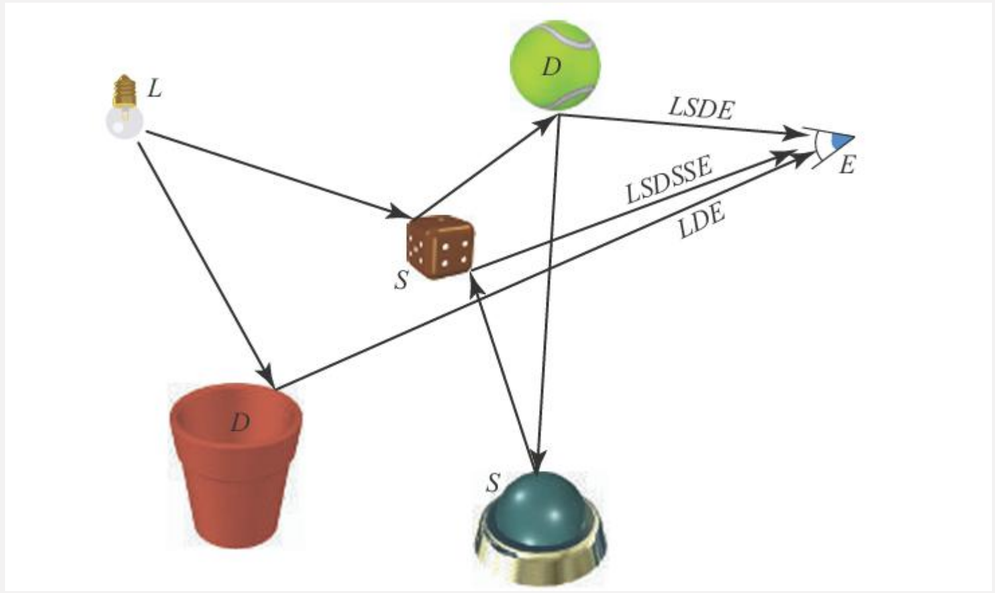
1. **Point light의 경우** 한 방향의 visibility만 보면 되기 때문에 가시성 함수의 평가가 쉽습니다. (단일 방향의 가시성만 고려하면 됨)
2. **Area light의 경우 visibility 평가가 적분**으로 평가됩니다. (Lambertian 표면의 경우 가시성과 입사각만 고려하면 됨)
   - 가시성과 입사각의 평가도 Cone이나 구면 조화 함수 기반의 overlap평가가 가능 (방향성 폐색의 동적계산 파트)
3. **환경(Environment)광의 경우 Lighting, Visibility, Cosine 계수를 모두 적분**해야 합니다 (입사광, 가시성이 입사각에 따라 모두 달라질 수 있음)
   - 적분을 더 싼 연산으로 바꾸거나, 복잡한 함수를 압축/손실된 표현으로 바꾸는 등의 노력이 필요 (11.5 파트부터 구체적으로 다룹니다)
4. 많은 정보를 Precompute 할수록 런타임이 빠르지만 자유도가 줄어듭니다.
   - Visibility와 cosine을 미리 결합하면 normal 정보까지 bake되어 런타임에 normal을 바꿀수 없다.
   - 결합될 수 있는 정보를 LookUp해두는건 런타임도 빠르고 자유도도 챙기지만 메모리가 필요하다
5. **Diffuse에서는 BRDF를 상수로 처리할 수 있었지만 Glossy한 재질등의 specular표현은 BRDF lobe와 함께 표현해야** 합니다.
   - BRDF lobe를 cone으로 근사하고 visibility도 cone으로 근사해서 overlap 하는 등의 계산을 시도할 수 있음



* AI 한테 시키면서 느끼는건데, GPT images 2.0은 normal, tangent 표현을 진짜 개 못해서 열불이 나요
* 하지만 감사합니다 직접 그릴 생각하면 까마득하다

Heckbert 표기법

11.1 장에서 생략하고 지나갔던 Heckbert 표기법을 보고
지금까지 다룬 내용과 앞으로 다룰 내용이 각각 어떤 빛의 경로를 표현하는지를 쉽게 표현해봅시다.





* 와! 태초마을이야!
* 아닙니다 잠깐 들렀다 가는거에요

11.1 Heckbert’s notation

Heckbert 표기법(Heckbert's notation)

Heckbert 표기법은 렌더링 과정에서 빛이 광원 (L)에서 출발해서 눈 (E)까지 도달할때까지 어떤 종류의 표면 상호 작용을 거쳤는지를 짧은 문자열로 표현합니다.
핵심은 복잡한 전역 조명 알고리즘을 수식 전체로 설명하는 대신, 어떤 light transport path를 계산할 수 있는지를 정규 표현식 형태로 압축한다는 점입니다.

L : Light (빛이 출발하는 광원)
E : Eye (최종적으로 빛을 관찰하는 카메라 또는 눈)
D : Diffuse interaction (난반사 표면에서의 상호작용)
S : Specular interaction (정반사 표면에서의 상호작용)
필요에 따라서 더 세분화된 표면 종류를 추가할 수도 있음

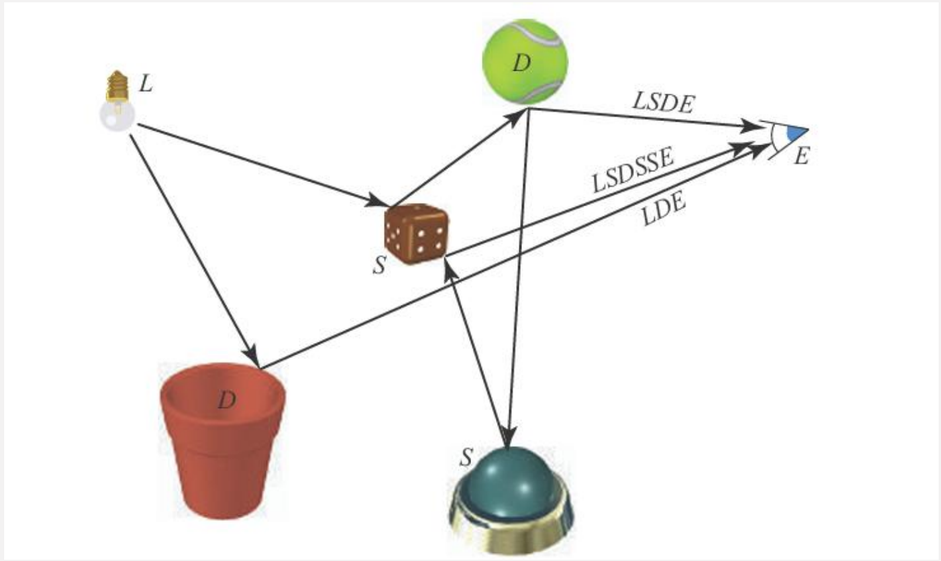
기본적으로 빛이 광원에서 눈까지 도달하는 경로를 다루기 때문에, L로 시작해서 E로 끝나는 표현 방식이 됩니다.

다음의 예시들을 보며 빛의 경로를 어떻게 표현하는지 확인해봅시다.

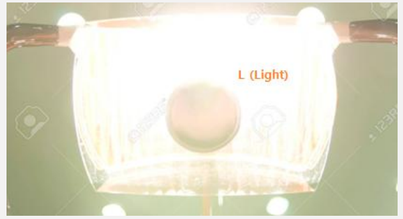
Heckbert 표기법(Heckbert's notation)

LE : 가장 간단한 경로, 직접 광원을 관찰합니다
LDE : 난반사 표면에서 반사된 빛을 관찰합니다
LSE : 정반사 표면에서 반사된 빛을 관찰합니다

LDSE : 난반사 표면에서 반사된 빛이 정반사 표면에 반사된 뒤 관찰
LSSSSSSSSE : 거울에비친거울에비친거울에비친....



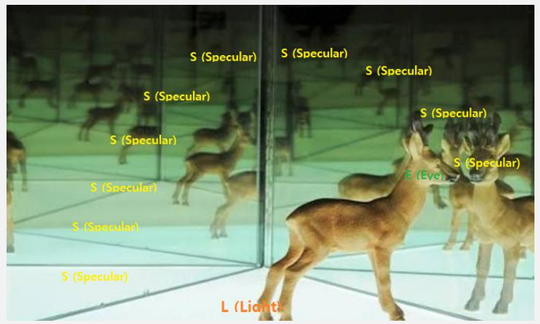












리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

Heckbert 표기법(Heckbert's notation)

표기법으로 뇌절하라고 쓴 게 아니라 정규식 표현법으로 간결하게 표현하기 위함입니다.
또한 다양한 빛의 경로를 하나의 표현식으로 작성할 수 있게 되어
알고리즘이 어떤 빛의 경로를 다루고자 하는지 이해하고
해당 알고리즘의 의의나 특이 사항을 보다 쉽게 이해할 수 있게 됩니다.

Operator	Description	Example	Explanation
$*$	zero or more	$S*$	zero or more specular bounces
$+$	one or more	$D+$	one or more diffuse bounces
$?$	zero or one	$S?$	zero or one specular bounces
$\|$	either/or	$D\|SS$	either a diffuse or two specular bounces
$()$	group	$(D\|S)*$	zero or more of diffuse or specular

Table 11.1. Regular expression notation.

Full Rendering Equation : L(D|S)*E
10장 지역 조명 (별도 매핑 참조 없음) : L(D|S)E
10장 Environment mapping : L(D|S)가 저장되고, 런타임에는 SE
10장 Irradiance Environment Mapping : L(D|S)가 저장되고, 런타임에는 DE
Radiosity : LD*E
Path Tracing : L(D|S)*E
Ambient Occlusion을 사용한 diffuse shading : LDE
상호반사를 고려한 AO : 표현하려는 것 $LD^{+}E$, 런타임에는 DE
Directional Occlusion을 사용한 diffuse shading : LDE

지금까지 본 여러 기법들은 모두 전체 렌더링 방정식 L(D|S)*E의 일부 경로를 계산하거나 근사합니다.
여기서 중요한 구분은, 중간 상호작용이 diffuse인지 specular인지에 따라 필요한 정보의 성격이 크게 달라진다는 점입니다.

확산 전역 조명 (Diffuse Global Illumination)

빛의 차폐만을 다룬 11.3, 11.4 파트를 지나 이제 빛의 반사를 실시간으로 시뮬레이션 하는 여러 방법을 다룹니다.

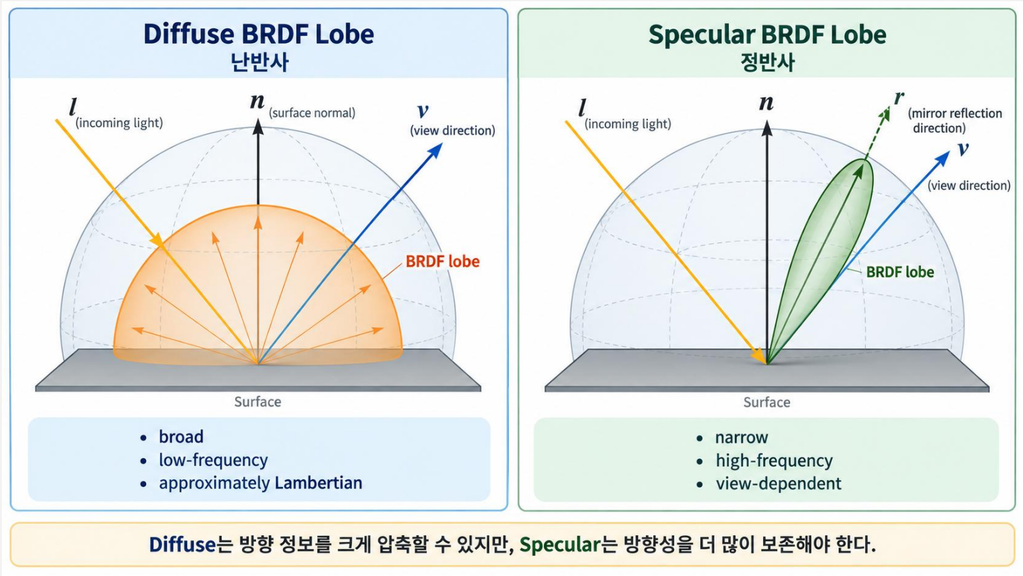
이 방법들은 빛이 눈에 들어오기 직전에 Diffuse 표면 (난반사 표면)에서 반사되는지
아니면 Specular 표면 (정반사 표면)에서 반사되는지를 가정하는 두 부류로 나눌 수 있습니다.

이에 대응하는 빛 경로는 각각 L(D|S)*DE와 L(D|S)*SE로 쓸 수 있으며
많은 기법이 눈으로 들어오기 가장 직전에 반사된 표면의 종류에 따라 일정한 제약을 둡니다.

첫 번째 부류 (시야에 들어오기 전 최종 반사 표면이 Diffuse)는 주로 셰이딩 지점 위 반구 전체의 입사 조명이 부드럽게 변한다고 가정하거나, 그 변화를 무시하고,
두 번째 부류 (시야에 들어오기 전 최종 반사 표면이 Specular)는 입사 방향에 따른 변화율이 크고, 조명이 비교적 작은 입체각 안에서만 참조된다는 사실에 의존합니다.

두 부류의 제약이 이처럼 크게 다르기 때문에, 이들을 따로 다루는 편이 유리합니다.

11.5 파트에서는 확산 전역 조명을,
11.6 파트에서는 스페큘러 표면을 다룬뒤
11.7 파트에서 전망이 밝은 통합적 접근을 살펴보게 됩니다.



표면 사전 조명(Surface Prelighting)

* 4차시에 다룰 내용이지만, 하이엔드 PC나 최신 콘솔 타겟 기기의 경우
많은 아이디어를 차용하고 온몸을 비틀어 어느정도? 성숙해진 기술들이 있습니다.

Radiocsity와 path tracing은 둘 다 오프라인 용도로 설계된 기법입니다.
이들을 실시간 환경에서 사용하려는 시도가 있어왔지만, 그 결과를 실제 제품에 쓰기에는 충분히 성숙하지 않습니다.

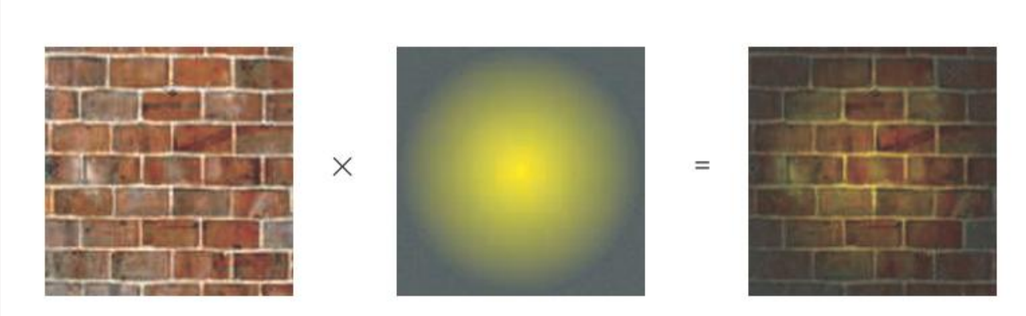
일반적인 방식 중 하나는 이 기법들을 조명 관련 정보를 미리 계산(precompute)하는데 사용하는 것입니다.
비용이 큰 오프라인 처리를 사전에 수행해두고, 그 결과를 저장했다가 화면에 출력할 때 사용하여 고품질 조명을 얻는 식입니다.
AO에서도 다루었듯이, 정적인 장면에 대해 이런 방식으로 미리 계산해 두는 것을 베이킹 (Baking)이라고 합니다.

AO때도 그랬듯이 구우면 런타임에 장면 구성을 변경할 수 없습니다.
모든 장면 지오메트리, 광원, 재질이 유지되어야 합니다.

이런 환경에서 동적인 객체를 구현한다면 지오메트리를 정적(static)인 객체와 동적(dynamic)인 객체로 나눌 수 있습니다.
정적 객체는 사전 계산 과정에 포함되어 조명과 완전히 상호작용하고 (정확히는 상호작용한 데이터를 구워두고)
동적 객체는 수신자 역할만 하게 됩니다. (빛을 차단하지도, 간접 조명 효과를 만들어내지도 않는다)

표면 사전 조명(Surface Prelighting)

미리 계산할 수 있는 조명 정보중, 가장 단순한 형태는 irradiance입니다.
평평한 램버시안 표면이라면 표면의 색상과 irradiance 값 만으로도 조명에 대한 재질의 반응을 완전히 표현할 수 있습니다.
또한 어떤 광원의 기여는 다른 광원과 독립적이기 때문에, 미리 계산된 irradiance 위에 동적 광원을 추가로 얹을 수도 있습니다.



1996년의 Quake와 Quake II는 irradiance 를 미리 계산해 활용한 최초의 상용 인터랙티브 애플리케이션이였습니다.
Quake는 주로 성능 향상을 위해 정적 광원의 직접광 기여분을 미리 계산했고,
Quake II는 여기에 간접광 성분까지 포함시켜 GI 알고리즘으로 더 사실적인 조명을 만들어낸 최초의 게임이 되었습니다.

이 게임은 radiosity 기반 알고리즘을 사용했는데, 램버시안 환경에서 irradiance를 계산하는데 잘 맞았고, 특유의 흐릿하고 저주파적인 그림자와도 어울렸습니다.

표면 사전 조명(Surface Prelighting)

미리 계산된 irradiance값은 보통 별도의 텍스쳐 집합에 저장된 diffuse 또는 albedo맵과 곱해집니다.
이론상으로는 방사도(irradiance * diffuse color)를 **미리 계산해 하나의 텍스쳐 집합에 저장할 수도 있지만, 실무적인 이유로 이 방식은 배제**됩니다.
색상 맵은 보통 주파수가 상당히 높고 각종 타일링을 사용하며 모델 곳곳에서 일부를 재사용합니다.
반명 irradiance값은 주파수가 훨씬 낮고, 쉽게 재사용할 수 없습니다.

오늘날 이 방식은 제약이 극심한 하드웨어 플랫폼을 제외하고는 사용되지 않습니다.
Irradiance는 정의상 특정 법선 방향에 대해 계산되기 때문에, normal mapping을 사용해 고주파 디테일을 표현할 수 없기 때문입니다.



* irradiance는 단위 면적당 표면에서 **받는** 에너지 양

오늘의 순서 - 4차시

00 확산 전역 조명 — 11.5

*아... 결국 5차시를 가는구나...

01 반사 전역 조명 — 11.6

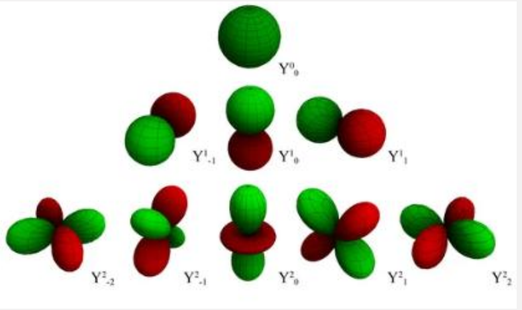
*아마도 오늘 못합니다 6차시 가니...?

방향성 표면 사전 조명(Directional Surface Prelighting)

Lambertian 표면에서 사전 조명과 normal mapping을 함께 쓰려면, irradiance가 법선에 따라 어떻게 변하는지를 표현할 방법이 필요합니다.
또는 동적인 지오메트리에 간접광을 제공하기 위해서는, 가능한 모든 표면 방향에 대한 값이 필요합니다.

하지만 이미 우리는 10.3 (지역조명 – 구면과 반구면 함수)에서 법선 방향에 달라지는 조명을 구하는 여러 방식을 보았습니다.

가장 일반적인 방법은 구면조화 함수 (SH)를 사용해 구면의 irradiance를 저장하는 것입니다.
구면 조화 계수 개수를 9개 (3차 SH) 사용하면 품질은 훌륭하지만 저장 공간과 대역폭이 비싸지고,
4개 (2차 SH)만 쓰면 디테일이 사라지고 조명의 대비가 약해지며, normal map도 덜 두드러지게 됩니다.



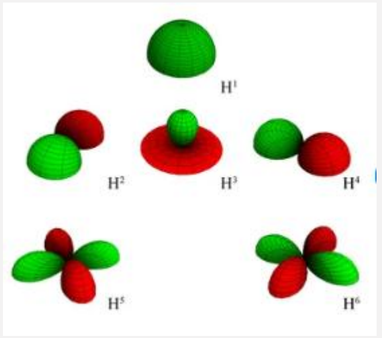
Chen은 Halo 3에서 이 방법의 변형을 사용했는데, 더 낮은 비용으로 3차 SH 수준의 품질을 얻기 위해
구면에서 가장 지배적인 광원을 뽑아 색과 방향을 따로 저장하고, 잔차를 2차 SH로 인코딩했습니다.

이렇게 하면 스칼라 계수가 27개 ($9_{(3차\ SH)} * 3_{(RGB)}$)에서 18개 ($4_{(2차\ SH)} * 3_{(RGB)} + 3_{(RGB)} + 3_{(Direction)}$)로 줄어들며 품질 손실은 거의 없습니다.

방향성 표면 사전 조명(Directional Surface Prelighting)

Habel등의 H-Basis는 또 다른 대안입니다. 이는 반구 신호만 인코딩 하기 때문에, 구면 조화 함수와 같은 정밀도를 더 적은 계수를 사용해 얻을 수 있습니다.
계수 6개 만으로 3차 SH에 필적하는 품질을 낼 수 있으나, 반구에 대해서만 정의되기 때문에 이를 올바르게 표현하기 위해서는
표면상의 지역 좌표계 (tangent space)가 필요합니다.

H-basis성분을 텍스쳐에 저장한다면, 그 해상도는 기저를 이루는 탄젠트 공간의 변화를 따라갈 수 있을 만큼 충분히 높아야 합니다.
탄젠트 공간이 크게 다른 여러 삼각형이 같은 텍셀을 덮으면 복원된 신호가 정확하지 않게 됩니다.



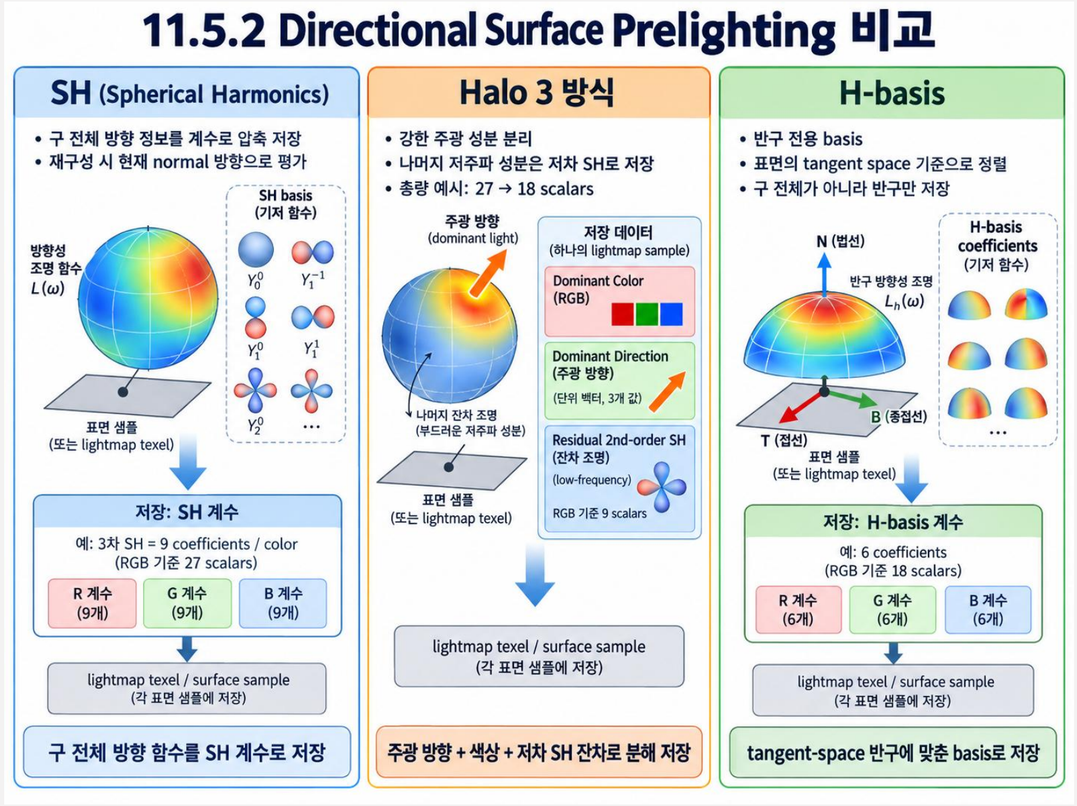


리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

방향성 표면 사전 조명(Directional Surface Prelighting)

구면 조화 함수 (SH)와 H-basis 모두 링잉 현상이 나타날 수 있다는 문제가 있습니다.
프리필터링으로 이 효과를 완화할 수 있지만, 그 과정에서 조명도 함께 부드러워지는데 이것이 늘 바람직하지는 않습니다.

게다가 비용이 덜한 변형들조차 저장공간과 연산 양쪽에서 부담이 상당하게 됩니다.
저사양 플랫폼이나 VR 렌더링 처럼 제약이 심한 경우, 이 비용을 감당하기 어려울 수 있습니다.

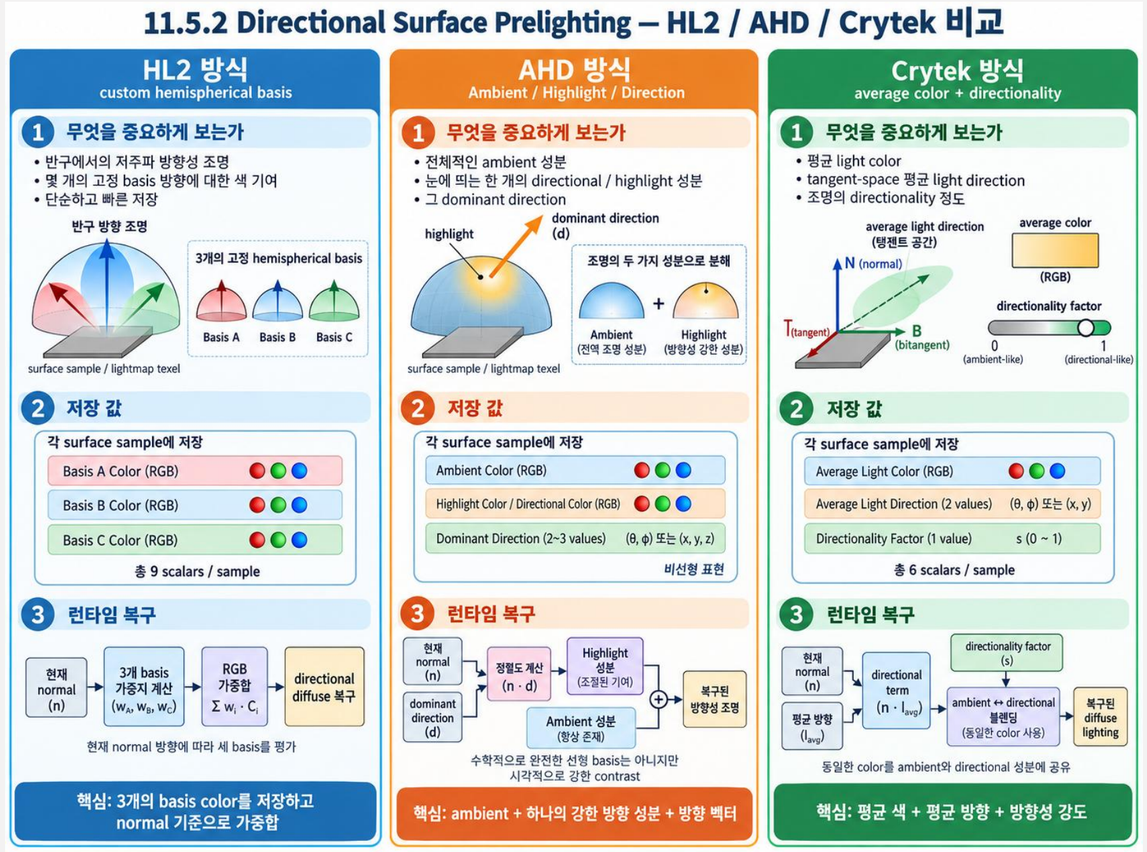


방향성 표면 사전 조명(Directional Surface Prelighting)

구면/반구면 기저의 비용 문제 때문에 더 단순한 대안들도 쓰이곤 합니다.
Half-Life 2는 세개의 색상값, 즉 샘플당 총 9개의 계수를 저장하는 맞춤형 반구 기저 (10.3.3)를 사용합니다.

또한 Ambient/HighLight/Direction(AHD)기저 역시 단순함에도 인기 있는 선택지입니다.

Crytek은 Far cry에서 이 방식의 변형을 사용하였고, 유니티의 LightMap도 이와 비슷한 형태의 구현을 포함하고 있습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

방향성 표면 사전 조명(Directional Surface Prelighting)

Half-Life 2, AHD, Crytek의 표현방법은 비선형입니다.
따라서 엄밀히 말하면 텍셀, 정점정보등을 통해 이 사이의 개별 성분을 선형보간 하는 것은 수학적으로 옳지 않습니다.
예를 들어 그림자의 경계처럼 지배적 광원의 영향이 급격히 바뀌는 곳에서는 시각적인 아티팩트가 나타날 수 있습니다.

하지만 이러한 부정확성에도 불구하고 결과물은 보기에 좋습니다.

Ambient영역과 방향성 조명 영역 사이의 대비가 크기 때문에 노멀 맵의 효과가 강조되는데, 이것은 종종 바람직한 결과가 됩니다.

덧붙여 지배적인 조명의 방향을 표현하는 성분은 BRDF의 specular 반응을 계산할 때도 활용할 수 있어,
광택이 낮은 재질에 대해 환경 맵을 대신할 저비용 대안이 될 수 있습니다.

방향성 표면 사전 조명(Directional Surface Prelighting)

저비용 처리 방법 대신 고비용 처리 방법도 알아봅시다.
디 오더 : 1886에서는 표면의 탄젠트 표면에서 입사 방향으로의 radiance를 표현한 가우시안 로브 집합 (10.3장에서 다루었던)으로 표현합니다.
장면에서 사용되는 조명의 복잡도에 따라 5~9개의 로브를 사용해 텍스쳐 맵에 저장하고
Diffuse 값을 계산 할때 각각의 구면 가우시안을 표면의 normal(법선) 방향의 코사인 로브와 곱하여 표현합니다.

이 표현은 Diffuse 뿐 아니라 스페큘러 BRDF로브와도 결합하여 흐린 광택 효과를 낼 수 있을만큼 정밀합니다.



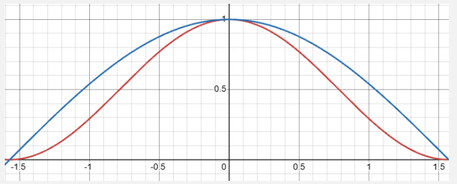
방향성 표면 사전 조명(Directional Surface Prelighting)

표면 위쪽 반구 뿐 아니라 임의의 방향에 대한 조명 정보가 필요하다면 (동적인 물체에도 간접광을 제공하려면)
반구가 아니라 전체 구 방향에서 오는 값을 인코딩 할 수 있어야 합니다.
SG lobe를 반대쪽 반구 방향에도 배치하여 이를 해결할 수 있겠지만, 2배 더 많은 비용이 필요해지는 문제가 생깁니다.

이 비용을 해결하기 위해 단일 구면 조화 함수에 인코딩하는건 링잉 현상이 눈에 띄고, 이럴때 사용되는 것이 10.3장에서 다루었던 Ambient Cube입니다.

이 6개의 방향 정보에 clamped $\cos^2$ lobes를 배치합니다.

런타임에서는 반구방향의 값만 평가하면 되기 때문에, 3개의 로브만 평가하게 되어 비용을 절감할 수 있게 됩니다.
이 Ambient Cube의 복원 품질은 2차 구면 조화 함수 (SH)와 비슷한 수준입니다.

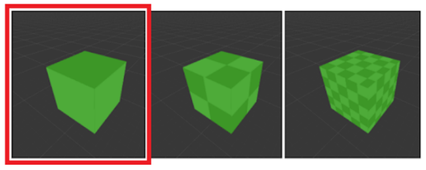


*왜 $\cos^2$인지는 파악 못했습니다
*그냥 방향에 대한 가중치를 cos보다 더 강하게 주기 위함인듯
*큐브 형태니까 뭔가 감은 오는데 제가 수알못이라

Ambient cube

- 각 큐브 면에 텍셀 하나가 대응되는 cube map과 비슷하게 볼 수 있다.
  + Sloan은 ambient cube와 spherical harmonics 사이의 변환식을 유도하였음
- 어떤 방향 d에 대해 평가할 때 d의 x, y, z 부호에 맞는 세 면만 읽으면 된다.
  + 기하 구조 상 최대 세 개의 면까지만 고려가 가능하기 때문
- 샘플이 극단적으로 적은 환경이나 마찬가지이기 때문에, 품질은 낮은 편이다.
  + 큐브의 여섯 면 대신 여덟 개의 꼭짓점을 보간하여 개선하는 우회법이 있음

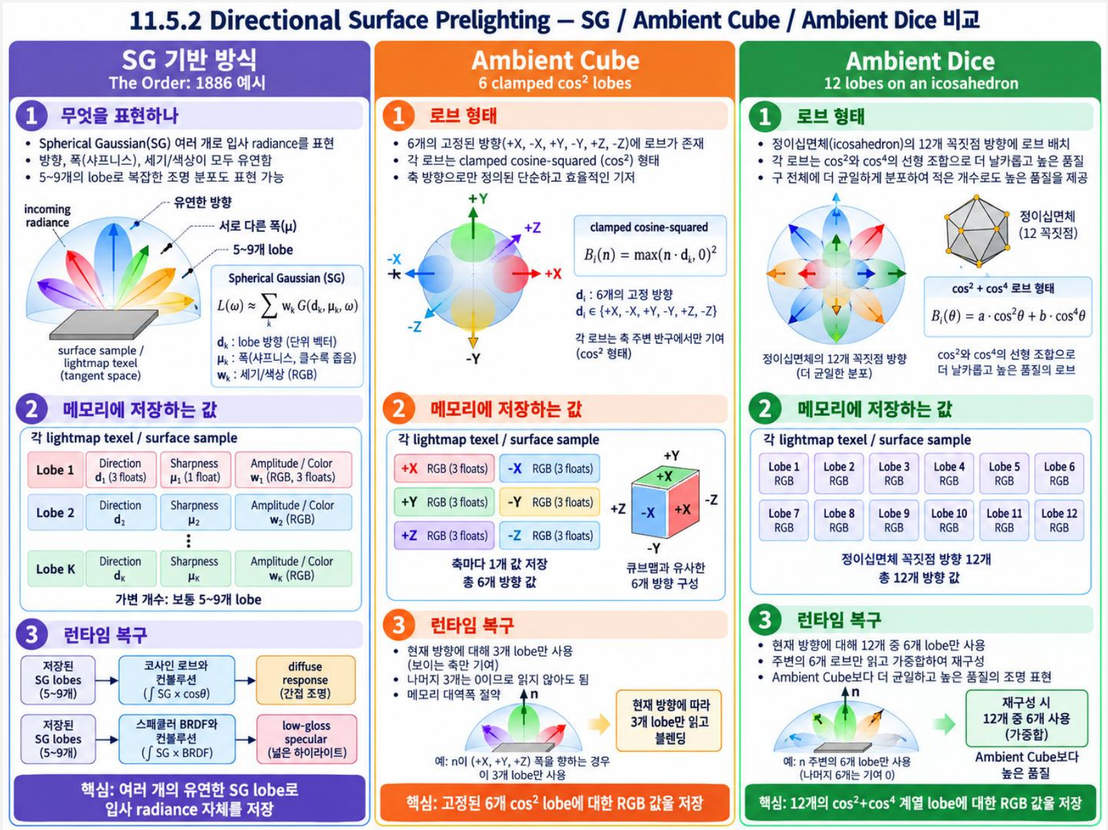
Lightmapping Settings(라이트매핑 설정)에서 **Lightmap Resolution**(라이트맵 해상도) 매개변수는 라이트맵에서 단위당 사용되는 텍셀 수를 제어합니다. 다음은 다른 **Lightmap Resolution**(라이트맵 해상도) 설정이 사용된 정육면체의 예입니다. 해상도가 높아지면 할 일이 많아짐을 확인할 수 있습니다.

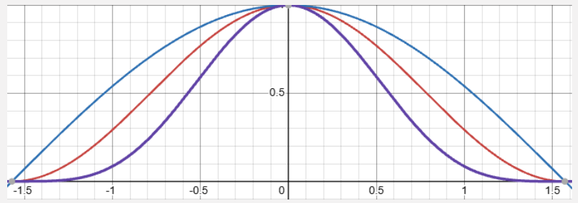


방향성 표면 사전 조명(Directional Surface Prelighting)

Ambient Cube보다 더 높은 품질이 필요하다면 Ambient Dice를 사용할 수 있습니다.
이 방식은 정이십면체의 꼭짓점 방향으로 정렬된 12개의 로브를 저장하며, 각 로브는 $\cos^2$와 $\cos^4$ 의 선형 결합입니다.
Ambient Cube가 그랬듯이, 반구 방향으로만 평가하면 되기 때문에 6개의 로브만 읽고 가중합으로 색상을 평가할 수 있게 됩니다.

이런 Ambient Cube의 복원 품질은 3차 구면 조화 함수에 견줄만 하다고 합니다.





*얘도 모르겠다! 면적이 좁아진만큼 더 방향에 대한 가중치가 강하게 들어가나

미리 계산된 전이 함수(Precomputed Transfer)

미리 계산된 조명은 대단히 훌륭해 보이지만, 본질적으로 정적씬을 구워둔 것입니다.
지오메트리나 조명에 조금이라도 변화가 생기면, 구워둔 전체 값이 무효가 될 수 있습니다.

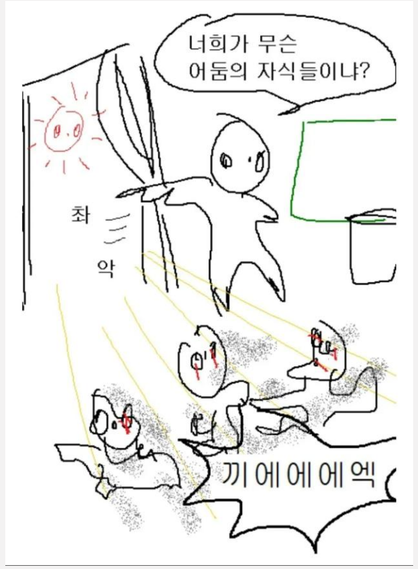
씬의 지오메트리가 변하지 않고 조명만 변한다고 가정하면,
빛과 모델이 어떻게 상호작용하는지를 미리 계산해둘 수 있습니다.

이런 입사 조명을 받아 장면 전반의 입사광 분포로 변환하는 함수를 전이함수 (transfer function)이라고 합니다.
이를 미리 계산하는 방법을 사전 계산 전이(precomputed transfer), precomputed radiance transfer(PRT) 방식이라고 부릅니다.

조명을 오프라인에서 완전히 베이킹 하는것과 달리, 이 파트부터 설명할 기법은 무시할 수 없는 런타임 비용을 수반합니다.

장면을 화면에 표시할 때 특정 조명 구성에 대한 radiance값을 계산해야 하기 때문입니다.
이를 위해 실제 직접 광의 양을 시스템이 주입(inject)한 뒤, 전이 함수를 통해 그것을 장면 전체로 전파합니다.

일부 기법은 이 직접광이 환경 맵에서 온다고 가정하며, 다른 방식들은 조명 구성을 임의로 두고 유연하게 변경할 수 있게 합니다.



* 어두운 실내에서 커튼이나 문을 열어 방 전체가 빛으로 가득 차는 모습

미리 계산된 전이 함수(Precomputed Transfer)

PRT 개념을 그래픽스에 도입한건 Sloan등입니다. 이들은 이를 구면 조화 함수의 관점에서 설명했지만, 이 방법이 반드시 SH를 사용해야 하는 것은 아닙니다.

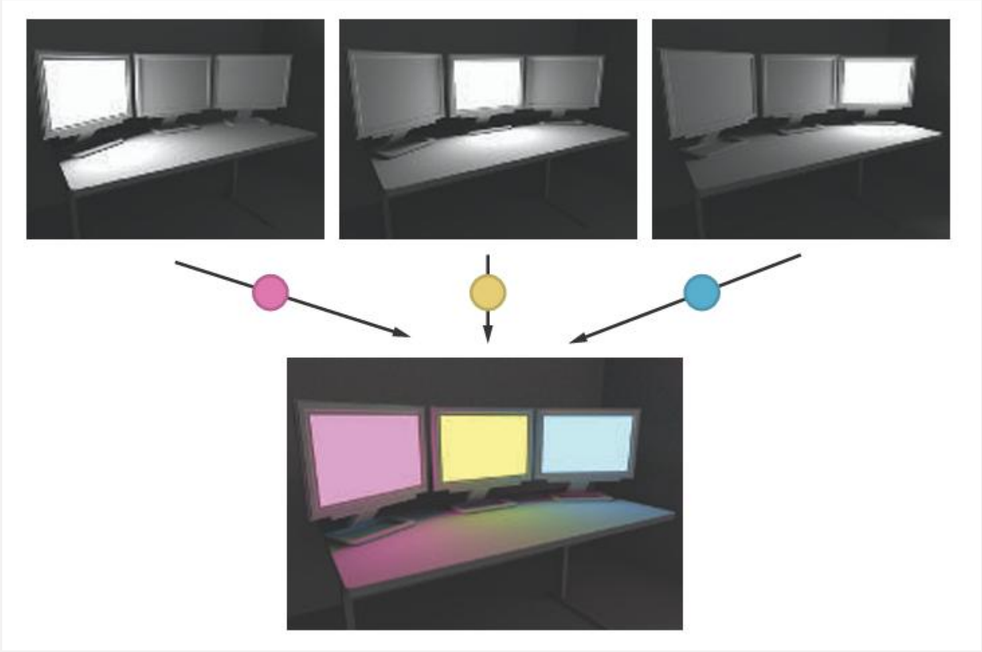
직접광을 (되도록이면 적은 개수의) 광원으로 기술 할 수 있다면, 각 광원이 장면을 어떻게 비추는지를 미리 계산해둘 수 있습니다.
아래 예시 이미지를 보도록 합시다.

각 화면의 모니터는 단일 색상만 표현할 수 있고, 밝기는 0에서 1까지로 제어할 수 있다면 각 모니터가 방에 미치는 영향을 독립적으로 미리 계산 할 수 있습니다.
빛의 전달과 합산은 선형 연산이기 때문에 세 모니터를 모두 켰을때 장면이 받는 조명은 각 모니터에서 오는 빛 (직, 간접광 모두)의 합과 같습니다.

각 모니터의 조명은 다른 해에 영향을 주지 않기 때문에,
한 화면의 밝기를 절반으로 낮추더라도 그 화면 자신의 기여분만 달라집니다.

미리 계산된 조명 해에 해당 화면의 밝기를 곱하고, 그 결과들을 더하기만 하면 됩니다.

모니터를 켜고 끄거나, 더 밝거나, 더 어둡게 하거나, 색을 바꾸더라도
최종 조명 결과를 얻는데 필요한 것은 곱셈과 덧셈 뿐입니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

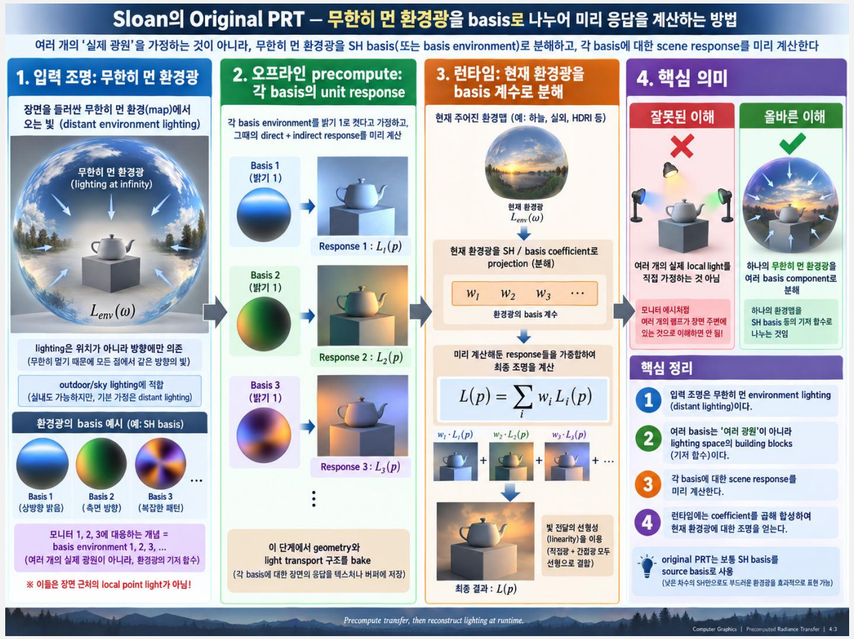
이런 미리 계산된 전이 함수를 사용해 최종 radiance를 계산하는것을 아래와 같이 수식으로 표현할 수 있습니다.

$$L(p) = \sum_i L_i(p) w_i$$

$L(p)$ = 점 p에서 최종 radiance $L_i(p)$ = 화면 i로부터의 미리 계산된 단위 기여분 w_i = 화면 i의 현재 밝기

이 식은 수학적 의미에서 벡터 공간을 정의하고, Li가 이 공간의 기저 벡터가 됩니다.
가능한 모든 조명은 각 광원 기여분의 선형 결합으로 만들어 낼 수 있습니다.

Sloan등의 원 PRT논문도 동일한 논리를 사용하지만, 구면 조화 함수로 표현된 무한히 먼 조명 환경이라는 맥락에서 다룹니다.
장면이 모니터 화면에 어떻게 반응하는지를 저장하는 대신, 환경광을 여러 개의 구면 조화 기저 함수로 분리하여 계산하는 방법을 다루었습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

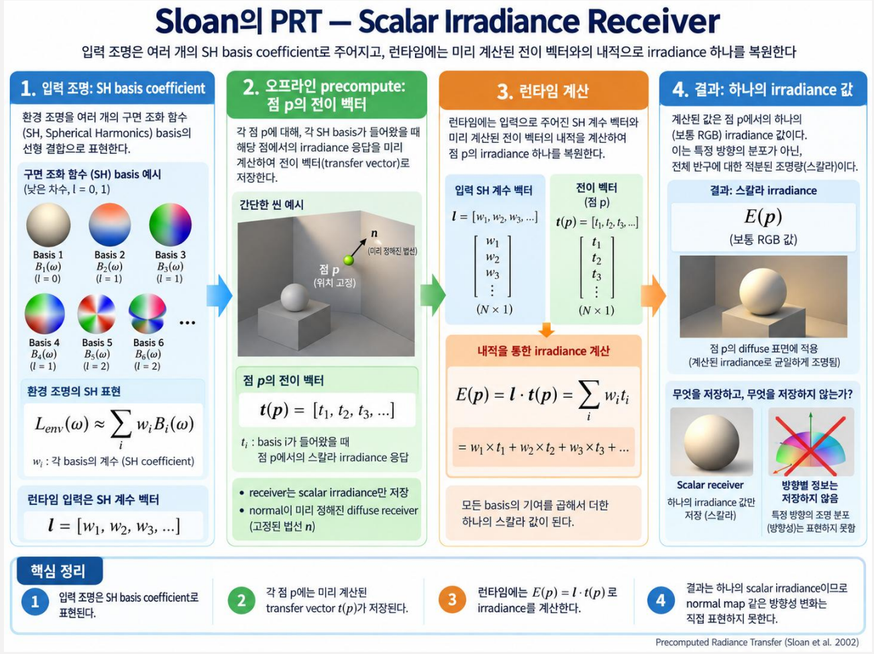
장면에 빛을 주입하는데에 쓰이는 기저의 선택은, 최종 조명을 표현하는데에 쓰이는 표현 방식과 무관합니다.

예를 들자면 장면에 광원이 어떻게 조명되는지는 구면 조화 함수를 사용하면서, 임의의 점에 도달하는 radiance의 양을 저장하는 데에는 다른 기저를 고를 수 있습니다.
저장에 ambient cube를 사용했다면 위쪽에서 오는 radiance와 옆면에서 오는 radiance를 계산하게 됩니다.
이 각 방향에 대한 전달이, 총 전달을 나타내는 하나의 스칼라 값이 아니라 따로 저장되게 됩니다.

* 당장 이해되지 않아도 괜찮습니다. 뒤에 구체적인 예시를 통해 자연스럽게 둘을 구분 할 수 있습니다.

Sloan의 PRT논문은 두가지 경우를 분석하였습니다.
첫 번째는 수신 기저 (receiver basis)가 표면의 irradiance 값인 경우입니다.
이때 수신 표면은 법선이 미리 정해진 완전한 확산 표면이여야 하며, 따라서 디테일을 위한 노멀 맵을 사용할 수 없습니다.

전달 함수는 입력 조명의 SH 투영과, 장면 전반에 걸쳐 공간적으로 변하는 미리 계산된 전달 벡터 사이의 내적 형태를 띄게 됩니다.



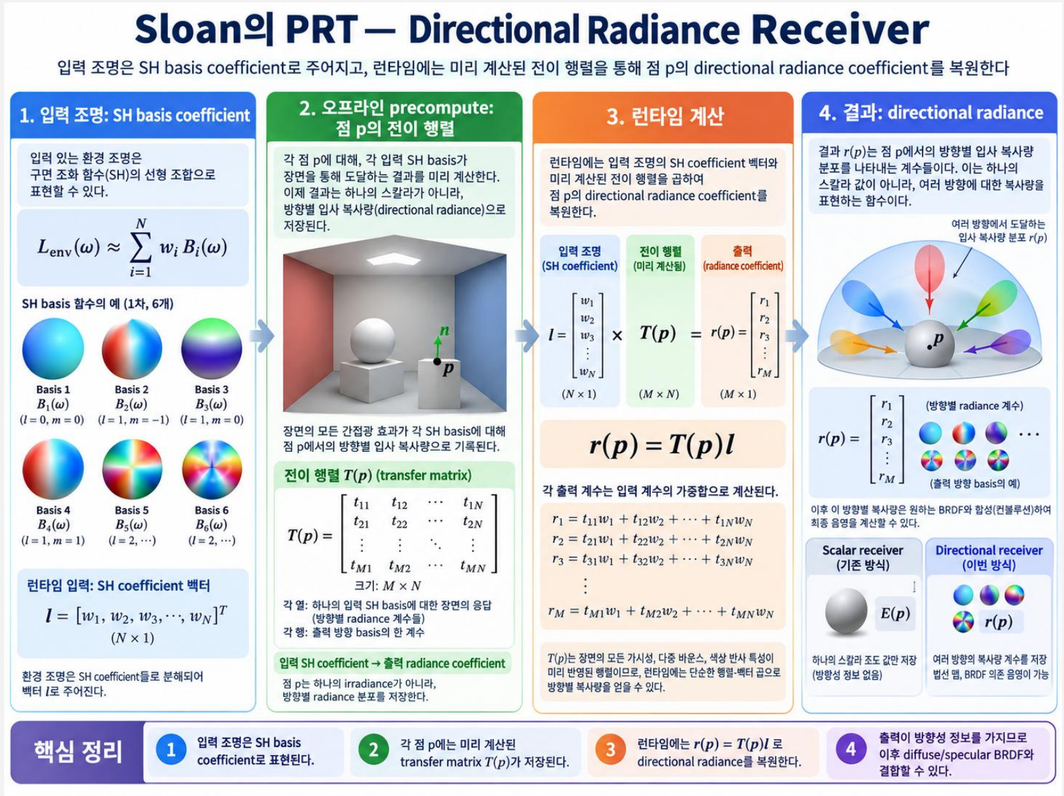
리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

Scalar Irradiance Receiver는 11.5.1장에서 다룬 것 처럼 normal의 변경에도 취약하고, 비 Lambertian 재질을 사용할 수도 없습니다.
논문에서는 이를 해결하기 위해 Receiver basis도 구면조화 벡터를 표현하게 하는 두번째 방법을 제시합니다.

이 연산은 구(또는 반구) 전체에 걸친 완전한 radiance분포를 제공하므로, 어떤 BRDF와도 제대로 연산할 수 있습니다.

이때 전이 함수는 SH벡터를 다른 SH벡터로 매핑하며, 행렬 곱셈의 형태를 가지게 됩니다.



미리 계산된 전이 함수(Precomputed Transfer)

이 곱셈 연산은 계산량과 메모리 양쪽 모두에서 비용이 큽니다. 소스와 수신 기저 양쪽에서 3차 SH를 사용한다면,
단색 표현을 위해 장면의 모든 점마다 9x9 행렬을 저장해야 합니다. RGB처리를 위해서는 한 점당 243개의 scalar값을 저장하게 됩니다.

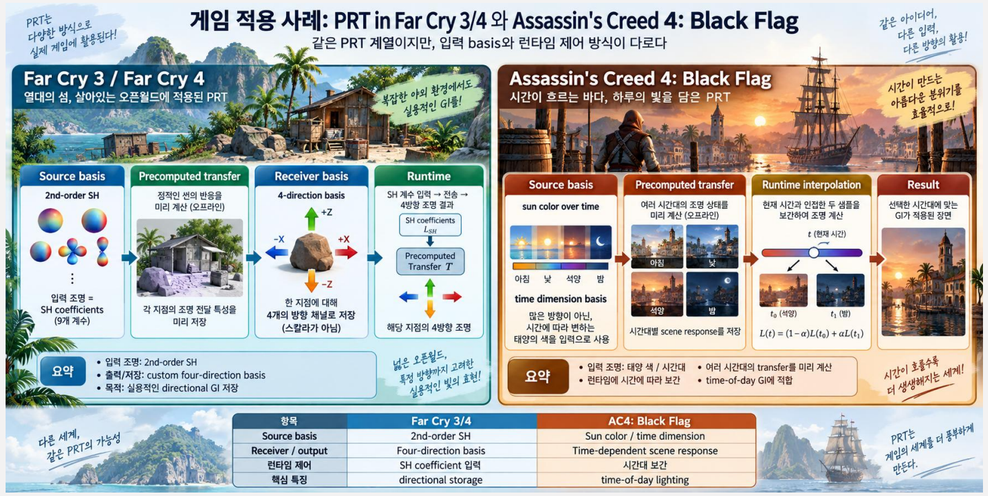
이 문제를 해결 하기 위해 전이 벡터나 행렬을 직접 저장하는 대신, 그 전체 집합을 주성분 분석 기법(PCA)으로 분석하였습니다.

Sloan의 경우는 25x25 transfer matrix (아마도 5차 SH를 사용했나봐 너무 무섭다), 625차원의 데이터를 256차원으로 줄였습니다.
이 정도도 일반적인 실시간 응용에서 사용하기에는 여전히 비싸지만, 이후의 많은 빛 전달 알고리즘이 데이터 압축 수단으로 PCA를 받아들였습니다.

이런 종류의 차원 축소는 본질적으로 손실이 발생합니다. 대개는 근사에 그치므로 어느정도 품질 저하가 발생하고
이를 해결하기 위해 전이 행렬 집합을 여러 군집으로 나누고, 각각에 대해 PCA를 수행하며
군집 경계에서 불연속이 생기지 않도록 하는 최적화 단계도 포함하였다고 합니다.

* PCA에 대한 세부 내용은 다루지 않습니다. 갈길이 멀어욧

PRT는 시간대와 날씨가 동적으로 변하는 야외 지역의 게임플레이 중심인 작품에서 특히 인기가 높았습니다.
Far Cry 3와 Far Cry 4는 Source Basis로 2차 SH를, 수신 기저로는 맞춤형 4방향 기저를 사용하는 PRT를 사용합니다.
Assassin's Creed 4는 Source Basis로 기저 함수 하나 (태양 색상)만 사용하되, 서로 다른 시간대에 대한 전이를 미리 계산하였습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

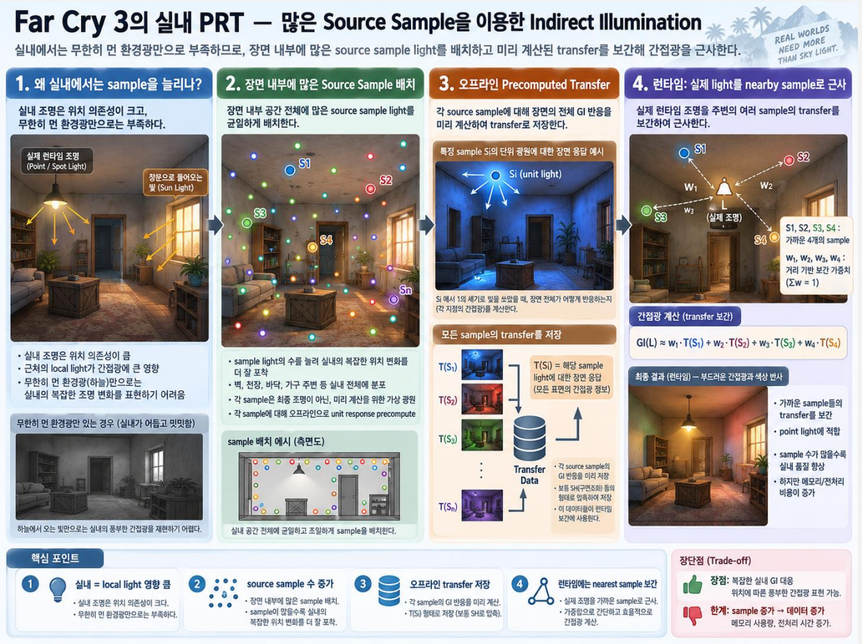
미리 계산된 전이 함수(Precomputed Transfer)

원래의 PRT 기법은 무한히 먼 주변 조명을 가정했습니다. 이는 야외 장면은 꽤 잘 모델링 하지만, 실내 환경에는 지나치게 제약적입니다.
하지만 앞서 모니터의 예시를 들었듯이 이 개념 자체는 조명의 원래 출처가 무엇인지에 완전히 무관합니다.

Kristensen등은 장면 전체에 흩어진 광원 집합에 대해 PRT를 계산하는 방법을 설명했습니다. 이는 Source basis 함수를 아주 많이 두는 것에 해당합니다.
이어서 광원들을 군집으로 묶고, 수신 지오메트리를 여러 구역으로 분할하여 각 구역이 서로 다른 광원 부분 집합의 영향을 받도록 합니다.
이 과정으로 전이 데이터를 크게 압축할 수 있습니다.

런타임에는 임의로 배치된 광원에 의한 조명을 미리 계산된 집합 중 가장 가까운 광원들의 데이터를 보간하여 근사합니다.

Gilabert와 Stefanov는 이 방법을 Far Cry 3의 간접 조명 생성에 활용했습니다.
이는 기본적으로 점 광원만 처리할 수 있으며, 다른 유형의 광원을 지원하도록 확장할 수는 있지만 각 광원의 자유도에 따라 비용이 지수적으로 늘어납니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

지금까지 논의한 PRT 기법들은 광원을 모델링 하는데 쓰이는 몇 개의 요소로부터의 전달을 미리 계산했습니다.
또 다른 기법들은 표면 사이의 전달을 미리 계산합니다. 이런 시스템에서는 조명의 실제 출처가 무관해지게 됩니다.

이들 기법의 입력을 어떤 표면 집합에서 나가는 Radiance (또는 Diffuse Surface를 가정하면 irradiance 같은 관련 물리량) 이므로, 어떤 광원이든 사용할 수 있습니다.

이 직접 조명 계산에는 7장에서 다룬 그림자나, 10장의 irradiance 환경맵, 또는 11장에서 다룬 Occlusion기법을 쓸 수 있습니다.
또한 어떤 표면이든, 나가는 radiance를 원하는 값으로 설정하기만 하면 손쉽게 발광체로 만들어 면광원으로 바꿀 수 있습니다.

이런 surface-to-surface transfer 계열의 가장 유명한 시스템이 Geomerics의 Enlighten입니다.

Enlighten에서는 빛 전달을 계산하기 위해 장면은 램버시안이라고 가정됩니다.
Heckbert의 표기법으로 쓰면 처리되는 경로 집합은 LD*(D|S)E 인데 눈에 들어오기 직전의 마지막 표면은 확산 전용일 필요가 없기 때문입니다.
(light의 transfer는 diffuse로 싸게 계산하지만, 최종 shading 단계에서는 surface의 BRDF를 정확히 반영한다)

https://docs.unity3d.com/2018.2/Documentation/uploads/ExpertGuides/Using_Enlighten_with_Unity.pdf

미리 계산된 전이 함수(Precomputed Transfer)

이런 Enlighten류 시스템은 scene surface들을 Source elements와 Receiver elements로 추상화 합니다.

Source elements는 표면 위에 존재하며, diffuse color, position, normal등 surface property의 일부를 가지고,
Precompute 과정에서는 source element의 light가 얼마나 receiver에게 전달되는지를 계산합니다.

이 정보의 정확한 형태는 소스 요소가 무엇인지, 수신자 측에서 조명을 모으는 데 어떤 기저를 쓰는지에 달려있습니다.
가장 단순한 형태는 소스 요소가 점이고, 수신 위치에서 irradiance를 생성하는 것이 관심사가 됩니다.
이 경우 전달 계수는 단지 소스와 수신자 사이의 상호 가시성 일 뿐입니다.
런타임에는 모든 소스 요소의 나가는 radiance가 시스템에 제공됩니다.
이 정보로부터 미리 계산된 가시성과 source, receiver의 위치, 방향과 같은 알려진 정보를 이용해 반사율 방정식을 수치적으로 적분할 수 있게 됩니다.

이런 방식으로 빛의 한번의 반사를 수행할 수 있으며, 간접 조명의 대부분은 이 첫 반사에서 나타나기 때문에 그럴듯한 조명을 얻을 수 있게 됩니다.

또한 한번 propagation하고 나면, receiver가 받은 indirect light를 통해 이를 다시 source로 두고 다음 프레임에 두번째 반사를 구현할 수 있게 됩니다.

* 당장 이해가 되지 않더라도 넘어갑시다, 이후 개념까지 설명한 뒤에 Radiosity 알고리즘과 비교해보면 조금 더 쉽게 파악할 수 있습니다

미리 계산된 전이 함수(Precomputed Transfer)

하지만 source를 점으로 두게 되면, 연결 수가 대단히 많아지게 됩니다.
성능을 개선하기 위해 법선과 색상이 비슷한 영역을 나타내는 점들의 군집을 소스 집합으로 사용할 수도 있습니다.

이 경우 전이 계수는 Radiosity 알고리즘에서 다루었던 form factor와 같아지게 됩니다.
이는 고전적인 radiosity알고리즘과 유사해보이지만, 이는 점진적 래디오시티 (progressive radiosity)의 아이디어에 기반하여
한 번에 빛의 반사 한 번만 계산하며, 연립 일차 방정식을 직접 풀지 않습니다.

이 방식에서는 하나의 패치가 반복적인 과정을 통해 다른 패치들로 부터 얼마만큼의 에너지를 받는지 결정할 수 있습니다.
저장된 radiance를 수신위치로 전달하는 과정을 gathering이라고 부릅니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

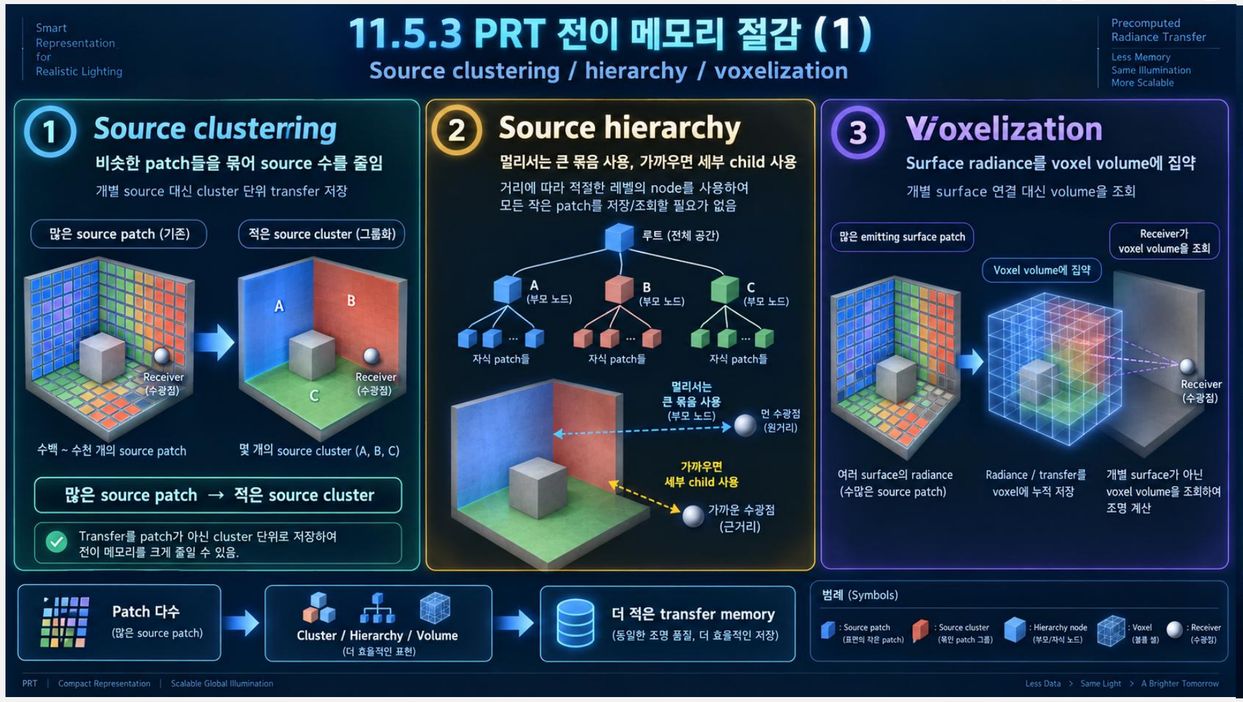
미리 계산된 전이 함수(Precomputed Transfer)

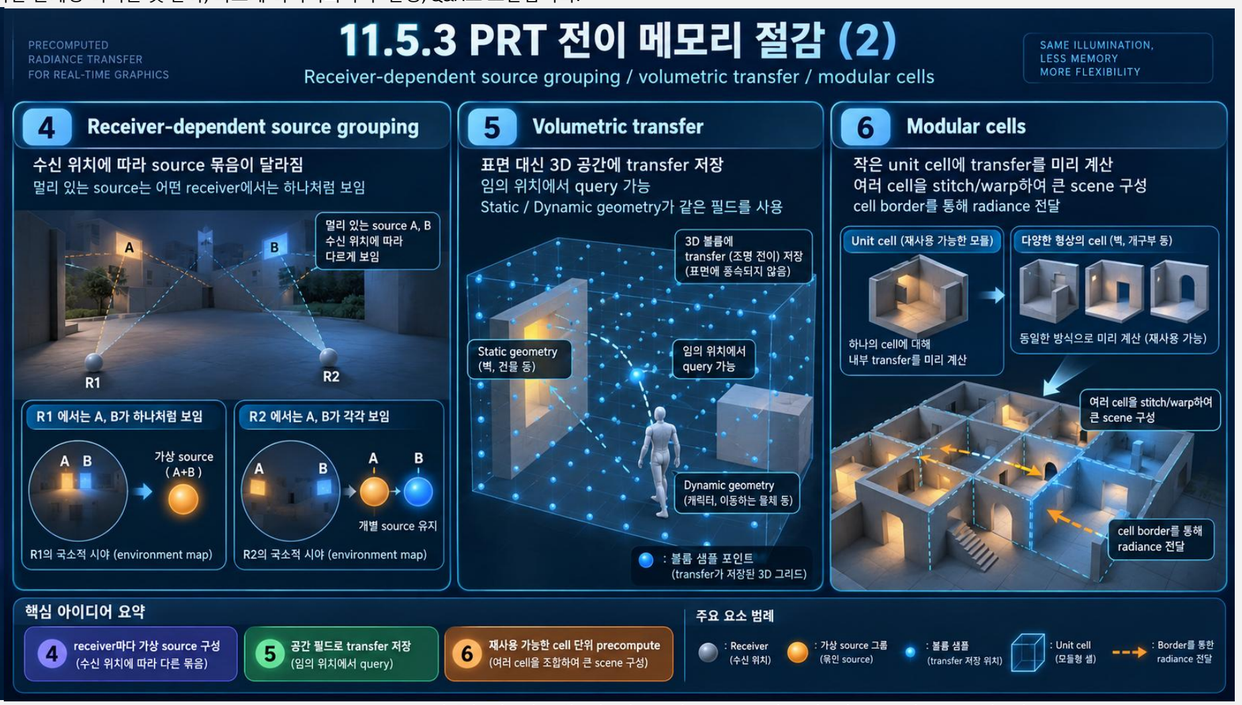
Source를 어떻게 분할하고, 어떻게 빛을 전파할지에 대해서도 다루었지만, Receiver가 어떻게 radiance를 수집할지도 이야기해야 합니다.
source에서 receiver에 전달하는 정보가 scalar하나라면, 다시 normal map을 사용할 수 없는 문제가 생기게 됩니다.

이를 해결하기 위해 receiver가 방향성 기저를 가지게 한다면 11.5.2 (방향성 표면 사전 조명)에서 다룬 것 처럼
노멀 매핑과 함께 사용하거나, 광택이 낮은 스페큘러 반응도 제공할 수 있게 됩니다.

11.5.2 파트의 경우에도 그렇듯이, 이런 방향성 기저 정보를 계산하기 위해서는 많은 양의 메모리와 런타임 연산 자원이 필요해져 이를 어떻게 줄이는지도 다룹니다.

* 세부 내용을 모두 다루기에는 시간 관계상 어려울 것 같아, 빠르게 이미지와 구두 설명, Q&A로 보완합니다.





리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

저장 방법(Storage Methods)

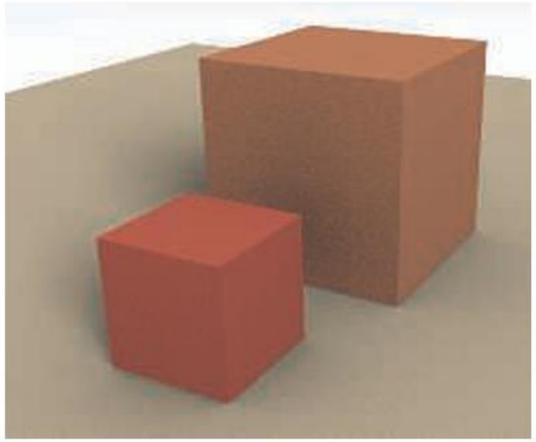
조명을 완전히 계산하는 방법이든, 전달 정보만 사전 계산하고 조명의 변화가 허용되든 그 결과는 어떤 형태로든 저장되어야 하기에 GPU 친화적이여야 합니다.

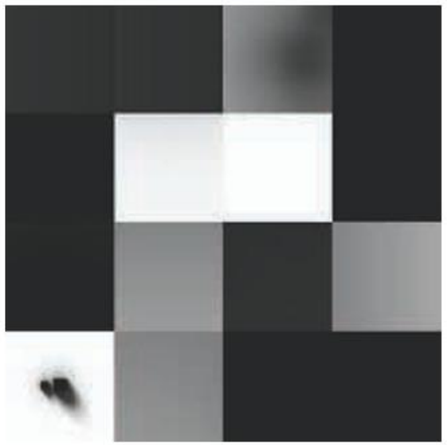
Light map은 미리 계산된 조명을 저장하는 가장 흔한 방법중 하나입니다. light map은 사전 계산된 정보를 담은 텍스쳐로,
저장된 데이터의 구체적인 종류를 가리키기 위해 irradiance map같은 용어를 쓰기도 하지만, 이런 것들을 통칭할 때 light map이라고 부릅니다.
런타임에는 GPU에 내장된 텍스쳐 메커니즘이 활용 됩니다. (irradiance를 저장할수도, SH 계수를 저장할수도, AHD, Ambient Cube, transfer data를 저장할수도 있음)

값은 보통 이중선형(bilinear)필터링을 거치는데, 일부 표현 방식에서는 이것이 엄밀히 옳지 않을 수도 있습니다.
예시로 AHD 표현을 사용하고자 할 때, 텍스쳐에 저장하여 표현하면 필터링된 방향 (Direction) 성분이 보간 후에 더이상 unit vector가 아니게 되어 재정규화가 필요하며
Ambient와 Highlight도 샘플링 지점에서 직접 계산하는 것과는 다른 색상이 나오게 되지만, 표현이 비선형이더라도 결과는 대체로 받아들일 만하게 보입니다.

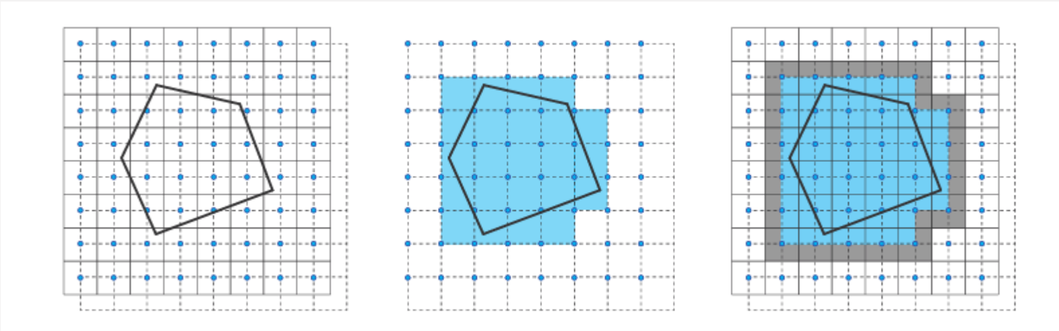
대부분의 경우 라이트 맵은 밉매핑을 사용하지 않습니다. 라이트 맵의 해상도는 일반적인 알베도 맵이나 노말 맵에 비해 작기 때문에 보통은 필요하지 않습니다.
고품질의 프로그램에서도 라이트맵 텍셀 하나가 대략 20x20센티미터 이상의 면적을 덮으며, 그보다 큰 경우도 많습니다. * 또한 밉 레벨을 쓴다 해도, 애초에 런타임에 어떤 레벨을 참조해야할지도 모호합니다.

* 씬기준 면적입니다. 런타임 화면 기준 아님에 유의





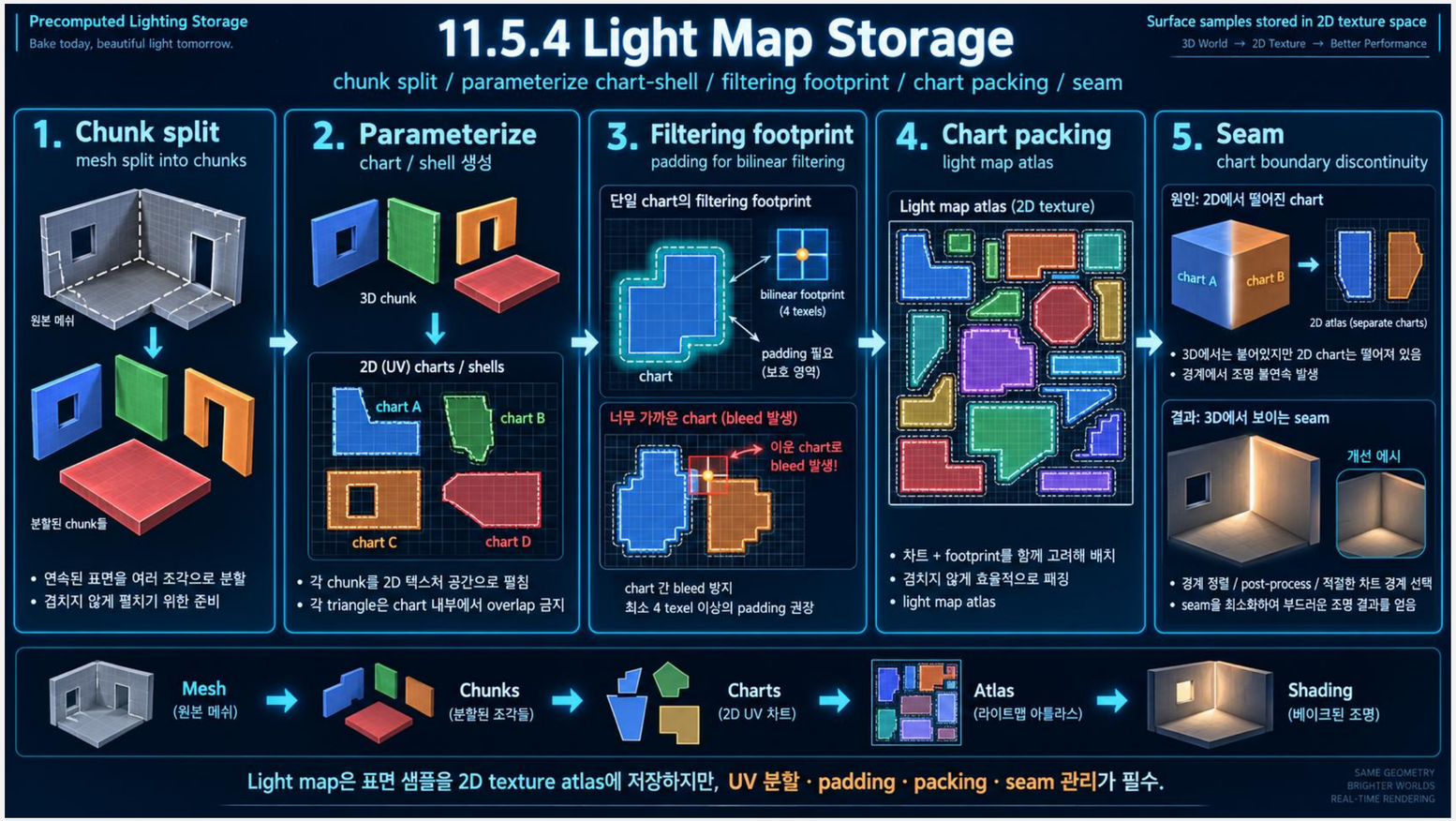
* 미리 구워진 라이트 맵의 예시
* 왼쪽 아래가 바닥면이고 오른쪽에서 2번째, 가장 윗줄이 뒤에 있는 Cube의 정면 방향 Light Map입니다.
* 이전 페이지의 PRT-메모리 절감 파트를 보면 같은 색상의 같은 normal을 가진 면이면 Source Clustering 할 수 있음을 알 수 있음



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

저장 방법(Storage Methods)

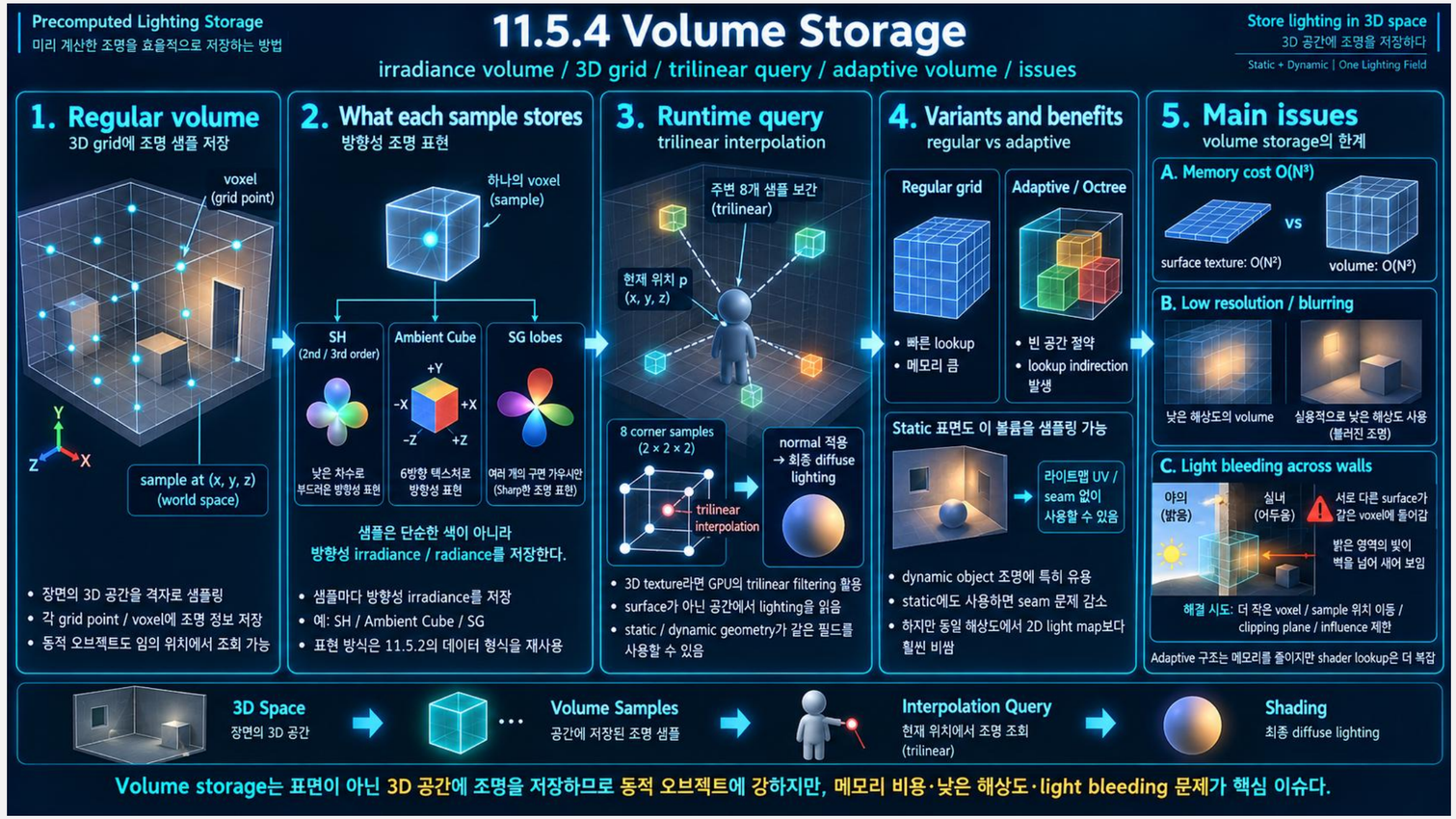
텍스쳐에 조명 정보를 저장하기 위해서는 객체가 고유한 uv 값을 샘플링 해야합니다.
일반 albedo texture는 UV를 겹쳐서 사용할 수 있지만, light map은 직/간접광의 영향으로 surface의 위치마다 고유한 texel영역을 가져야 합니다.
모든 surface의 위치가 고유한 texel영역을 가질 수는 없기 때문에, 최대한 이 Texel영역의 차이에서 발생하는 이슈를 막기 위한 여러가지 고민과 방법론이 존재합니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

저장 방법(Storage Methods)

Mesh Vertex에 이를 저장하는 방법도 있지만, 너무 퀄리티가 낮아 생략하고, surface가 아닌 3D volume에 저장하는 방법을 소개합니다.
3D Volume에 매핑할 경우, Dynamic한 물체들에도 효과적으로 미리 구워둔 조명 값을 전달 할 수 있다는 장점이 생깁니다.
마찬가지로 모든 volume을 고유한 gpu 메모리 위에 올릴 수는 없기 때문에, 이를 고려해서 어떻게 저장하고 샘플링하는지도 나열해보았습니다.



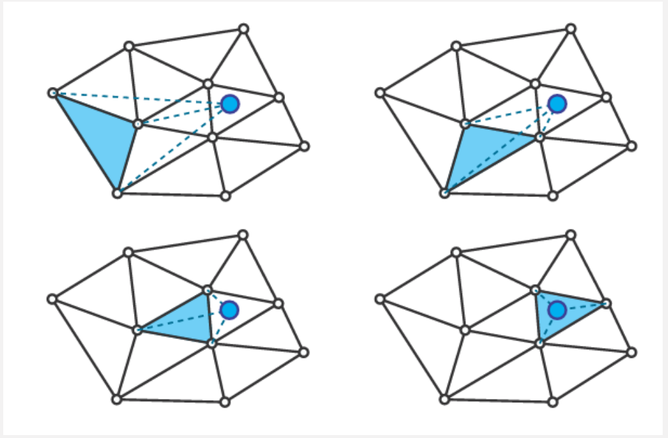
* Light Map만큼 보편적이지는 않지만
* Unreal에서는 Volumetric Light Map이
* Unity에는 Light Probe Proxy Volume (레거시)이 있습니다.

저장 방법(Storage Methods)



순수하게 3D 공간을 Volume Storage에 저장하는 것은 $O(n^3)$의 공간 복잡도를 가지게 됩니다.
런타임에 잘 닿지 않을 위치나, 조명이 천천히 변하는 곳에서 높은 밀도로 저장 할 필요는 없습니다.

이를 더 효과적으로 저장하는 널리 쓰이는 선택지는 불규칙한 vertex group에 저장한뒤, 이 점들을 연결해 들로네 사면체 분할을 구성하는 것입니다.
조명을 조회하기 전에 샘플링 위치가 속한 사면체를 찾는 과정을 거칩니다.



해당 정보는 GPU친화적이지 않기 때문에 GPU 가속을 받기 어렵지만, 3D Volume에서 8개의 지점에서
삼선형 (Trilinear) 보간 하는것보다 적은 값(4개 지점)으로 보간 할 수 있게 됩니다.

조명을 미리 계산해 저장할 위치는 수동으로 배치할수도 있고, 자동으로 배치할 수도 있습니다.

이런 위치들은 조명 신호를 탐침(probe)하듯 샘플링 한다는 뜻에서 라이트 프로브 (Light Probe)라고 불리웁니다.

이런 Light Probe를 자동으로 배치하는 방법론이나, Mesh나 씬의 형태에 따른 여러 휴리스틱들도 논의되었습니다.

저장 방법(Storage Methods)

엔진과 게임마다 처리 방법과 방점은 다르지만

Surface LightMap (Texture)이 가지는 상대적으로 고해상도지만 Dynamic Object에는 적용이 어렵다는 점,
Volume이 가지는 GPU 친화적이면서 Dynamic Object에도 적용할 수 있지만 많은 양의 메모리를 차지하고 효율적으로 밀도를 채우기 어렵다는 점
Light Probe가 가지는 밀도를 원하는대로 커스텀하기 좋지만 GPU 친화적이지 않은 점을 적절히 섞어
런타임에 미리 구워둔 GI 정보를 참조해 계산하도록 하고 있습니다.

* static mesh에는 Light Map을
* dynamic object에는 Light Probe나 Volumes을 사용하는 등...
* 아예 엔진에서 적절한 지점을 제공하는 시스템을 제공하기도

Static 객체와 Dynamic 객체의 특성에 맞춰 서로 다른 storage를 사용하는 경우,
서로 다른 저장 방식의 차이에서 발생하는 해상도나 필터링 방식으로 인해 조명 색과 밝기가 다르게 보일 수 있습니다.

이런 괴리를 줄이기 위해 서로 다른 Storage에 저장된 정보를 다시 평균화 하는 regularization을 거치기도 합니다.